系统分析与设计 · 课程作业

# Foxcore Retail
# 现场零售运营管理系统
# 分析设计文档


| | |
|---|---|
| 班　　级 | 信管 T2401 |
| 姓　　名 | 于董雅池 |
| 学　　号 | 8304240304 |
| 日　　期 | 2026 年 10 月 14 日 |


文档与图纸：github.com/xingchenyd/foxcore-retail

# 目 录

01 愿景与业务分析 4
02 软件需求规格说明书（SRS） 14
03 系统设计说明（SDD） 24
04 数据库设计与 ERD 43
05 软件测试与验收说明（STD） 72
06 用户界面与操作流程手册（UI & Flow） 82
07 本地部署、运维与验收 90
08 · 云端部署（Vercel + TiDB Cloud） 99

> 作业文档与图纸仓库：https://github.com/xingchenyd/foxcore-retail —— 文档位于 `docs/`，图纸位于 docs/diagrams（实体类图 class-diagram.png/svg、物理 ERD physical-erd.png、概念 ERD core-barker.png/svg、架构图、业务序列图、用例图、状态图等全套 .puml/.svg/.png）。

# 01 愿景与业务分析

| 文档属性 | 内容 |
| --- | --- |
| 文档名称 | Foxcore Retail 愿景与业务分析 |
| 当前版本 | 2.0 |
| 编写日期 | 2026-09-27 |
| 文档状态 | 已发布 |
| 关联文档 | 02-SRS 需求规格 · 03-SDD 设计说明 · 04-数据库设计 |

> 来源声明：业务背景取自 Foxcore Retail A 第 2–3 页、B 第 1–3 页。本文为独立教学实现，与案例企业无官方关联；所有样例人物、价格和业绩均为合成数据。

## 1.1 引言

### 1.1.1 编写目的

本文档定义 Foxcore Retail 现场销售管理系统的**项目愿景**与**业务模型**，回答三个问题：当前业务如何运作（AS-IS）、为什么必须改变（问题与影响）、改变后应达到什么状态（TO-BE 愿景与度量）。它是后续 软件需求规格（SRS） 与 软件设计说明（SDD） 的业务依据：SRS 中的每条功能需求都应能回溯到本文档的某个业务问题、业务目标或业务用例。

### 1.1.2 项目背景

Foxcore 是一家从事现场活动销售的零售企业：在展会、市集、社区市集等**七类活动**中设置展位，由销售顾问当班销售商品。业务规模具有三个特征：

1. **人员多**：现场销售顾问 17 名，同一天可能有多名顾问在不同展位当班；
2. **品类多**：在售商品超过 28 种，价格允许现场协商，但不得低于管理员设定的最低售价；
3. **地点散**：同一周末可能参加多个活动，一个活动设置多个展位，销售归属需要精确到”活动 → 展位 → 班次 → 销售员”。

当前的记录方式以纸笔为主、电子表格为辅，难以支撑上述规模的精细化经营。

### 1.1.3 适用读者

| 读者 | 关注点 | 建议阅读 |
| --- | --- | --- |
| 业务负责人 / 管理员（OWNER） | 系统能否解决佣金争议、库存不准、分析困难 | 第 1.2、1.3、1.6 节 |
| 现场销售顾问（SALESPERSON） | 日常操作是否更省事、佣金是否透明 | 第 1.3、1.5 节 |
| 需求/开发人员 | 业务规则与范围边界 | 全文，尤其第 1.5、1.6 节 |
| 课程评审 / 教师 | 建模完整性 | 全文 |

### 1.1.4 参考资料

- Foxcore Retail A 案例原文（第 2–3 页）：现状与数据要求；
- Foxcore Retail B 案例原文（第 1–3 页）：实体与关系要求；
- 02-SRS：来源需求追踪（CASE-01 至 CASE-12）；
- 05-STD：面向本业务目标的验收测试。

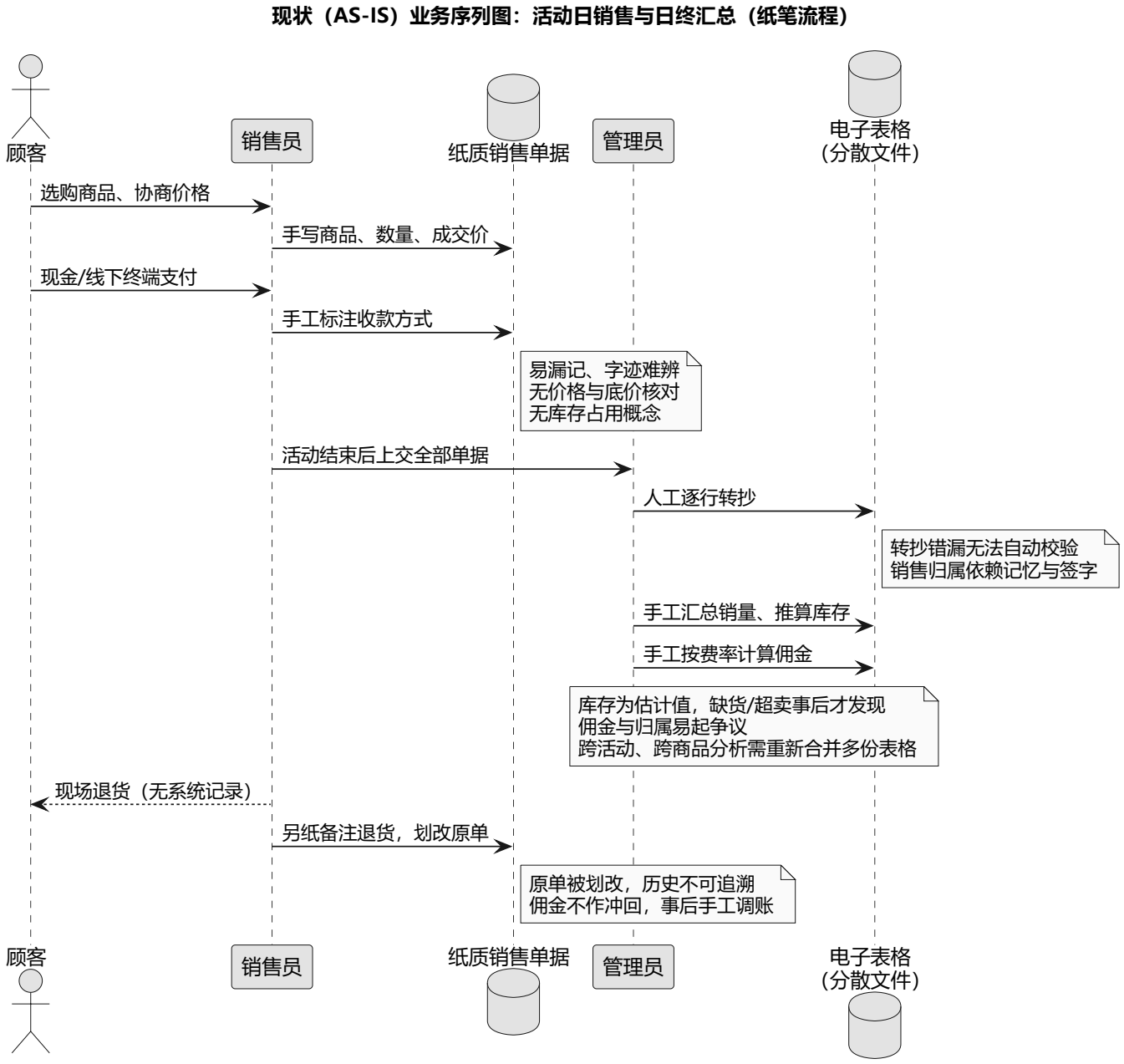
## 1.2 现状分析与问题陈述

### 1.2.1 现状业务流程（AS-IS）

一个典型活动日的现状流程如下（图示见 2.2 现状业务序列图）：

1. 活动前，管理员用表格登记活动、展位和商品清单，为每位销售顾问手工排班；
2. 开场时，销售员从管理员处领取商品实物，**口头/纸质交接**，库存数量以估计为主；
3. 顾客选购时，销售员在纸质单据上**手写**商品、数量和协商后的价格；
4. 顾客以现金或线下终端付款，销售员在单据上标注收款方式；**没有任何系统机制核对成交价与最低售价、可售库存量**；
5. 活动结束，销售员上交全部纸质单据，剩余实物交回；
6. 管理员**人工逐行转抄**单据到电子表格，再手工汇总销量、推算库存、按费率计算佣金；
7. 发生退货时，销售员划改原单、另纸备注；**原单被破坏，退款与佣金不作系统冲回**；
8. 月末/季末，管理员需要**手工合并多份表格**才能回答”哪个活动、哪类商品、哪位顾问表现最好”。

### 1.2.2 现状业务序列图（AS-IS）




**现状业务序列图**

图源：business-sequence-asis.puml

## 1.2.3 问题清单

| 编号 | 问题 | 根因 | 业务影响 |
|---|---|---|---|
| P-01 | 销售漏记、字迹难辨 | 现场忙碌时手工记录 | 佣金与库存估计不可靠，直接损失信任 |
| P-02 | 无价格与库存校验 | 纸单无法实时核对 | 低于底价成交、展位超卖事后才发现 |
| P-03 | 收款归属争议 | 销售归属依赖记忆与签字 | “这单是谁卖的”难以裁决，团队矛盾 |
| P-04 | 转抄错漏 | 人工二次录入 | 汇总数据失真，报表不可信 |

| 编号 | 问题 | 根因 | 业务影响 |
|---|---|---|---|
| P-05 | 库存为估计值 | 收付实物与记录脱节 | 缺货错失销售，积压占用资金 |
| P-06 | 退货破坏历史 | 划改原单 | 无法审计”原价卖了什么、退了多少” |
| P-07 | 跨活动分析困难 | 数据分散多表 | 商品/员工/活动维度对比需数天人工 |
| P-08 | 佣金计算易错 | 手工按费率核算 | 多算/漏算引发结算纠纷 |

### 1.2.4 业务影响汇总

上述问题共同导致三个层面的损失：**收入层**（超卖、缺货、错失销售）、**成本层**（人工汇总与对账耗时、退货纠纷处理）、**管理决策层**（缺乏可信数据支撑活动与人员决策）。改进的核心不是”把纸换成屏幕”，而是**把每一笔成交变成结构化、可追溯、不可篡改的业务事实。**

## 1.3 项目愿景

### 1.3.1 愿景陈述

> 在一个活动密集的周末，Foxcore 的每一位销售顾问都能在数秒内完成一笔合规的成交记录；管理员打开系统即可看到**实时、可信、可追溯**的销售、库存与佣金数据——每一笔成交都能回答：**何人、何时、在何地、以何价格、销售了何商品。**

### 1.3.2 业务目标与成功度量

| 编号 | 业务目标 | 成功度量 | 现状基线 |
|---|---|---|---|
| G-01 | 消除漏记与转抄错漏 | 成交 100% 系统录入，汇总零人工转抄 | 纸单 + 手工转抄 |
| G-02 | 成交全程可追溯 | 每笔成交沿 活动→展位→班次→销售→明细 可定位，含价格/税率/佣金率快照 | 无法追溯 |
| G-03 | 库存账实一致 | 展位可售量实时准确，收款前即占用，过账后扣减 | 事后估计 |
| G-04 | 佣金无争议 | 正/负佣金流水自动计提，结算有据可查 | 手工计算 |
| G-05 | 多维分析即时化 | 按活动/商品/员工维度的报表与导出秒级可得 | 数天人工合并 |
| G-06 | 退货不破坏历史 | 原单永久保留，退货与佣金冲回自动关联 | 划改原单 |

### 1.3.3 系统范围（In Scope）

- 主数据管理：场地、活动（七类）、展位、商品（批发成本 + 最低售价）、销售顾问档案；

- 排班与班次：计划时段、开班/关班、一班一展位；
- 库存管理：主仓收货、库位间调拨、盘点调整、可售量占用；
- 现场销售：草稿 → 准备收款（核价/冻结）→ 登记线下收款 → 成交过账；
- 退货与更正：授权 → 线下退款 → 登记过账、良品/损坏品区分处理；
- 佣金管理：正负流水计提、结算准备、线下支付登记；
- 报表与导出：活动/商品/员工三维度聚合、CSV 导出；
- 账户与审计：角色账户、一次性临时密码、会话与 CSRF 防护、关键操作审计。

### 1.3.4 非目标与约束（Out of Scope）

1. **不接入银行卡网关**：系统只记录线下收/退款的确认事实，不执行资金转账、不保存卡号；
2. **顾客不注册账号**：顾客是线下参与者，不与系统交互；
3. **税率与佣金率由管理员确认**：系统内的示例费率不表示法定税率；
4. **本实现为独立教学版本**：与案例企业无官方关联，数据为合成数据；
5. 教学定位下的取舍：单仓库部署、单一货币（CAD）、单一业务时区（`America/Toronto`），不追求多租户与国际化。

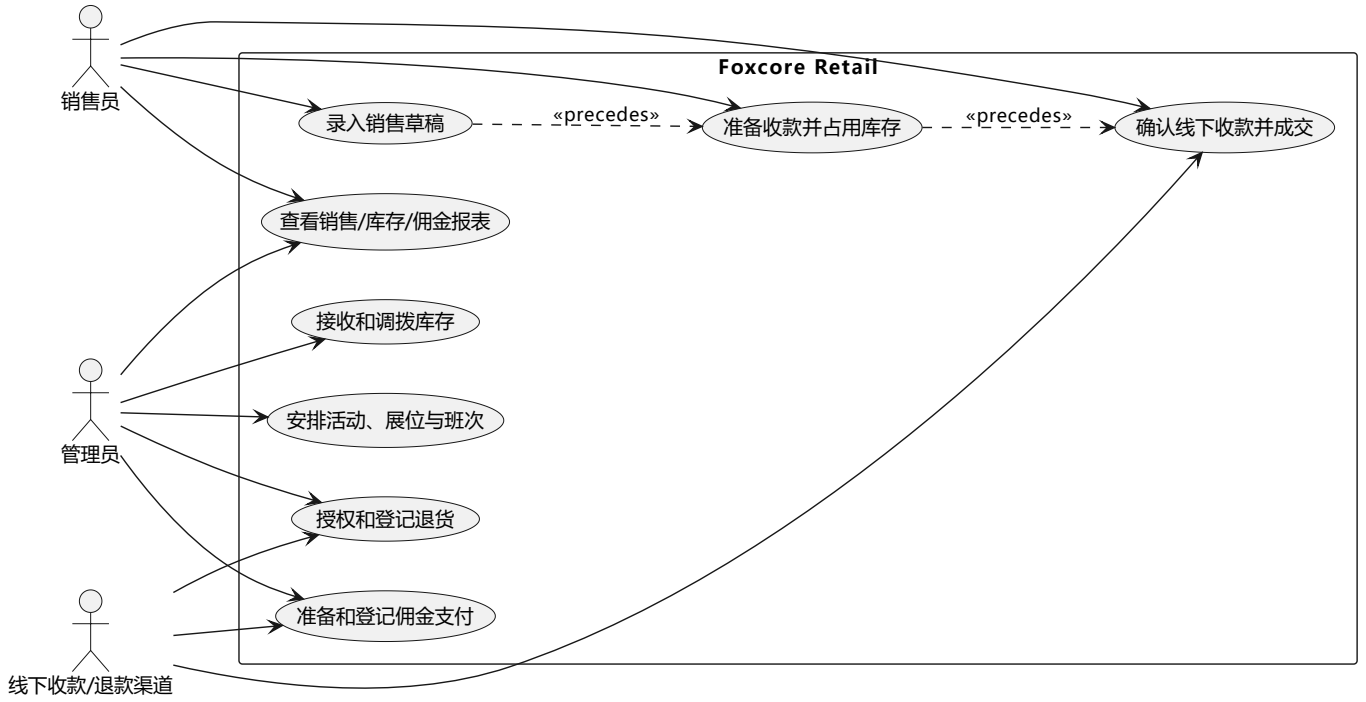
## 1.4 干系人分析

| 干系人 | 在业务中的位置 | 关心的问题 | 系统中的角色 |
| --- | --- | --- | --- |
| 管理员 | 经营者兼现场统筹 | 佣金准确、库存可信、活动盈亏一目了然 | `OWNER`（全权） |
| 销售顾问 | 现场成交执行者 | 录入快、归属清、佣金透明、不被无关数据干扰 | `SALESPERSON`（限本人） |
| 顾客 | 购买者/退货者 | 退款有依据、处理不扯皮 | 不注册，线下参与 |
| 外部收款渠道 | 现金/线下终端 | 不参与系统 | 仅作为线下事实来源 |
| 教师/课程评审 | 教学评价者 | 模型与实现的完整性、可验证性 | 文档读者 |

## 1.5 业务建模

### 1.5.1 业务用例模型

业务用例图如下（图源：business-usecases.puml）：




**业务用例图**

业务用例清单与简要说明：

| 编号 | 业务用例 | 主参与者 | 目标简述 |
| --- | --- | --- | --- |
| BUC-01 | 安排活动、展位与班次 | 管理员 | 活动前完成场地/展位建置与人员排班 |
| BUC-02 | 接收和调拨库存 | 管理员 | 主仓收货并把实物交接至展位，账实一致 |
| BUC-03 | 录入销售草稿 | 销售员 | 把顾客成交意向快速结构化记录 |
| BUC-04 | 准备收款并占用库存 | 销售员 | 收款前核价、冻结金额、锁定可售量 |
| BUC-05 | 确认线下收款并成交 | 销售员 | 登记收款事实，原子生成成交/库存/佣金记录 |
| BUC-06 | 授权和登记退货 | 管理员 | 先授权后退款，原单保留、佣金冲回 |
| BUC-07 | 准备和登记佣金支付 | 管理员 | 占用未结流水、登记线下支付，防止重复结算 |
| BUC-08 | 查看销售/库存/佣金报表 | 管理员、销售员 | 按维度即时聚合，销售员仅见本人范围 |

用例间的依赖关系：销售主线为 `BUC-03 → BUC-04 → BUC-05`（草稿先行、准备收款次之、确认成交最后）；线下收款/退款渠道作为外部参与者出现在收款、退货与佣金支付三个环节。

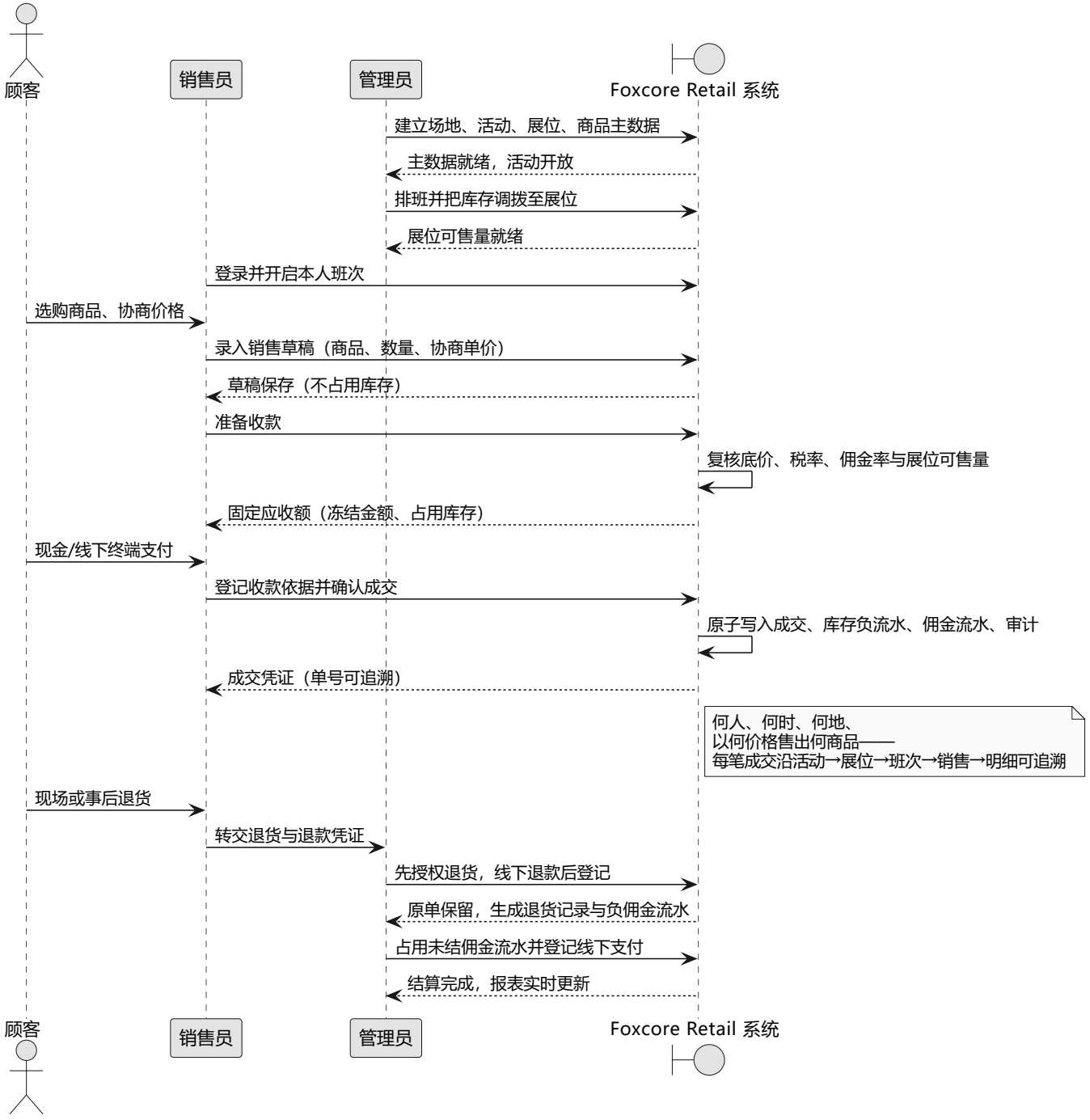
### 1.5.2 改进后业务流程（TO-BE）

改进后的流程（图示见 5.3 改进后业务序列图）：

1. **活动前**：管理员在系统中建立场地、活动、展位、商品与员工档案，确认活动税率与员工佣金率，排班并把库存从主仓调拨至展位——**基础数据一次录入，全程复用**；
2. **开班**：销售员在计划时段内登录系统开启本人班次——班次即销售归属的载体；
3. **成交中**：销售员录入草稿（商品、数量、协商单价）→ 点击准备收款，**系统复核底价、税率、佣金率与展位可售量**，冻结应收金额并占用库存 → 顾客线下支付后，销售员登记现金实收或终端参考号 → **系统在同一事务内**写入成交、库存负流水、佣金流水与审计；
4. **异常路径**：未成交可取消并释放占用；已过账的交易走退货：管理员先取得授权，线下退款后登记，**原单保留**、良品回库、损坏品不回可售、佣金自动冲回；
5. **结算与分析**：管理员按销售员结算佣金（先占用全部未结流水，登记线下支付）；报表按活动/商品/员工维度实时聚合，支持 CSV 导出核对。

### 1.5.3 改进后业务序列图（TO-BE）

**改进后（TO-BE）业务序列图：系统支持的现场销售与结算流程**




**改进后业务序列图**

图源：business-sequence-tobe.puml

## 1.5.4 AS-IS 与 TO-BE 对比

| 维度 | 现状 AS-IS | 改进后 TO-BE |
| --- | --- | --- |
| 记录方式 | 纸笔记录商品、数量和价格；日终人工转抄 | 销售员登录，关联当前班次，录草稿并准备收款 |
| 价格控制 | 无校验，低于底价事后才发现 | 准备收款时服务器重新核价，低于底价拒绝 |

| 维度 | 现状 AS-IS | 改进后 TO-BE |
| --- | --- | --- |
| 库存状态 | 未明确收款前的库存状态，事后估计 | 准备收款冻结金额、占用展位库存；过账后才减少实物 |
| 销售归属 | 依赖记忆与签字，归属易争议 | 活动、班次、人员、商品和佣金流水在同一数据库关联 |
| 退货更正 | 划改原单，历史丢失 | 保留原单，另建退款授权、退货记录和负佣金流水 |
| 佣金核算 | 手工按费率计算 | 正负流水自动计提，结算占用防重复 |
| 多维分析 | 手工合并多份表格，耗时数天 | 服务端实时聚合，秒级可得，可导出 |
| 异常防护 | 无 | 幂等键防重复收款；审计日志记录关键操作 |

### 1.5.5 核心业务协同（文字规约）

1. 管理员建场地、活动、展位、商品和销售员，确认活动税率及员工佣金率；
2. 管理员排班并把实物从主仓交接到展位；销售员在计划时段内开班；
3. 销售员录入商品、数量和协商单价；系统保存草稿。准备收款时服务器重新核价、验库存并冻结应收金额；
4. 销售员在线下收款并登记现金实收或终端参考号。系统原子写入成交、库存负流水、佣金正流水及审计。响应丢失时使用原幂等键或按单号查询，避免二次收款；
5. 管理员先取得退款授权，再操作线下退款，随后登记退货；佣金冲回形成负流水。结佣先占用全部未结流水，再登记线下付款。

> 上述收款、退款和佣金规则是为可运行系统作的 **DEC/EXT** 设计，并非案例原文规定；状态与异常路径见 SRS 和 SDD。

## 1.6 业务规则汇总

| 编号 | 规则 | 来源 |
| --- | --- | --- |
| BR-01 | 每笔销售必须归属一个”活动 + 展位 + 班次 + 销售员”组合，系统按班次推导归属 | CASE-06/07/10 |
| BR-02 | 成交单价不得低于商品最低售价；准备收款时以**当时生效**的底价复核 | CASE-04 |
| BR-03 | 展位可售量 = 在手量 − 占用量，任何时点不得为负 | EXT |
| BR-04 | 收款前必须”准备收款”（冻结金额、占用库存）；未登记线下收款依据不得过账 | EXT |
| BR-05 | 已过账交易不可修改/删除；更正一律走退货授权流程 | EXT |
| BR-06 | 佣金按活动/员工费率以 bps 整数化计提，退货按比例生成负佣金 | DEC-03 |

| 编号 | 规则 | 来源 |
| --- | --- | --- |
| BR-07 | 现金实收不得低于应收；外部终端支付必须登记参考号 | DEC-04 |
| BR-08 | 所有资金、库存、佣金变更操作写审计日志，与业务动作同事务提交 | FR-08 |
| BR-09 | 佣金结算先占用全部未结流水；净额非正不得发起支付登记 | FR-06 |
| BR-10 | 业务日期按 `America/Toronto` 判定，操作时刻以 UTC 存储 | DEC-01 |
| BR-11 | 金额以十进制字符串输入（最多两位小数），库内整数分运算 | DEC-02 |
| BR-12 | 写命令携带 UUID 幂等键：同键同体重试返回同结果，同键异体拒绝 | NFR-02 |

## 1.7 术语表

| 术语 | 英文/系统对应 | 定义 |
| --- | --- | --- |
| 活动 | Event | 在某场地某日期举办的销售活动，分七类 |
| 展位 | Booth | 活动内的销售点位，一个活动至少一个 |
| 班次 | Shift | 销售员在某展位的计划工作时段；一班只在一个展位 |
| 销售草稿 | DRAFT Sale | 已录入未准备收款的交易，可编辑，不占库存 |
| 准备收款 | CHECKOUT | 已核价冻结、占用库存、等待线下收款的交易 |
| 已过账 | POSTED Sale | 已登记收款并生效的成交，不可修改 |
| 占用 | Hold / Reserved | 为待收款交易锁定的库存量 |
| 库存流水 | Stock Movement | 收货/调拨/盘点/销售/退货引起的数量变更记录 |
| 佣金流水 | Commission Line | 按成交/退货计提的正/负佣金记录 |
| 幂等键 | Idempotency Key | 客户端生成的 UUID，用于写命令重试去重 |
| 审计 | Audit Event | 关键业务操作的时间、操作者、对象记录 |

> 作业文档与图纸仓库：https://github.com/xingchenyd/foxcore-retail —— 文档位于 `docs/`，图纸位于 docs/diagrams（实体类图 class-diagram.png/svg、物理 ERD physical-erd.png、概念 ERD core-barker.png/svg、架构图、业务序列图、用例图、状态图等全套 .puml/.svg/.png）。

# 02 软件需求规格说明书（SRS）

| 文档属性 | 内容 |
| --- | --- |
| 文档名称 | Foxcore Retail 软件需求规格说明书 |
| 当前版本 | 2.0 |
| 编写日期 | 2026-09-27 |
| 文档状态 | 已发布 |
| 关联文档 | 01 愿景与业务分析 · 03-SDD 设计说明 · 05-STD 验收测试 · openapi.json |

> 标记约定：CASE 为 Foxcore A/B 原文要求，DERIVED 为原文痛点推导，EXT 为本系统运行补充，DEC 为案例未定的实现规则。原始 PDF 不随软件分发。

## 2.1 引言

### 2.1.1 编写目的

本文档定义 Foxcore Retail 现场销售管理系统的**功能性需求**与**非功能性需求**，作为设计（SDD）、验收（STD）与需求变更控制的基准。每条需求具有唯一编号，可通过 traceability.csv 追踪至案例来源与验证方式。

### 2.1.2 文档范围

覆盖系统的全部对外可见行为：业务主数据、排班、库存、销售、退货、佣金、报表、账户与审计，以及安全、幂等、性能等质量属性。不覆盖内部实现细节（见 SDD）与部署运维（见 07-部署文档、08-云端部署）。

### 2.1.3 预期读者与阅读建议

| 读者 | 阅读建议 |
| --- | --- |
| 开发人员 | 第 2.3–2.5 节（用例模型与具体需求）+ openapi.json 接口合同 |
| 测试人员 | 第 2.4–2.6 节 + 05-STD 验收编号对照 |
| 业务评审 | 第 2.2 节（总体描述）与第 2.5 节（用例规约） |
| 课程教师 | 第 2.1–2.3 节了解建模方法，第 2.5 节核对用例规约深度 |

### 2.1.4 术语定义

业务术语（活动、展位、班次、草稿/准备收款/过账、占用、佣金流水、幂等键等）见 01 愿景与业务分析 · 术语表。

### 2.1.5 参考资料

1. Foxcore Retail A 案例原文（现状与数据要求）；
2. Foxcore Retail B 案例原文（实体与关系要求）；
3. 01-vision-business.md：业务问题、目标与规则；

4. GB/T 8567-2006《计算机软件文档编制规范》与 IEEE Std 830-1998（SRS 结构参考）。

## 2.2 总体描述

### 2.2.1 产品定位与前景

Foxcore Retail 是面向现场活动零售的销售管理系统，替代"纸笔记录 + 日终人工转抄"的现状流程。产品前景与业务目标详见 01 · 3 项目愿景。系统只记录**线下收付款的确认事实**，不接入银行卡网关、不执行资金转账、不保存卡号。

### 2.2.2 用户特征与用户故事

系统用户仅两类角色：管理员（OWNER）与销售顾问（SALESPERSON）。顾客线下参与，不注册账号。

#### Persona 1：管理员（OWNER）——"陈经理"

经营 Foxcore 的负责人，同时承担现场统筹。熟悉商品与人员，但不熟悉软件运维；关心佣金是否算得准、库存是否对得上、每个活动赚了多少钱；讨厌月底对账和下属来争议提成。

| 编号 | 用户故事 | 验收要点 |
| --- | --- | --- |
| US-O1 | 作为管理员，我要建立场地、活动、展位、商品和员工档案，以便活动开展前基础数据就绪 | SKU/展位位置唯一；七类活动可选；税率与佣金率显式确认 |
| US-O2 | 作为管理员，我要排班并把库存调拨到展位，以便开场时账实一致 | 同人时段不重叠；调拨需双方库位余额核对 |
| US-O3 | 作为管理员，我要看到实时可信的销售/库存/佣金报表并可导出，以便随时判断活动盈亏 | 只统计已过账事实；销售员不可见成本毛利 |
| US-O4 | 作为管理员，我要先授权再登记退货与退款，以便原单历史不被破坏、佣金自动冲回 | 同一原单仅一个待授权退款；良品/损坏品区分 |
| US-O5 | 作为管理员，我要先占用后登记佣金支付，以便绝不重复结算 | 净额非正不可结算；取消未付结算释放占用 |
| US-O6 | 作为管理员，我要给员工发账号、停用账号并查看审计，以便掌控权限与追责 | 一次性临时密码，首登强制改密；最后 OWNER 不可停用 |

#### Persona 2：销售顾问（SALESPERSON）——"小李"

周末在展位当班的在校学生兼职。手机操作为主，现场只有几秒空闲；关心录入快不快、这单算不算自己的、这个月佣金有多少；不想看到成本和别人的数据。

| 编号 | 用户故事 | 验收要点 |
| --- | --- | --- |
| US-S1 | 作为销售员，我要在自己班次内快速录入成交（商品、数量、协商价），以便不漏单 | 限本人进行中班次；草稿可编辑不占库存 |
| US-S2 | 作为销售员，我要收款前得到系统核价与锁库存，以便绝不超卖或低于底价成交 | 服务器复核当时底价/税率/佣金率/可售量 |

| 编号 | 用户故事 | 验收要点 |
| --- | --- | --- |
| US-S3 | 作为销售员，我要在收到钱后一键登记收款并拿到凭证，以便顾客等待时间最短 | 原子过账返回单号；幂等键重试不重复收款 |
| US-S4 | 作为销售员，我要随时查看本人佣金明细与销售记录，以便心里有数 | 仅本人范围；不含成本与全局审计 |

### 2.2.3 运行环境

| 项 | 本地部署 | 云端部署（可选） |
| --- | --- | --- |
| 服务器 | Docker Compose（app + MariaDB 11.8） | Vercel Serverless + TiDB Cloud |
| 运行时 | Node.js 24（engines: >=24 <25） | 同左（云端见 08 文档） |
| 数据库 | MariaDB 11.8，UTF8MB4 | MySQL 协议兼容（TiDB） |
| 客户端 | 桌面与手机现代浏览器，与 API 同源 | 同左（HTTPS） |
| 业务时区 | America/Toronto（UTC 存储） | 同左 |

### 2.2.4 设计与实现约束

1. 浏览器端与 API 同源，Cookie 会话 + CSRF 令牌，不引入第三方认证；
2. 金额、税率、佣金率全部整数化（分 / bps）运算，避免浮点误差（DEC-02/03）；
3. 写命令必须携带 UUID 幂等键（NFR-02）；
4. 单货币（CAD）、单业务时区；时间戳以 UTC 保存；
5. 数据库为唯一事实来源，应用无状态、重启不丢交易（NFR-01）。

### 2.2.5 假设与依赖

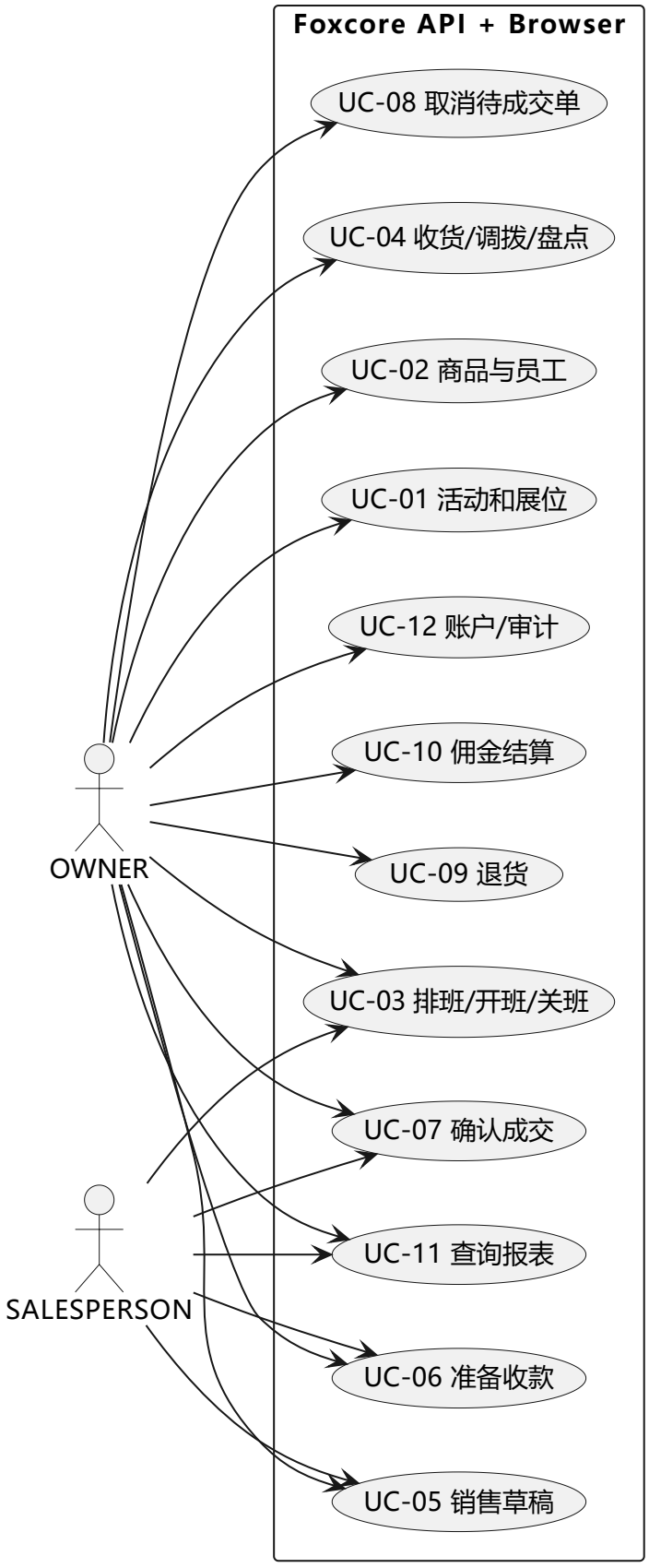
1. 现场存在线下收款渠道（现金或外部终端），系统能拿到的只是人工确认与参考号；
2. 管理员在活动开始前完成主数据与税率/佣金率确认；
3. 实物交接（主仓 → 展位）在调拨登记前已经完成；
4. 教学场景下并发规模有限（≤ 数十用户同时在线），无需读写分离。

## 2.3 系统用例模型

### 2.3.1 参与者定义

| 参与者 | 定义 | 权限范围 |
| --- | --- | --- |
| OWNER | 管理员，系统的全权角色 | 全部业务功能与全局审计 |
| SALESPERSON | 销售顾问，受限角色 | 本人班次内销售全流程 + 本人范围查询 |
| 顾客 | 线下购买者 | 不与系统交互，是系统的**业务对象**而非用户 |
| 外部收款渠道 | 现金/线下终端 | 不与系统交互，其结果由人工登记 |

### 2.3.2 系统用例图




**系统用例图**

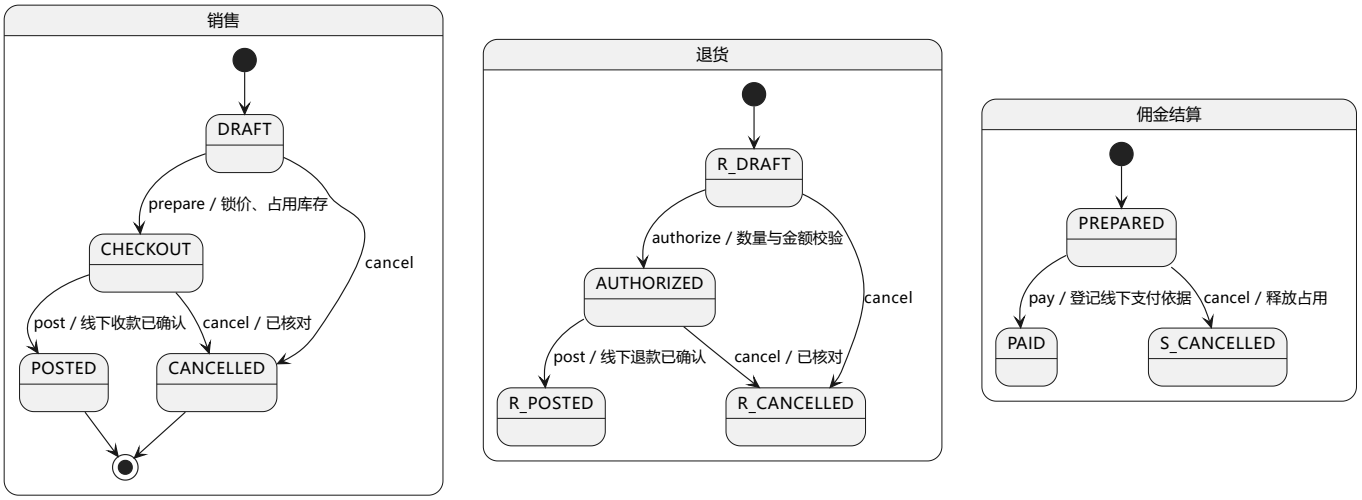
图源：system-usecases.puml

### 2.3.3 用例概览与关系

| 编号 | 用例 | 主参与者 | 次要参与者 | 简述 |
|---|---|---|---|---|
| UC-01 | 管理活动与展位 | OWNER | — | 建场地/活动/展位并开放 |
| UC-02 | 维护商品、员工和账户 | OWNER | — | 主数据建档与账号发放 |
| UC-03 | 排班、开班和关班 | OWNER / SALESPERSON | — | 计划时段与现场开关班 |
| UC-04 | 收货、调拨和盘点 | OWNER | — | 主仓接收与库位间流转 |
| UC-05 | 录入销售草稿 | SALESPERSON / OWNER | — | 结构化记录成交意向 |
| UC-06 | 准备收款 | SALESPERSON / OWNER | — | 核价冻结并占用库存 |
| UC-07 | 确认成交 | SALESPERSON / OWNER | 顾客（线下付款） | 登记收款并原子过账 |
| UC-08 | 取消与核对待成交单 | OWNER / SALESPERSON | — | 释放占用、记录原因 |
| UC-09 | 退货与更正 | OWNER | 顾客（线下退款） | 授权—退款—登记—冲佣 |
| UC-10 | 佣金结算 | OWNER | 员工（线下收款） | 占用流水并登记支付 |
| UC-11 | 查询与导出报表 | 全部登录用户 | — | 三维度聚合与 CSV |
| UC-12 | 账户与审计 | OWNER | — | 账号生命周期与审计查询 |
| UC-13 | 身份认证与会话管理 | 全部账户 | — | 登录、首次改密、退出与会话撤销（EXT） |

用例间顺序依赖：`UC-05 → UC-06 → UC-07` 构成销售主链；`UC-09` 依赖 `UC-07` 已完成；`UC-10` 依赖 UC-07/09 产生的正负流水。

关键对象状态机与销售内部时序：




**销售过账状态**

图源：states.puml · 系统内部时序：sale-sequence.puml

## 2.4 具体需求

### 2.4.1 来源需求追踪

| ID | 要求 | 当前实现位置 | 验证 |
|---|---|---|---|
| CASE-01 | 场地名称、地址、说明单独存储，一个场地有多个活动 | `venues`、`events.venue_id`；活动 API | 迁移 FK；HTTP 流程 |
| CASE-02 | 活动名称、日期、说明、七类活动类型 | `event_categories` 七种子、`events` | 种子查询、日期负例 |
| CASE-03 | 活动至少一个展位，位置属活动 | `booths` 唯一键；活动开放规则 | 无展位开放冲突 |
| CASE-04 | 商品批发成本和最低售价，价格可协商 | `products`、`sale_lines` 快照 | 低价准备拒绝 |
| CASE-05 | 员工姓名、地址与电话 | `salespeople` 字段 | 建档查询 |
| CASE-06 | 员工可有多班，一班只在一展位，记录时段 | `shifts`、班次 API | 重叠校验与开班 |
| CASE-07 | 销售归属当时员工班次并记录时间 | `sales.shift_id`；后端推导员工 | 销售查询与权限 |
| CASE-08 | 商品、数量、最终成交价；一单多明细 | `sale_lines(sale_id,line_no)` | 金额/数量测试 |
| CASE-09 | 按活动、商品、销售员分析 | `/reports/sales` | 销售与退款报表核对 |
| CASE-10 | SALE 同时关联 EVENT 与 SHIFT | 复合外键 `sales(shift_id,event_id)` | 不一致 DB 拒绝 |
| CASE-11 | 主键、外键及 MariaDB 实现 | `db/migrations` | 空库迁移/对账 |
| CASE-12 | 概念实体关系与 Barker 表示 | `docs/diagrams`、数据字典 | 人工图文核对 |

完整行级映射见 traceability.csv。B 的 `PRODUCT_SALE` 原本就存在，此项目保留 `Line_No`，不将其冒称为新增实体。

### 2.4.2 功能需求

- **FR-01 主数据**：OWNER 创建场地、活动、展位、商品、员工；SKU 与展位位置唯一。已过账关联的主数据保留历史，不级联删除。
- **FR-02 排班**：同一员工的计划时段不能重叠，相邻半开区间允许；开班需活动已开放且当前时间在时段内。SALESPERSON 只见自己的班次。
- **FR-03 库存**：主仓接收、确认实物交接后的调拨、按版本盘点调整；`available = on_hand - reserved`，不得为负。
- **FR-04 销售**：草稿可编辑；准备收款复查商品、底价、税率、佣金率及展位可售量，冻结快照并占用库存；成交需登记线下收款，原子扣实物并计提佣金；未成交单可按核对要求取消。
- **FR-05 退货**：仅 OWNER 对原已过账单建立退货，先授权数量和退款额，再凭线下退款依据过账；良品入库，损坏品不回可售。
- **FR-06 佣金**：按销售员正负流水计提；准备结算自动占用所有未结流水，正余额方可支付登记；取消未付款结算释放占用。
- **FR-07 报表**：按活动、商品、员工汇总已过账销售与退货，税前净额、税额、净数量、佣金、标准成本毛利估计；CSV 仅 OWNER 可导出。
- **FR-08 安全与审计**：仅管理员发账户；随机服务端会话、Argon2id、HttpOnly Cookie、CSRF、角色范围；关键命令写审计和幂等结果。

### 2.4.3 业务规则（实现约定）

- **DEC-01** 业务日期按 `America/Toronto`，操作时间以 UTC 保存。计划时段输入须带 UTC 偏移，避免夏令时静默猜测。
- **DEC-02** CAD 金额输入为十进制字符串，最多两位小数；库中整数分，内部 bigint 运算。数量为正整数，一单最多 100 行。
- **DEC-03** 税前行额 = 数量 × 最终单位价；税与佣金分别按 bps 做整数 half-up。底价在准备收款时重新校验。
- **DEC-04** 现金实收不低于应收；外部终端必须有参考号。系统只记录人工确认，不验证银行收单。

### 2.4.4 非功能需求

- **NFR-01** 浏览器端与 API 同源；数据库真实持久化，应用重启不重置交易。
- **NFR-02** 写命令需 UUID 幂等键；同键同体重试得同结果，异体冲突；库存、退款、佣金须在事务内核对。
- **NFR-03** API 禁止泄露密码、会话、SQL 详情。销售员不可查看批发成本、他人销售及全局审计。
- **NFR-04** 界面含加载、空结果、失败、状态与成功反馈，支持桌面与手机完成关键操作。
- **NFR-05 性能**：常规列表与报表查询在数万级流水下应秒级返回（单机 MariaDB）；关键写路径为单次事务，无跨网络二次提交。
- **NFR-06 可审计性**：审计事件由数据库触发与业务事务一并提交，运行账号不可更新/删除审计。

## 2.5 用例规约

以下编号与 STD 验收编号 对应。所有业务写入要求有效会话、CSRF 和 UUID 幂等键；业务动作与审计在同一事务提交。`404` 可用于隐藏非本人资源。

## UC-01 管理活动与展位

- **角色/前置**：OWNER 已登录，已明确活动当地日期及税务口径。
- **人→系统**：录入场地地址与说明→保存 VENUE；选七类 CATEGORY、日期和场地→系统校验并建 DRAFT EVENT；录入展位位置→系统核查同活动位置唯一并建 BOOTH；点击开放→系统复查至少一个有效展位及税率确认。
- **后置/数据**：活动 OPEN，可用于排班；`events.venue_id/category_id` 与 `booths.event_id` 有 FK。无展位或税率未确认保持 DRAFT 并报 409。验证 AT-03。

## UC-02 维护商品、员工和账户

- **角色/前置**：OWNER 已登录，掌握真实 SKU、成本、底价和员工资料。
- **人→系统**：录商品→系统校验 SKU、成本和价格后保存；录员工姓名、地址、电话和佣金率→系统建档；指定员工并发 16 字符以上临时密码→系统创建 SALESPERSON 账号。
- **后置/异常**：重复 SKU、重复账号或无效费率拒绝；员工账号首次登录须改密，已过账交易保留原快照。商品可按版本更新并停用；账户可停用且撤销会话，最后 OWNER 不能停用。验证 AT-03、AT-10。

## UC-03 排班、开班和关班

- **角色/前置**：OWNER 建班；本人销售员或 OWNER 开关班。活动和展位存在。
- **人→系统**：提交员工、展位、带 UTC 偏移的时段→系统换算当地业务日期、排除重叠并存 PLANNED；到时点击开班→系统确认活动 OPEN 与当前时刻在半开时段内；营业后处理草稿/待收款单，点击关班。
- **后置/异常**：一班一展位、员工可有多班；重叠/未开放/未到时或仍有未结销售返回 409，班次状态不变。验证 AT-03、AT-05。

## UC-04 收货、调拨和盘点

- **角色/前置**：OWNER，商品及库位已存在；调拨时实物已交接。
- **人→系统**：输入收货数量与依据→主仓增加；选择来源/目的地并确认交接→系统锁两端余额、核可售量、写两边流水；盘点时输入实际数量、读到的余额版本和原因→系统计算差额并写调整流水。
- **后置/异常**：`on_hand >= reserved >= 0`，余额等于历史流水和；未确认交接、库存不足或旧版本拒绝，整单回滚。验证 AT-04、AT-07、AT-12。

## UC-05 录入销售草稿

- **角色/前置**：OWNER 或班次本人，班次 ACTIVE、活动 OPEN。
- **人→系统**：选班次，添加 1–100 行商品、正整数数量和十进制 CAD 协商价→系统核权限和格式并保存 DRAFT；可用版本更新未准备草稿。
- **后置/异常**：草稿关联原活动与班次，不扣库存；非本人班次隐藏，重复行/无效金额拒绝。验证 AT-05、AT-10。

## UC-06 准备收款

- **角色/前置**：有权访问且版本未变的 DRAFT。

- **人→系统**：点击准备→系统重读活动、班次、商品底价、税率、佣金率和展位可售量；按整数分/half-up 算额，冻结行快照并建立 hold。
- **后置/异常**：状态 CHECKOUT，`reserved` 增加而 `on_hand` 不变；价格不足、库存争用或旧版本返回 409，失败无 hold。验证 AT-05、AT-07。

### UC-07 确认成交

- **角色/前置**：有权访问的 CHECKOUT，线下收款已实际完成，商品尚未重复交付。
- **人→系统**：核对固定应收；现金输入实收额或终端输入外部参考号→系统在同一事务扣在手及占用、记库存负流水、销售时间/单号与正佣金流水，返回成交凭证。
- **后置/异常**：POSTED 不可编辑；现金不足/终端缺参考拒绝。同键同体重试返回原结果，同键异体 409；响应不明先查单或原键重试，不二次收款。验证 AT-05、AT-06。

### UC-08 取消与核对待成交单

- **角色/前置**：本人或 OWNER 可取消 DRAFT；CHECKOUT 仅 OWNER，须确认收款和货物去向。
- **人→系统**：填原因；CHECKOUT 另确认付款已核对且商品未交付或已收回→系统释放 hold 和 reserved，再将单据置 CANCELLED 并审计。
- **后置/异常**：POSTED 不能取消；缺核对确认拒绝且保留占用，须走退货更正。验证 AT-05、AT-06。

### UC-09 退货与更正

- **角色/前置**：OWNER，原单 POSTED，已核对退货行及良品/损坏品去向。
- **人→系统**：建 DRAFT RETURN→系统按原行及累计已退量授权，并按累计比例分摊税/佣金；人工执行退款→登记退款参考后 POSTED，良品入指定库位、损坏品不入可售，并写负佣金。
- **后置/异常**：原销售不删除；同一原单只有一个待授权退款；超原量、重复授权返回 409。付款结果不明先保持 AUTHORIZED 待核对，取消需确认退款未执行/已撤销。验证 AT-08。

### UC-10 佣金结算

- **角色/前置**：OWNER，员工有正的未结净佣金，线下支付未执行。
- **人→系统**：选择员工→系统锁其全部未结正负流水并建 PREPARED，汇总支付额；人工付款后登记外部参考→PAID。若未付款，可输入原因并核对后取消，释放占用。
- **后置/异常**：同一流水只被一个有效结算占用；净额非正/已有 PREPARED/重复支付拒绝。退款后的负佣金保留待下次结算。验证 AT-09、AT-12。

### UC-11 查询与导出报表

- **角色/前置**：已登录；CSV 仅 OWNER。
- **人→系统**：选活动、商品或员工维度→系统从 POSTED 销售和 POSTED 退货聚合净数量、税前额、税额、佣金及标准成本毛利估计；OWNER 可导出 CSV。
- **后置/异常**：销售员查询限定本人，不能读批发成本；CSV 的文本防公式注入。草稿和待收款单不算收入。验证 AT-10、AT-11。

### UC-12 账户与审计

- **角色/前置**：OWNER 管账号和看全局审计；每个登录用户可改自身密码。

- **人→系统**：OWNER 创建员工账号或停用账号→系统撤销旧会话；用户改密→旧会话撤销；管理员查看带时间、操作者及对象的关键事件。
- **后置/异常**：最后 OWNER 不可停用；密码只存 Argon2id 散列，Cookie 只含随机会话令牌。审计不可由运行账号更新/删除。验证 AT-02、AT-10。

## 2.6 权限矩阵

| 能力 | OWNER | SALESPERSON |
| --- | --- | --- |
| 场地/活动/展位/商品/员工/账户写入 | 允许 | 拒绝 |
| 排班创建；开关班 | 创建和所有班次 | 仅本人开关班 |
| 库存接收/调拨/盘点及流水 | 允许 | 拒绝；只读本人活动展位可售量 |
| 草稿、准备、确认成交 | 所有班次 | 仅本人进行中班次 |
| 取消 DRAFT；取消 CHECKOUT | 全部 | 本人 DRAFT；CHECKOUT 拒绝 |
| 退货/结佣登记 | 允许 | 拒绝；可读本人佣金 |
| 报表/CSV/审计 | 全局/允许/允许 | 本人范围/拒绝/拒绝 |

## 2.7 需求追踪与验收

- 需求 ↔ 案例 ↔ 实现行级映射：traceability.csv；
- 验收用例编号（AT-xx）与执行方法：05-STD.md；
- 接口合同（每个用例对应的 HTTP 端点与 Schema）：openapi.json；
- 状态机与异常路径：states.puml、SDD。

> 作业文档与图纸仓库：https://github.com/xingchenyd/foxcore-retail —— 文档位于 `docs/`，图纸位于 docs/diagrams（实体类图 class-diagram.png/svg、物理 ERD physical-erd.png、概念 ERD core-barker.png/svg、架构图、业务序列图、用例图、状态图等全套 .puml/.svg/.png）。

# 03 系统设计说明（SDD）

**Foxcore Retail — 现场零售运营管理系统 · 系统设计文档**

## 3.1 引言

### 3.1.1 目的

本文档描述 Foxcore Retail 系统的**软件设计**，将 SRS 中定义的功能需求（FR）与非功能需求（NFR）转化为可实现、可验证的技术设计。本文档回答三个问题：

1. 系统被分解成哪些构件，各构件的职责边界是什么；
2. 这些构件如何协作完成一次关键业务交易（尤其是过账销售这一强一致性场景）；
3. 为什么做出这些技术选择，代价是什么。

本文档与 01 愿景与业务建模（为什么做）、02 SRS（做什么）共同构成需求—设计的完整链条，并向下游的 04 数据库设计、05 测试说明、06 操作手册、07 部署运维 提供设计基线。

### 3.1.2 范围

**本文档覆盖的设计范围：**

- 应用的分层结构与模块划分（8 个服务端模块 + 单页前端）；
- 4+1 架构视图：用例视图、逻辑视图、交互视图、进程视图、物理/部署视图；
- 关键事务的并发控制、幂等性与审计策略；
- 金额计算、快照与舍入规则的服务层实现约定；
- 安全设计：认证、会话、CSRF、角色授权与响应头；
- 数据设计的总体原则（物理细节见 04）；
- 界面设计的导航结构与交互原则（细节见 06）；
- 架构决策记录（ADR）及其理由与代价。

**本文档不覆盖的内容：**

- 业务愿景、业务流程现状与改进分析（见 01）；
- 用例规约、功能需求编号与验收准则（见 02）；
- 完整的表结构、数据字典、归一化推导与 DDL（见 04）；
- 测试用例步骤、实测结果与缺陷记录（见 05）；
- 逐步操作截图与业务人员作业指导（见 06）；
- 安装命令、备份恢复与故障处置流程（见 07、08 云端部署）。

### 3.1.3 定义、缩写与缩略语

| 术语/缩写 | 全称或定义 |
| --- | --- |
| **Fastify** | Node.js 高性能 Web 框架，本项目服务端基础框架（v5） |

| 术语/缩写 | 全称或定义 |
| --- | --- |
| **SPA** | Single Page Application，单页应用；本项目前端为 React 单页应用 |
| **BFF** | Backend for Frontend；本项目同源 Fastify 同时承载 API 与静态资源 |
| **Argon2id** | 现代密码散列算法，本项目用于口令存储 |
| **CSRF** | Cross-Site Request Forgery，跨站请求伪造 |
| **CSP** | Content Security Policy，内容安全策略 |
| **bps** | basis point，万分之一；税率与佣金率的存储单位（如 1300 bps = 13%） |
| **cents** | 金额最小单位（分）；所有金额在服务端以 `bigint` 整数分存储与计算 |
| **half-up** | 四舍五入（.5 向上进位）的舍入模式，本项目税费与佣金计算采用 |
| **Idempotency-Key** | 幂等键；业务写请求必须携带的 UUID 请求头，用于重试去重 |
| **READ COMMITTED** | MariaDB/InnoDB 事务隔离级别之一，本项目事务默认级别 |
| **FOR UPDATE** | InnoDB 行级排他锁定语法，用于关键行读取加锁 |
| **不变量（Invariant）** | 任何时刻都必须成立的数据一致性断言，由 `db/check-invariants.sql` 核查 |
| **OWNER / SALESPERSON** | 系统两种角色：业主管理员 / 现场销售员 |
| **4+1 视图** | Kruchten 提出的架构描述模型：用例、逻辑、进程、物理、开发视图 |
| **ADR** | Architecture Decision Record，架构决策记录 |
| **模块化单体** | Modular Monolith：单一可部署进程，内部按领域模块严格分层与隔离 |

## 3.1.4 参考文献

| 编号 | 文献 | 用途 |
| --- | --- | --- |
| REF-01 | Foxcore Retail A — Designing a Database（课程案例） | 概念数据模型基线 |
| REF-02 | Foxcore Retail B — Database Implementation（课程案例） | 九核心实体与物理实现基线 |
| REF-03 | IEEE Std 1016-2009, Standard for Information Technology — Systems Design — Software Design Descriptions | 本文档结构依据 |
| REF-04 | P. Kruchten, The 4+1 View Model of Architecture, IEEE Software, 1995 | 第 3.4 节架构视图模型 |

| 编号 | 文献 | 用途 |
| --- | --- | --- |
| REF-05 | 02 软件需求规格 | 需求基线（FR/NFR/UC） |
| REF-06 | OpenAPI 合同 | API 接口契约（45 个路径） |
| REF-07 | MariaDB 11.8 Reference Manual | 事务、锁与 DDL 隐式提交行为 |
| REF-08 | 08 Vercel 云端部署 | 云端形态的部署视图补充 |

### 3.1.5 概述

本文档第 3.2 节描述各系统特性的设计（构件职责、事务边界、错误处理）；第 3.3 节说明架构风格与决策理由；第 3.4 节以 4+1 视图给出系统全貌；第 3.5、3.6 节分别索引数据设计与界面设计的详细文档；第 3.7 节给出需求—设计—测试的三向追踪矩阵；附录 A 汇总 12 项架构决策。

设计的核心命题是：**在一个单机可部署的模块化单体内，用关系数据库的事务与约束，保证现场零售场景下”库存不超卖、金额不错算、操作可追溯”这三条底线。**所有设计取舍均围绕该命题展开。

---

## 3.2 特定需求

本章将 SRS 的功能需求映射为系统特性（System Feature, SF），逐项给出设计要点。

### 3.2.1 用户特征

| 角色 | 技术特征 | 使用环境 | 设计应对 |
| --- | --- | --- | --- |
| **OWNER（业主管理员）** | 具备基本 Web 操作能力；需管理主数据、库存、退货授权、佣金结算与审计 | 后台办公电脑（桌面浏览器）为主，偶用手机查看报表 | 全量导航可见；管理类接口强制角色校验；报表支持 CSV 导出；首登录强制改密 |
| **SALESPERSON（现场销售员）** | 仅需在班次内快速录单成交；对商品价格与成本不敏感 | 展会现场，桌面或手机浏览器，网络可能不稳定 | 导航裁剪（不可见退货、审计）；默认进入销售工作台；按钮尺寸适配移动端；写操作强制幂等键以免网络重试造成重复过账 |
| **运维/部署人员** | 熟悉 Docker Compose 与命令行 | 服务器或本地开发机 | 一键 `docker compose up`；密钥由 Secret 卷注入；健康检查端点 `/health/ready` |

### 3.2.2 系统特性总览

| 编号 | 系统特性 | 实现构件 | 对应需求 | 持久化主要表 |
| --- | --- | --- | --- | --- |
| SF-01 | 身份认证与会话管理 | `auth.ts` | UC-13（认证扩展） | `users`, `sessions`, `idempotency_keys`, `audit_events` |

| 编号 | 系统特性 | 实现构件 | 对应需求 | 持久化主要表 |
|---|---|---|---|---|
| SF-02 | 主数据管理（场地/活动/展位/商品/员工/班次） | catalog.ts, catalog-updates.ts | UC-01, UC-02, UC-03 | venues, events, booths, products, salespeople, shifts, event_categories |
| SF-03 | 库存与库内作业（收货/调拨/盘点/占用） | inventory.ts | UC-04 | stock_locations, stock_documents, stock_document_lines, stock_balances, stock_ledger, stock_holds |
| SF-04 | 销售交易（草稿/准备收款/过账/取消） | sales.ts | UC-05, UC-06, UC-07, UC-08 | sales, sale_lines, stock_holds, commission_entries |
| SF-05 | 退货与退款 | returns.ts, return-target.ts | UC-09 | returns, return_lines, commission_entries |
| SF-06 | 佣金计提与结算 | commissions.ts | UC-10 | commission_entries, commission_settlements, commission_settlement_lines, commission_claims |
| SF-07 | 报表、审计与导出 | reports.ts（报表）、index.ts（审计查询）、core.ts（审计写入） | UC-11, UC-12 | 只读查询 + audit_events |
| SF-08 | 平台服务（事务、金额、幂等、审计、错误） | db.ts, amounts.ts, core.ts | NFR-PERF, NFR-REL, NFR-SEC | idempotency_keys, audit_events |

## 3.2.3 系统特性设计

### 3.2.3.1 SF-01 身份认证与会话管理

#### 设计要点

- 口令存储使用 **Argon2id**；不以任何形式保存明文或可逆加密口令。
- 登录成功后生成 **32 字节**密码学安全随机令牌；数据库仅保存该令牌的 **SHA-256 摘要**（`token_hash CHAR(64)`），即使库被读取也无法反推会话令牌。
- 会话 Cookie 属性：`HttpOnly`（脚本不可读）、`SameSite=Lax`、`Path=/`；本地 HTTP 下 `Secure=false`，迁移到 HTTPS 时必须配置 `COOKIE_SECURE=true` 且 `PUBLIC_ORIGIN` 与实际同源地址一致。
- 会话有效期：**绝对过期 12 小时**，**空闲过期 30 分钟**；修改口令、停用账号时撤销该用户全部现存会话。
- 写请求执行 **CSRF 双重提交校验**，并核对 `Origin` 头在允许集合内。
- 登录失败计数与锁定：`failed_logins` 递增，`locked_until` 到期自动解锁，抵御在线口令爆破。

**事务与错误处理**

认证写操作（登录、登出、改密）在单连接事务内完成；失败的登录不泄露”用户名存在/口令错误”的区别，统一返回 401 与通用消息，避免账号枚举。

### 3.2.3.2 SF-02 主数据管理

**设计要点**

- 六类主数据各自提供列表查询与创建接口；商品支持 PATCH 局部更新（价格/成本策略调整），其余主数据一经被业务引用即不提供删除，避免历史单据悬空引用。
- **复合外键链**保证归属不可篡改：`booths UNIQUE(id, event_id)` → `shifts(booth_id, event_id)` → `sales(shift_id, event_id)`。这使得”把一笔销售强行挂到别的活动下”在数据库层面即被拒绝。
- 活动开放前置校验：至少存在一个有效展位，否则开放请求返回 409（业务规则 BR-05）。
- 七类活动类别（trade show / rib fest / waterfront festival / music festival / beer festival / sporting event / street festival）作为**种子数据**写入 `event_categories`，不提供用户维护入口。

**事务与错误处理**

主数据创建为短事务；唯一键冲突（如 SKU 重复、同一活动下展位位置重复）映射为 409 并给出可定位的字段信息。

### 3.2.3.3 SF-03 库存与库内作业

**设计要点**

- **两类库位**：`WAREHOUSE`（主仓，不关联展位）与 `BOOTH`（展位库位，必关联展位），由 `CHECK (kind='WAREHOUSE' AND booth_id IS NULL) OR (kind='BOOTH' AND booth_id IS NOT NULL)` 强制。
- **余额与流水分离**：`stock_balances` 是**可重算的缓存**（`on_hand, reserved`），`stock_ledger` 是**只增不改的权威流水**（`delta` 带符号，`CHECK (delta <> 0)`）。二者由不变量 I-02 约束一致。
- **可售量定义**：`available = on_hand − reserved`；`CHECK (on_hand >= reserved)` 保证可售量非负。
- **单据驱动**：收货（RECEIVE）、调拨（TRANSFER）、盘点（ADJUST）、退货（RETURN）四类库存单据均需理由文本；调拨按当前实现为**已完成实物交接后的原子登记**：请求必须包含 `handoverConfirmed=true`，同一事务扣来源余额、加目标余额并写双向流水；不存在在途库存和二次接收接口。
- **占用（hold）**：销售准备收款阶段创建 `stock_holds`，状态 `ACTIVE`；过账时转 `CONSUMED` 并使 `on_hand` 与 `reserved` 同量下降；取消时转 `RELEASED` 并归还 `reserved`。

**事务与错误处理**

库存单据写入与余额更新在同一事务内；`stock_balances.version` 配合乐观校验，`stock_ledger` 的 `UNIQUE (source_kind, source_id, source_line_no, location_id)` 保证同一来源不重复入账。

### 3.2.3.4 SF-04 销售交易（核心特性）

**状态机**：`DRAFT → CHECKOUT → POSTED`，任一前序状态可 `CANCELLED`。

| 转移 | 触发接口 | 关键动作 |
| --- | --- | --- |
| → DRAFT | `POST /api/v1/sales` | 校验班次 ACTIVE、活动 OPEN，落草稿行 |

| 转移 | 触发接口 | 关键动作 |
| --- | --- | --- |
| DRAFT → CHECKOUT | `POST /api/v1/sales/{id}/prepare` | 重算底价/税率/佣金率/可售量，创建 ACTIVE hold（仅增 `reserved`，不动 `on_hand`） |
| CHECKOUT → POSTED | `POST /api/v1/sales/{id}/post` | 记录线下收款确认，`on_hand` 与 `reserved` 同量下降，写负库存流水、正佣金流水、成交时间与审计 |
| → CANCELLED | `POST /api/v1/sales/{id}/cancel` | 释放或冲销 hold；已 POSTED 单据不可取消，须走退货 |

**准备收款事务的加锁顺序（严格固定，防止死锁）**

```
1. idempotency_keys（幂等行，最先）

2. events（校验活动 OPEN）

3. shifts → sales（校验班次与单据状态）

4. products（重读当前底价与成本）

5. stock_balances（展位库位余额，最后）
```

事务内**重新读取**当前底价、税率、佣金率、活动/班次状态与可售量——不使用请求体携带的价格作为成交依据。只增加 `reserved`，保留 `on_hand`，因此”待收款”期间的实物仍在展位，符合现场作业语义（ADR-04）。

**快照机制**：每条 `sale_lines` 保存 `final_unit_price_cents`、`name_snapshot`、`sku_snapshot`、`cost_snapshot_cents`、`min_snapshot_cents`、`tax_bps_snapshot`、`commission_bps_snapshot`。已过账历史不受后续主数据变更影响（ADR-03）。

**外部收款边界**：系统仅登记”已收到款项”的确认信息（现金实收额或终端参考号），**不发起、不对接任何银行卡或线上支付渠道**；收款动作不在数据库事务内。

### 3.2.3.5 SF-05 退货与退款

**状态机**：`DRAFT → AUTHORIZED → POSTED`，可 `CANCELLED`。

- 退货**锁定原销售单及其明细行**，按累计已退数量分摊原单的税额与佣金，生成**负向**佣金流水；
- 选择 `RESTOCK`（良品）时按指定目标库位加回可售库存，`DAMAGED`（损坏）不入可售；
- 累计退货数量超过原销量时拒绝（409），由不变量 I-05 复核；
- 退款参考号由管理员在线下完成退款后登记，系统不触发资金动作。

### 3.2.3.6 SF-06 佣金计提与结算

- **计提**：过账销售时按行的 `commission_bps_snapshot` 生成正向 `commission_entries`；过账退货时按分摊比例生成负向条目。`UNIQUE (source_kind, source_id, source_line_no)` 保证同一来源只计提一次。
- **结算**：`PREPARED → PAID`，可 `CANCELLED`。准备结算时**锁定员工并占用其当前全部未结正负流水**（写入 `commission_claims`，一笔流水只被一个结算占用）；取消准备释放占用；登记支付仅保存付款参考，不执行转账。
- 结算金额与明细金额合计由不变量 I-07 复核。

#### 3.2.3.7 SF-07 报表、审计与导出

- 报表按**活动 / 商品 / 销售员**三个维度汇总，**只计已过账销售并扣除已过账退货**；
- 维度白名单校验 + **参数化 SQL**；动态报表不允许拼接任意列名；
- CSV 导出限 OWNER；文本字段做**公式注入防护**（前置单引号或转义 `= + - @` 起始字符）；
- 毛利为**商品标准成本估算**（`final_unit_price − cost_snapshot`），不含活动费、差旅与工资，且销售员角色不可见（SRS NFR-SEC-03）；
- 关键写操作写入 `audit_events`（actor、action、subject、detail_json），管理员可分页查询。

#### 3.2.3.8 SF-08 平台服务

| 服务 | 设计约定 |
| --- | --- |
| **金额** (`amounts.ts`) | API 输入为 CAD 十进制字符串，内部转整数分 `bigint` 运算；税与佣金按 bps 计算并 **half-up** 取整；输出金额字段为整数分字符串；浏览器仅做格式化显示 |
| **事务** (`db.ts`) | 全程**同一连接**；隔离级别 `READ COMMITTED`；关键行 `FOR UPDATE`；捕获 `ER_LOCK_DEADLOCK`（errno 1213）、`ER_LOCK_WAIT_TIMEOUT`（1205）、`ER_CHECKREAD`（1020）三类错误，回滚后随机退避 20–70 ms，**最多 3 次尝试**（首次 + 最多 2 次重试） |
| **幂等** (`core.ts`) | 任意业务 POST 必须携带 `Idempotency-Key`（UUID）；键、请求摘要与响应在**同一事务**提交；相同键相同内容返回首次响应（不重复过账），相同键不同内容返回 409 |
| **审计** | 与业务数据同事务写入，保证"有业务必有审计" |
| **错误** | 统一 `{code, message, requestId}`；已识别输入/状态/权限错误映射 400/401/403/404/409；500 **不回显 SQL 栈与内部消息** |

### 3.2.4 性能与其他非功能需求的设计应对

| SRS 编号 | 需求 | 设计应对 | 验证方式 |
| --- | --- | --- | --- |
| NFR-PERF-01 | 现场录单响应可接受 | 单进程内存态路由；金额计算纯整数运算；列表默认分页 20 条；选择器最多载入 100 项 | STD TC-014 |
| NFR-PERF-02 | 并发抢最后一件不超卖 | 固定加锁顺序 + `FOR UPDATE` + 重试；不足量直接 409 | STD TC-007（20 并发 1 成功 19 冲突） |
| NFR-REL-01 | 重试不产生重复过账 | 幂等键与业务同事务提交 | STD TC-006 |
| NFR-SEC-01 | 口令不可反推 | Argon2id + 令牌仅存 SHA-256 摘要 | STD TC-002、代码审查 |
| NFR-SEC-02 | 越权访问被拒 | 角色中间件 + 销售员查询按 `salesperson_id` 限定 | STD TC-010 |
| NFR-SEC-03 | 成本不泄露给销售员 | 序列化层按角色裁剪字段 | STD TC-010 |
| NFR-AUD-01 | 关键操作可追溯 | 同事务审计写入 | STD TC-013 |

| SRS 编号 | 需求 | 设计应对 | 验证方式 |
| --- | --- | --- | --- |
| NFR-MAIN-01 | 结构清晰可维护 | 模块化单体，模块间仅经服务层调用 | 类型检查 + 构建 |

### 3.2.5 设计约束

| 编号 | 约束 | 来源 | 影响 |
| --- | --- | --- | --- |
| DC-01 | 数据库为 MariaDB 11.8（InnoDB） | 课程案例 B | `FOR UPDATE`、CHECK、复合 FK 可用；DDL 隐式提交不可整体回滚 |
| DC-02 | 运行时 Node ≥ 24 | 项目 `.node-version` | 使用现代内置 API；Vercel 需指定运行时 |
| DC-03 | 单一可部署单元优先 | 教学与部署简易性 | 采用模块化单体，非微服务 |
| DC-04 | 本地一键启动不得手工配置口令 | 部署简易性要求 | 密钥走 Docker Secret 卷 / 环境变量 |
| DC-05 | 发布包不含课程原件与真实凭据 | 信息安全要求 | 构建与打包脚本排除敏感文件 |
| DC-06 | 单时区 `America/Toronto` | 业务现场单一 | 不做跨时区门店支持 |
| DC-07 | 不得改动既有本地部署流程 | 用户约束 | 云端适配全部以环境变量门控（见 08） |

## 3.3 架构原理

### 3.3.1 架构风格

系统采用 **模块化单体（Modular Monolith）**，具体形态为：**React SPA 前端 + 同源 Fastify API + MariaDB**，单进程、单端口（3000）对外服务。

```
浏览器 ──HTTP──> Fastify 单进程（同源）
                  ├─ /api/v1/*   JSON API（业务逻辑）
                  ├─ /*          静态资源（生产构建的 SPA）
                  └─ /health/*   健康检查
                        │
                        └─ mysql2 连接池 ──> MariaDB 11.8（仅容器内网可达）
```

### 3.3.2 决策理由

| 备选方案 | 评价 | 结论 |
| --- | --- | --- |
| **模块化单体 + 同源静态托管** | 一次构建、一次部署；会话边界天然一致（无跨域 Cookie 问题）；事务在单进程内可控 | **采纳** |
| 前后端分离部署（SPA 独立域名 + 独立 API） | 需处理 CORS、跨站 Cookie、双份部署与版本对齐；对单组织现场系统收益有限 | 否决 |
| 微服务拆分（库存/销售/佣金各自服务） | 引入分布式事务与最终一致；本系统核心诉求恰是**强一致的库存与金额**，拆分会使复杂度与风险陡增 | 否决 |
| 嵌入式数据库（SQLite） | 无并发写事务能力，不满足”20 并发抢最后一件”的强一致要求 | 否决 |

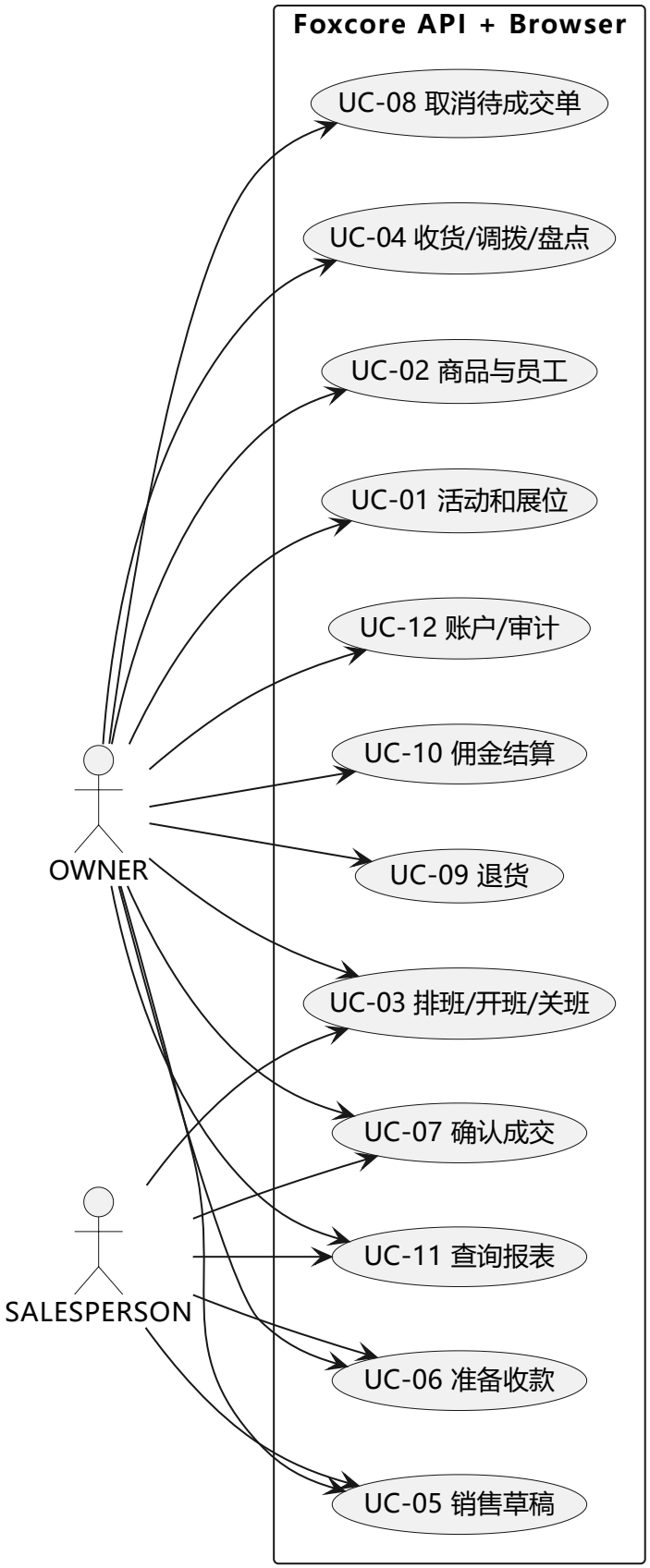
### 3.3.3 关键质量属性策略

| 质量属性 | 策略 | 具体机制 |
| --- | --- | --- |
| **一致性** | 把正确性放进数据库 | 事务 + 行锁 + CHECK + 复合外键 + 唯一键 + 不变量核查脚本 |
| **可靠性（重试安全）** | 幂等优先 | `Idempotency-Key` 与业务同事务；来源唯一键防重复入账 |
| **可审计性** | 写即记录 | `audit_events` 同事务；不可 UPDATE/DELETE 的流水表 |
| **安全性** | 纵深防御 | Argon2id、摘要令牌、CSRF、角色裁剪、CSP、no-store、参数化 SQL |
| **可移植性** | 环境无关 | 通用 Dockerfile + 环境变量；云端形态以 `VERCEL` 等变量门控，不改动本地路径 |
| **可维护性** | 显式边界 | 模块只经服务层交互；共享逻辑集中在 `core.ts` / `amounts.ts` |

## 3.4 架构视图（4+1）

### 3.4.1 用例视图（Use Case View）

描述系统为角色提供的能力边界。完整用例规约见 02 SRS；业务用例图见 业务用例图，系统用例图见 系统用例图。



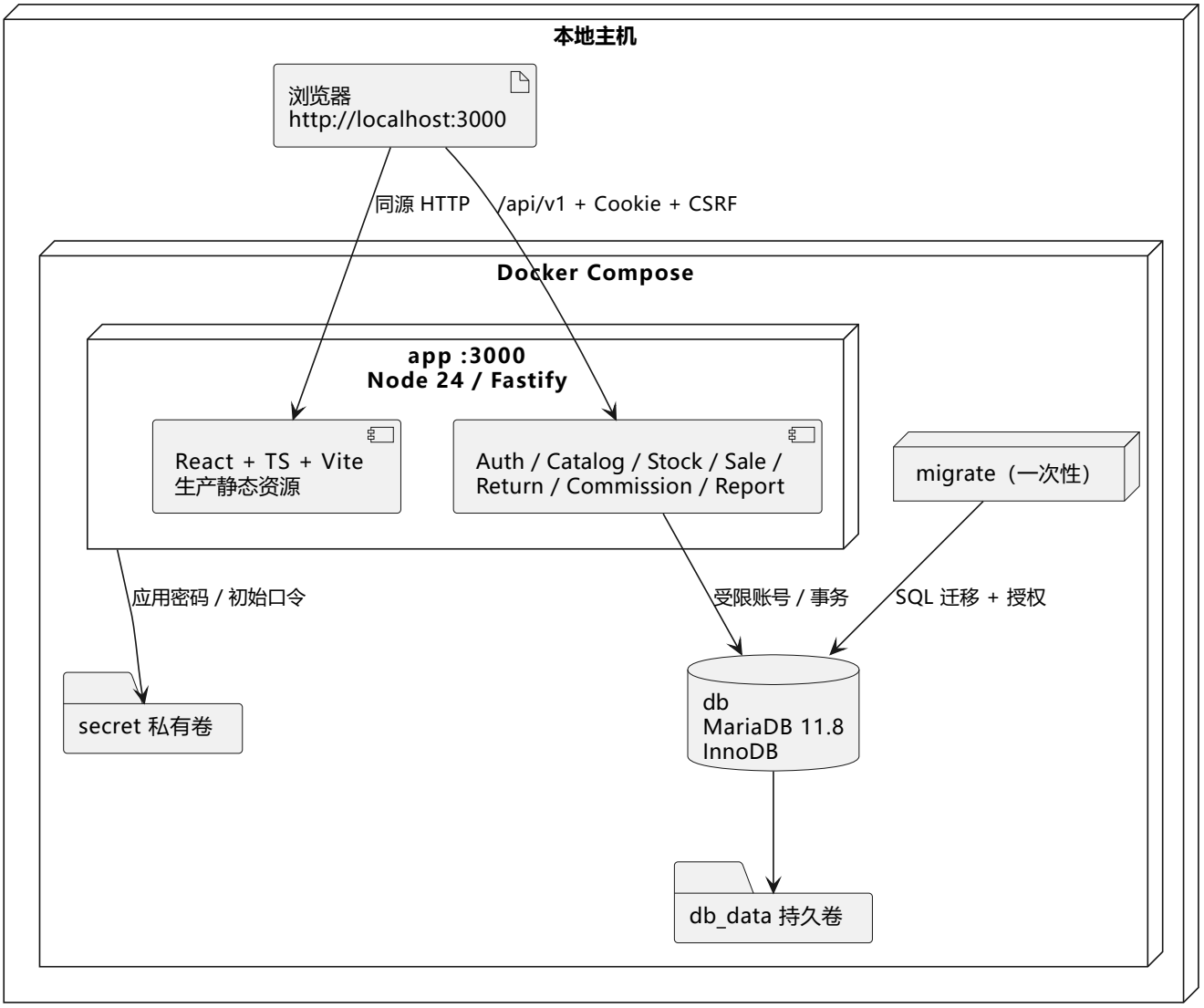

**系统用例图**

**图 3-1 系统用例图**（源：`diagrams/system-usecases.puml`）

| 角色 | 可见能力 |
| --- | --- |
| OWNER | 全部 12 个用例，含主数据维护、库存作业、退货授权、佣金结算、报表导出、审计查询 |
| SALESPERSON | 登录、主数据只读、班次内销售（UC-08/UC-09）、本人佣金查询（UC-11）、报表查看（UC-12，不含成本口径） |

### 3.4.2 逻辑视图（Logical View）

按职责划分的构件与依赖方向（依赖自上而下，无环）。




**架构图**

**图 3-2 逻辑架构与部署结构**（源：`diagrams/architecture.puml`）

| 层 | 构件 | 职责 | 关键持久化 |
| --- | --- | --- | --- |
| 表现层 | `apps/web`（React 19 + TypeScript + Vite） | 角色化导航、表单、服务端数据查询、错误与状态显示 | 不保存认证令牌；已保存单据一律从服务器读取 |
| 接口层 | `apps/server/src/index.ts` | 路由注册、静态资源托管、健康检查、错误统一处理 | — |

| 层 | 构件 | 职责 | 关键持久化 |
|---|---|---|---|
| 领域服务层 | `auth.ts` | Argon2id、会话、角色与 CSRF 校验 | `users`, `sessions`, `audit_events` |
| | `catalog.ts` | 场地、活动、展位、商品、员工、班次；报表查询 | 六类主数据 + `event_categories` |
| | `inventory.ts` | 收货、调拨确认、盘点、余额与占用 | `stock_*` 全族 |
| | `sales.ts` | 草稿、占用、线下收款确认、过账、取消 | `sales`, `sale_lines`, `stock_holds` |
| | `returns.ts` | 授权、部分退货、良品/损坏去向、负佣金 | `returns`, `return_lines` |
| | `commissions.ts` | 正负计提、结算占用、付款登记 | `commission_*` |
| 平台层 | `db.ts` / `core.ts` / `amounts.ts` | 单连接事务、整数金额与舍入、幂等键、审计、错误模型 | `idempotency_keys`, `audit_events` |

**依赖规则**：领域服务之间**不互相直接调用数据库以外的私有状态**；跨模块共享逻辑必须下沉到平台层；表现层只能通过 HTTP API 访问领域逻辑。

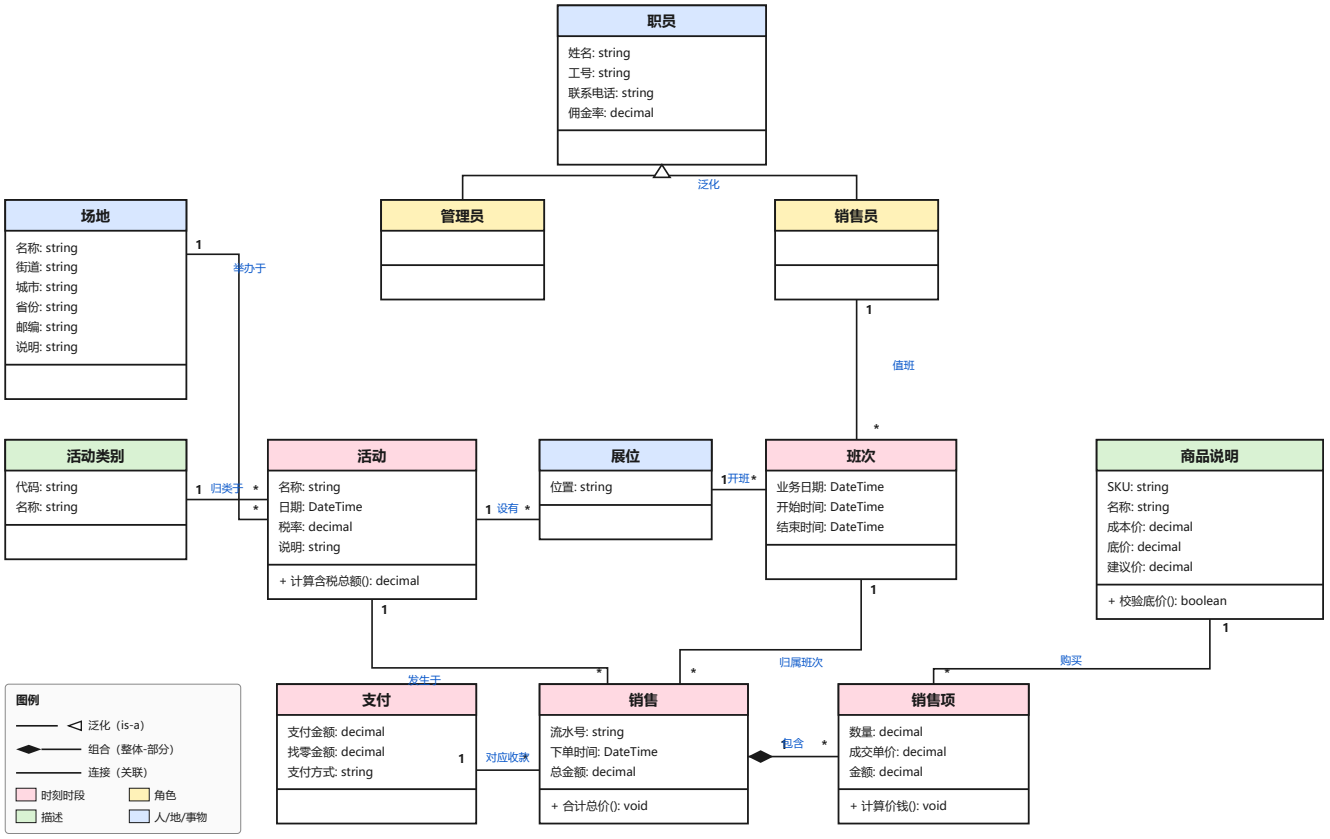
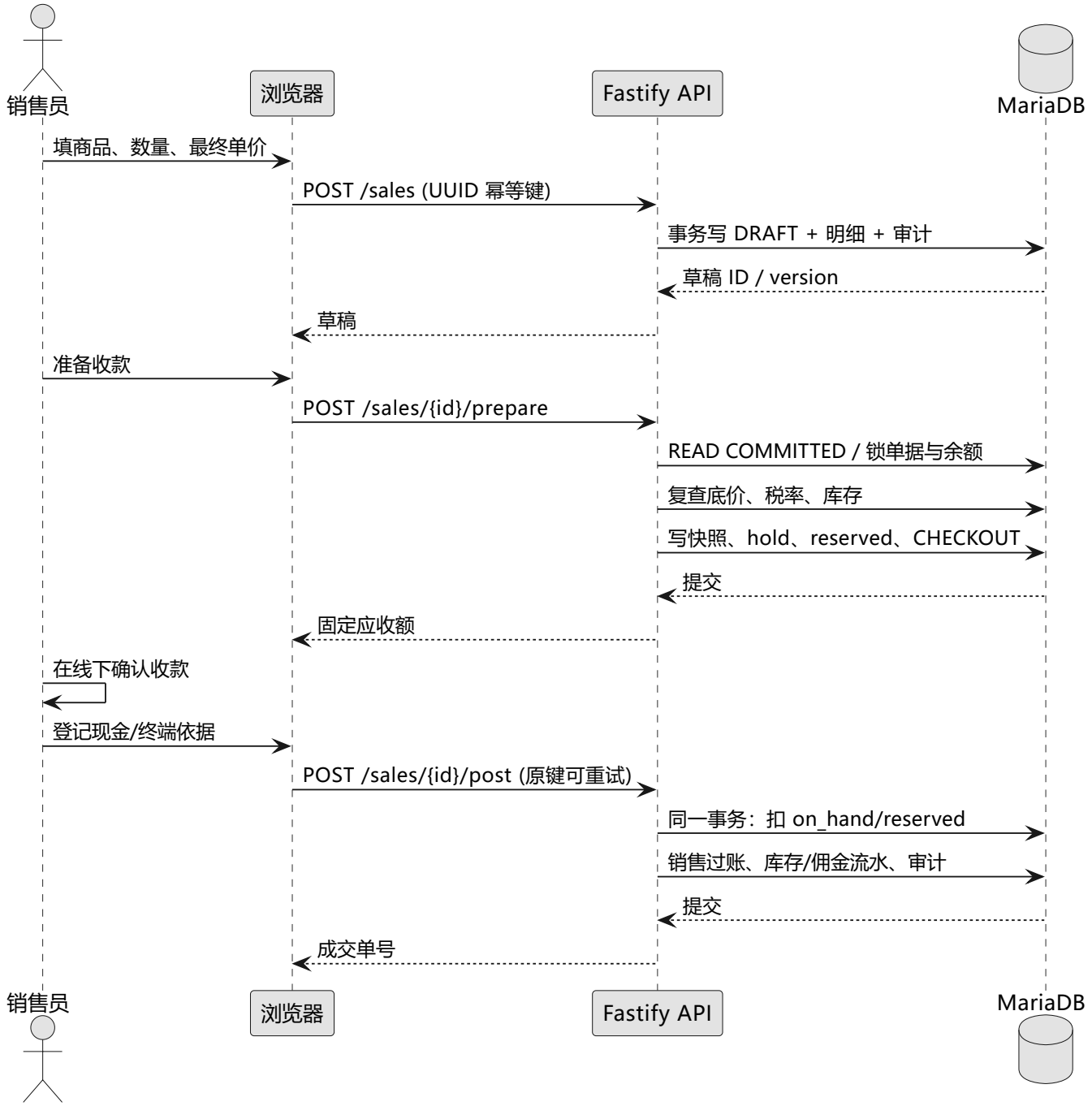



图 1 Foxcore Retail 分析类图（实体类图）——泛化：管理员/销售员-职员；组合：销售-销售项；连接：销售项-商品说明

**图 3-3　Foxcore Retail 实体类图（分析类图：泛化 / 组合 / 连接，架构型四色标注）**

## 3.4.3 交互视图（Interaction / Process View 中的时序）

关键交易——**准备收款并过账销售**的时序如下。




**销售时序图**

**图 3-4 销售准备收款与过账时序**（源：`diagrams/sale-sequence.puml`）

时序要点：

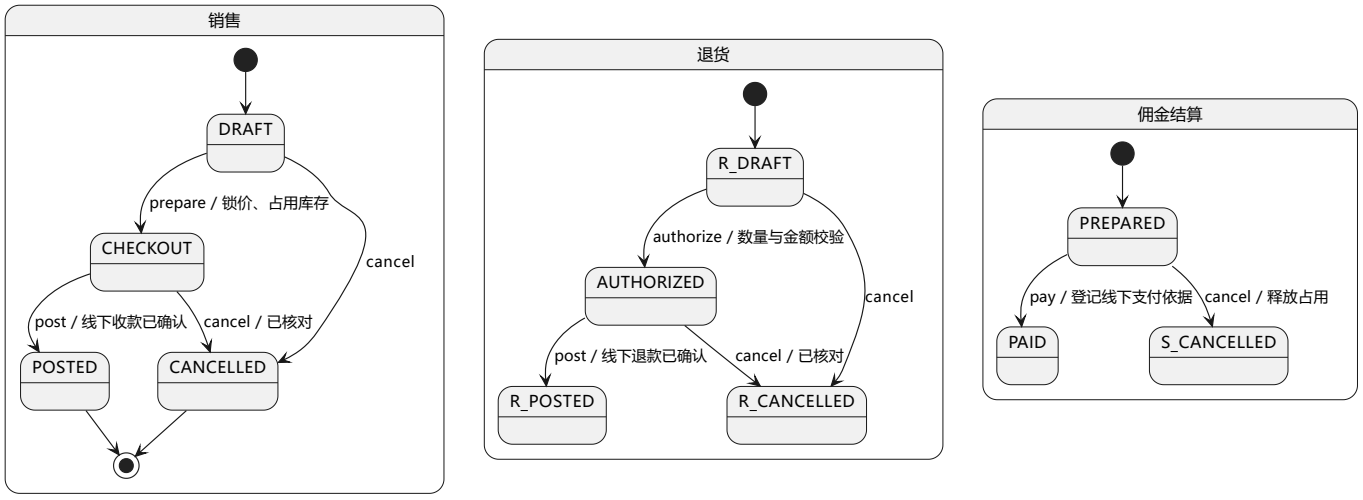
1. 客户端发起准备收款时携带 `Idempotency-Key`；服务端首先在同一事务中处理幂等行（命中且摘要一致则直接返回首次响应）；
2. 事务内依次校验活动状态、班次状态、商品底价与可售量，创建 `ACTIVE` 占用；
3. 客户端在线下完成收款后，携带收款确认调用过账；服务端将占用转为 `CONSUMED`，`on_hand` 与 `reserved` 同量下降，写入库存流水、佣金流水、成交时间与审计；
4. 任一步失败即整体回滚，占用不残留；网络不确定时客户端应以**同一幂等键**重试，而非重新发起新单。

### 3.4.4 进程视图（Process View）

描述运行时的并发、事务与锁行为。

| 方面 | 设计 |
| --- | --- |
| 进程模型 | 单 Node 进程；Fastify 事件循环处理请求；CPU 密集的 Argon2id 与金额计算为短任务 |
| 连接池 | `mysql2` 连接池；**业务事务强制从池中取出同一连接**并在其上开启事务 |
| 隔离级别 | `READ COMMITTED` |
| 加锁 | 关键行 `SELECT ... FOR UPDATE`；全局固定加锁顺序（幂等 → 活动 → 班次/销售 → 商品 → 余额） |
| 冲突处理 | 死锁（1213）、锁等待超时（1205）、并发读取冲突 → 回滚并最多重试 3 次；仍失败则返回 409，由客户端决定是否重试 |
| 幂等 | `(actor_id, operation, command_key)` 为主键；请求摘要 `request_hash` 不一致返回 409 |
| 并发验证 | 20 个并发请求争抢最后一件商品：恰 1 成功，19 个冲突（STD TC-007） |

**状态机**（完整状态源见 `diagrams/states.puml`）：




**状态图**

**图 3-5 核心实体状态机**（源：`diagrams/states.puml`）

- 活动：`DRAFT → OPEN → CLOSED`
- 班次：`PLANNED → ACTIVE → CLOSED`
- 销售：`DRAFT → CHECKOUT → POSTED`，未成交可 `CANCELLED`
- 退货：`DRAFT → AUTHORIZED → POSTED`，未完成可 `CANCELLED`
- 佣金结算：`PREPARED → PAID`，未付款可 `CANCELLED`
- 库存占用：`ACTIVE → CONSUMED | RELEASED`

### 3.4.5 物理/部署视图（Physical / Deployment View）

#### 3.4.5.1 本地形态（当前基线）

| 服务 | 镜像/基准 | 职责 | 网络 |
| --- | --- | --- | --- |
| db | MariaDB 11.8 | 数据持久化；带健康检查 | 仅容器内网，**不映射主机端口** |
| migrate | 一次性容器 | 以管理员身份应用 db/migrations/*.sql，随后收紧运行账号权限；退出码 0 后 app 才启动 | 容器内网 |
| app | Node 24 + 构建产物 | Fastify API + 静态资源；健康检查 /health/ready | 127.0.0.1:3000 |

- 随机数据库口令与首位管理员临时口令写入 **Docker Secret 卷**；app 只挂载应用数据库口令，不接触 root 口令卷；
- 运行账号仅对必要表拥有 UPDATE/DELETE，**库存流水、佣金流水、审计表不授 UPDATE/DELETE**；
- db_data 为命名卷，docker compose down 不删除数据（down -v 会删除，文档明确禁止）。

#### 3.4.5.2 云端形态（可选，见 08）

| 项 | 本地 | 云端 |
| --- | --- | --- |
| 应用 | Docker 容器常驻 | Vercel Serverless Function（api/index.js 适配 app.server.emit('request', req, res)） |
| 数据库 | 容器 MariaDB | TiDB Cloud Starter（MySQL 兼容），DB_SSL=true 开启 TLS |
| 静态资源 | Fastify @fastify/static | Vercel outputDirectory 直出 + rewrite 回退 |
| 启动 | app.listen(...) | VERCEL 环境变量存在时**跳过** listen（ADR-09） |
| 初始化 | migrate 服务 | scripts/vercel-setup.mjs 在构建阶段幂等执行迁移与管理员初始化 |

**兼容性保证**：云端适配全部以环境变量门控，本地 Compose 流程零改动（约束 DC-07）。

---

## 3.5 数据设计

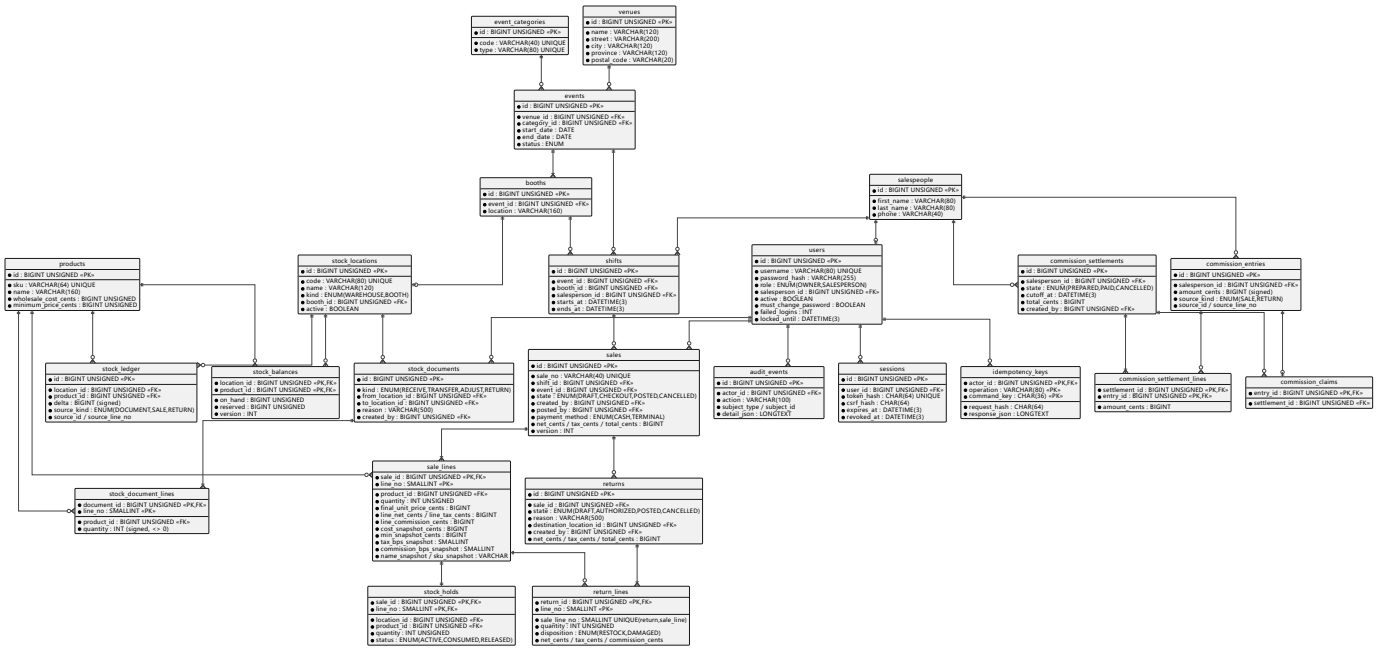
本章给出数据设计的**总体原则**，物理结构、数据字典、归一化推导与 DDL 详见 04 数据库设计与 ERD。

### 3.5.1 设计原则

1. **权威在流水，余额是缓存**：stock_ledger、commission_entries 为只增不改的权威记录；stock_balances 可随时由流水重算。
2. **历史靠快照**：成交行冻结单价、名称、SKU、成本、底价、税率与佣金率，主数据后续变更不影响历史口径。

3. **约束下沉数据库**：归属、取值域、符号、非负等由 CHECK / 外键 / 唯一键强制，不依赖应用层自觉。
4. **金额一律整数分**：`bigint` 存储，服务端 half-up 计算，杜绝浮点误差。
5. **一切可重算**：16 项不变量由 `db/check-invariants.sql` 定期核查，`npm run db:check` 在差异非零时退出非零。

### 3.5.2 物理模型概览




**物理 ERD**

**图 3-6 物理 ERD（24 张表）**（源：`diagrams/physical-erd.puml`）

| 表族 | 表数 | 代表表 |
|---|---|---|
| 主数据 | 7 | event_categories, venues, events, booths, products, salespeople, shifts |
| 身份与会话 | 2 | users, sessions |
| 交易 | 2 | sales, sale_lines |
| 库存 | 6 | stock_locations, stock_balances, stock_documents, stock_document_lines, stock_ledger, stock_holds |
| 退货 | 2 | returns, return_lines |
| 佣金 | 4 | commission_entries, commission_settlements, commission_settlement_lines, commission_claims |
| 平台 | 2 | idempotency_keys, audit_events |

概念层 Barker 模型见 `diagrams/core-barker.svg`；与课程案例 A/B 的九核心实体映射见 04 第 4.2 节。

# 3.6 用户界面设计

本章给出界面设计的**原则与结构**，逐屏操作步骤与截图见 06 用户操作手册。

## 3.6.1 设计原则

| 原则 | 落实 |
| --- | --- |
| 角色可见即角色可用 | 导航按角色裁剪：销售员不渲染“退货与核对”“操作日志”入口，服务端同时做二次校验 |
| 服务端为唯一真相 | 页面不缓存业务金额，刷新即从 API 重取；不保存认证令牌 |
| 错误可操作 | 错误以 {code，message，requestId} 呈现，requestId 可与服务端日志对应 |
| 现场可用性 | 关键操作按钮尺寸适配移动端；目录类列表每页 20 条、销售列表每页 100 条；选择器上限 100 项 |
| 不误导 | 毛利标注为“标准成本估算”；报表口径标注“仅计已过账销售减已过账退货” |

## 3.6.2 导航结构

| 路径 | 页面 | OWNER | SALESPERSON |
| --- | --- | --- | --- |
| / | 工作台 | √ | √ |
| /events | 活动与展位 | √ | √（只读） |
| /products | 商品 | √ | √（只读） |
| /people | 人员与班次 | √ | √（只读） |
| /stock | 库存 | √ | √（查询与确认） |
| /sales | 销售工作台 | √ | √（仅本人班次） |
| /returns | 退货与核对 | √ | — |
| /commissions | 佣金 | √ | √（仅本人） |
| /reports | 报表 | √ | √（不含成本口径） |
| /audit | 操作日志 | √ | — |

## 3.6.3 关键界面流程

- 登录与首次改密 → 见 06 第 6.1 节；
- 开场作业（建活动 → 建展位 → 收货 → 调拨 → 开班） → 见 06 第 6.2 节；
- 成交（草稿 → 准备收款 → 线下收款 → 确认成交） → 见 06 第 6.3 节；
- 退货、佣金与报表 → 见 06 第 6.4 节。

# 3.7 可追踪性

## 3.7.1 需求 → 设计 → 测试 追踪矩阵

| 需求编号 | 需求简述 | 设计（本文档） | 数据库（04） | 测试（05） |
|---|---|---|---|---|
| FR-01 / UC-01 | 登录、改密、登出 | 2.3.1 | users, sessions | TC-001, TC-002 |
| FR-02 / UC-02 | 场地与活动管理 | 2.3.2 | venues, events, event_categories | TC-003 |
| FR-03 / UC-03 | 展位管理 | 2.3.2 | booths | TC-003 |
| FR-04 / UC-04 | 商品管理 | 2.3.2 | products | TC-003 |
| FR-05 / UC-05 | 员工与班次 | 2.3.2 | salespeople, shifts, users | TC-004 |
| FR-06 / UC-06 | 收货与调拨 | 2.3.3 | stock_locations, stock_documents, stock_ledger | TC-005 |
| FR-07 / UC-07 | 盘点与余额 | 2.3.3 | stock_balances, stock_document_lines | TC-005 |
| FR-08 / UC-08 | 销售草稿与准备收款 | 2.3.4 | sales, sale_lines, stock_holds | TC-006, TC-007 |
| FR-09 / UC-09 | 过账与取消 | 2.3.4 | sales, stock_ledger, commission_entries | TC-006, TC-008 |
| FR-10 / UC-10 | 退货与退款 | 2.3.5 | returns, return_lines | TC-009 |
| FR-11 / UC-11 | 佣金计提与结算 | 2.3.6 | commission_* | TC-010 |
| FR-12 / UC-12 | 报表与导出 | 2.3.7 | 只读查询 | TC-011 |
| NFR-PERF-02 | 并发不超卖 | 2.3.4、4.4 | FOR UPDATE, 唯一键 | TC-007 |
| NFR-REL-01 | 重试不重复过账 | 2.3.8 | idempotency_keys | TC-006 |
| NFR-SEC-01/02/03 | 口令、越权、成本隔离 | 2.3.1、2.3.8 | 角色字段与 CHECK | TC-002, TC-013 |
| NFR-AUD-01 | 关键操作可追溯 | 2.3.8 | audit_events | TC-013 |

完整的 CASE/DERIVED/EXT 追踪见 traceability.csv。

## 3.7.2 设计 → 构件追踪

| 设计要素 | 构件 | 可验证方式 |
|---|---|---|
| 固定加锁顺序 | sales.ts 事务块 | 代码审查 + TC-007 并发结果 |
| 幂等与业务同事务 | core.ts + sales.ts | TC-006 |

| 设计要素 | 构件 | 可验证方式 |
| --- | --- | --- |
| 金额整数分与 half-up | `amounts.ts` | 单元测试 10/10（05 第 5.2 节） |
| 快照冻结 | `sale_lines` 快照列 | 不变量 I-04 + TC-008 |
| 角色裁剪 | 服务层序列化 | TC-013 |
| 不变量核查 | `db/check-invariants.sql` | `npm run db:check` 16/16 差异为 0 |

# 附录 A：架构决策记录（ADR）

| 编号 | 决策 | 理由 | 代价/风险 |
| --- | --- | --- | --- |
| ADR-01 | MariaDB 11.8 LTS + 版本化 SQL 迁移 | 与课程案例 B 对齐；行锁与 CHECK 是强一致的前提 | DDL 隐式提交，迁移失败需停止修复而非声称整体回滚 |
| ADR-02 | 单进程 Fastify + 同源 React | 简化安装与会话边界；单组织场景足够 | 扩容前需评估锁竞争；单实例故障即整体不可用 |
| ADR-03 | 整数分 + 成交行快照 | 消除浮点误差；保留历史价格与佣金口径 | 存储冗余增加；主数据变更需新版本而非原地改 |
| ADR-04 | 先占用再确认线下收款 | 杜绝超卖；待核对单保留占用以便人工查明 | 待收款单占用不自动释放，需人工取消 |
| ADR-05 | 本地随机初始口令存 Secret 卷 | `docker compose up` 后即可安全初始化 | 首次登录强制改密；口令文件须妥善保管 |
| ADR-06 | 权威在流水、余额作缓存 | 账本可重算、可审计 | 需额外维护余额与流水一致性（不变量核查） |
| ADR-07 | 约束下沉数据库 | 应用层 bug 不能破坏数据完整性 | 部分校验错误表现为数据库错误，需映射为业务错误码 |
| ADR-08 | 幂等键强制且同事务 | 现场弱网重试安全 | 客户端必须正确管理与复用键 |
| ADR-09 | 云端适配以环境变量门控 | 本地部署流程零改动（DC-07） | 存在两套运行形态，需分别验收 |
| ADR-10 | 不对接支付/退款/转账渠道 | 明确系统边界，避免资金风险与合规负担 | 收款核对依赖人工 |
| ADR-11 | 报表口径仅计已过账 | 口径单一、可解释 | 草稿与待收款单不计入，业务需理解差异 |
| ADR-12 | 单时区 `America/Toronto` | 与业务现场一致 | 不支持跨时区门店 |

**文档结束。** 下游：数据库设计见 04，测试与验收见 05。

> 作业文档与图纸仓库：https://github.com/xingchenyd/foxcore-retail —— 文档位于 `docs/`，图纸位于 docs/diagrams（实体类图 class-diagram.png/svg、物理 ERD physical-erd.png、概念 ERD core-barker.png/svg、架构图、业务序列图、用例图、状态图等全套 .puml/.svg/.png）。

# 04 数据库设计与 ERD

**Foxcore Retail — 数据库设计说明（概念 / 逻辑 / 物理三层）**

## 4.1 引言

### 4.1.1 目的

本文档给出 Foxcore Retail 数据库的**完整设计说明**，覆盖从概念模型到可执行 DDL 的全过程，用于：

1. 说明课程案例 A/B 的概念模型如何映射为可实现的物理模式；
2. 给出每一张表、每一个字段、每一个约束的权威定义（数据字典）；
3. 论证模式满足第三范式（3NF）并说明有意的冗余取舍；
4. 定义数据完整性断言（不变量）及其核查方式；
5. 规定迁移、种子数据与备份恢复的规程。

### 4.1.2 范围

- 概念层：按业务模块的局部 ERD 与全局概念 ERD（Barker 记号）；
- 逻辑层：逻辑 ERD、逻辑数据字典、关系模式、归一化推导；
- 物理层：24 张表的完整定义、索引策略、约束清单、种子数据；
- 完整性：16 项不变量与核查脚本；
- 运维：迁移顺序、校验和、备份恢复。

不含：SQL 调优基准、分片与读写分离、跨时区与多币种扩展。

### 4.1.3 定义、缩写与记号

| 记号/术语 | 含义 |
| --- | --- |
| # | Barker 记号：**标识符**（唯一标识实体的属性） |
| * | Barker 记号：**强制**属性（必填） |
| o | Barker 记号：**可选**属性（可空） |
| **PK / FK / AK** | 主码 / 外码 / 候选码（Alternate Key） |
| **1 : N** | 一对多关系 |
| **弱实体** | 依赖父实体存在、主键部分或全部来自父实体的实体（如 `sale_lines`） |
| **3NF** | 第三范式：无传递函数依赖；本设计的目标范式 |
| **不变量** | 任意事务提交后必须为真的断言，由 `db/check-invariants.sql` 核查 |
| **快照列** | 成交时冻结的主数据副本，用于保证历史口径 |

| 记号/术语 | 含义 |
| --- | --- |
| **bps** | basis point，万分之一（税率、佣金率的存储单位） |

### 4.1.4 参考文献

| 编号 | 文献 | 用途 |
| --- | --- | --- |
| REF-01 | Foxcore Retail A — Designing a Database | 概念基线：六类信息 |
| REF-02 | Foxcore Retail B — Database Implementation, Exhibit 1 | 物理基线：九核心实体 |
| REF-03 | 02 SRS | 数据相关需求 |
| REF-04 | 03 SDD | 数据设计原则 |
| REF-05 | `db/migrations/001_core.sql` | 权威 DDL |
| REF-06 | `db/check-invariants.sql` | 不变量核查 |

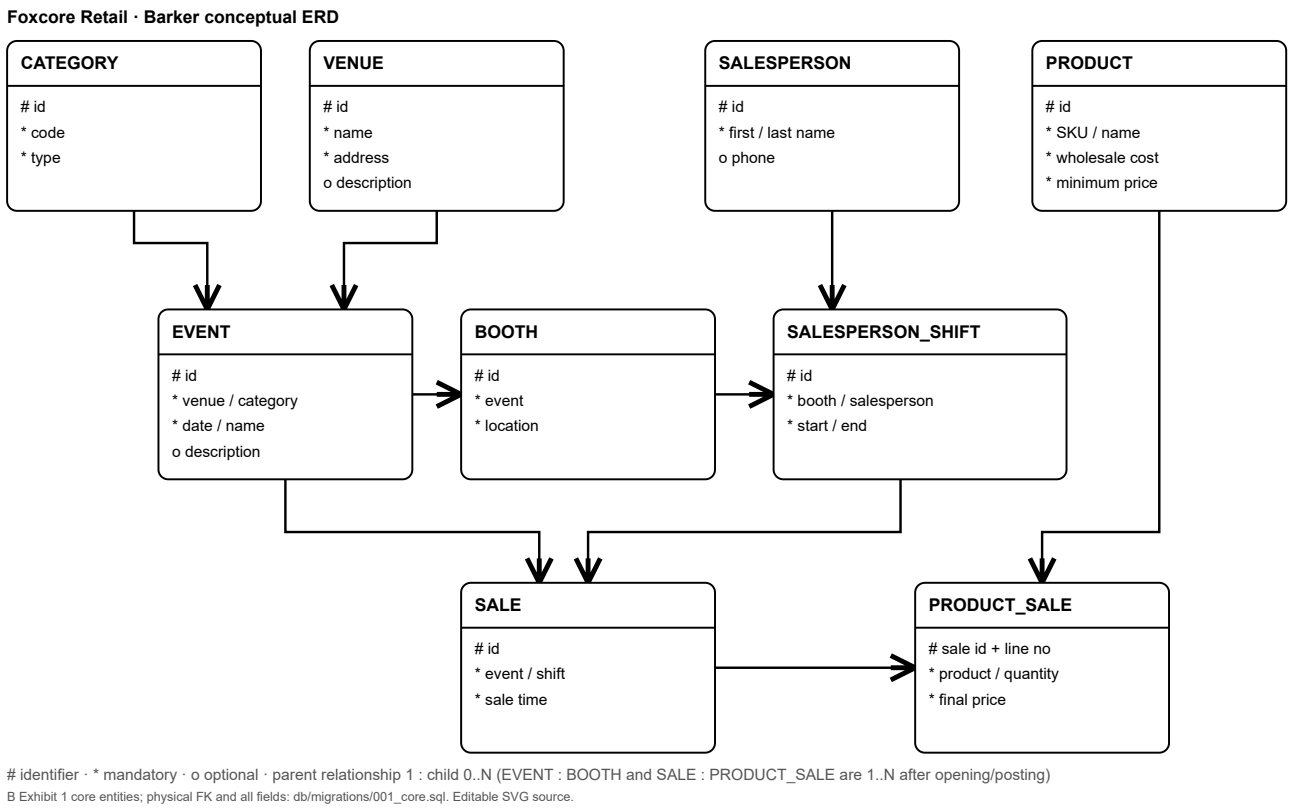
### 4.1.5 概述：现有系统 vs 拟议系统

| 维度 | 现有（AS-IS） | 拟议（TO-BE） |
| --- | --- | --- |
| 数据载体 | 纸质交接表、Excel 台账、事后汇总 | 单一关系数据库，事务化写入 |
| 库存可见性 | 事后盘点才知道差额 | 实时余额 + 只增流水，随时重算 |
| 价格口径 | 现场口头议价，无底价校验 | 商品底价快照 + 服务端复核 |
| 成交归属 | 手写班次，难以定位责任班次 | 销售强制绑定 (`shift_id`, `event_id`) 复合外键 |
| 退货 | 无冲回记录，佣金照提 | 退货行按原单分摊税佣，生成负向佣金 |
| 追溯 | 无法回答”谁在何时改了什么” | `audit_events` 与业务同事务写入 |

---

## 4.2 概念数据库设计

概念模型以 **Barker 记号**表达，完整图见 `diagrams/core-barker.svg`。




**概念 ERD**

**图 4-1 全局概念 ERD（Barker 记号）**

### 4.2.1 课程案例基线与映射

案例 A 列出六类信息（Venue、Event、Booth、Product、Salesperson、Sale 相关），案例 B 的 Exhibit 1 给出**九个核心实体**：CATEGORY、VENUE、EVENT、BOOTH、PRODUCT、SALESPERSON、SALESPERSON_SHIFT、SALE、PRODUCT_SALE。**“六类信息”不等于”只能建六张表”**——A 的六类是信息分类，B 的九实体是实体级基线，运行所需的身份、库存、退货、佣金、审计等属于在九实体之上的**运行扩展（EXT）**。

| B 实体 | 物理表 | 主键 | 关键关系 | 说明 |
|---|---|---|---|---|
| CATEGORY | `event_categories` | `id` | 1:N → `events` | 七类原文类别作种子；`code/type` 均唯一 |
| VENUE | `venues` | `id` | 1:N → `events` | B 图拆分地址为 Street/City/Province/PCode；A 的完整地址由界面组合显示 |
| EVENT | `events` | `id` | FK `venue_id,category_id`；1:N → `booths,shifts,sales` | 活动日期、状态、`tax_bps`、政策确认为运行补充 |
| BOOTH | `booths` | `id` | FK `event_id`；UNIQUE(`event_id,location`) | 开放活动前至少存在一个有效展位 |

| B 实体 | 物理表 | 主键 | 关键关系 | 说明 |
|---|---|---|---|---|
| PRODUCT | products | id | 1:N → sale_lines,stock_balances | sku 唯一；保留成本与底价 |
| SALESPERSON | salespeople | id | 1:N → shifts,users,commission_entries | A 中的电话保留且可空；B 图未列电话不代表删除 |
| SALESPERSON_SHIFT | shifts | id | 复合 FK (booth_id,event_id)；UNIQUE(id,event_id) | 一班一展位；一名员工可有多班 |
| SALE | sales | id | 复合 FK (shift_id,event_id) | 保留 B 的 SALE→EVENT 与 SALE→SHIFT **双关联** |
| PRODUCT_SALE | sale_lines | PK(sale_id,line_no) | FK product_id | B 原有 Line_No、Quantity、Final_Price；本系统**额外冻结快照** |

**概念关系骨架：**

```
CATEGORY   1————N  EVENT  N————1  VENUE

EVENT      1————N  BOOTH  1————N  SALESPERSON_SHIFT  N————1  SALESPERSON

SALESPERSON_SHIFT 1—N  SALE ;       EVENT  1—N  SALE

SALE       1————N  PRODUCT_SALE  N————1  PRODUCT
```

## 4.2.2 分模块概念设计

按 KADA 概念设计文档的惯例，逐模块给出：模块描述 → 事务需求（录入 / 更新删除 / 查询）→ 局部概念 ERD。

### 4.2.2.1 模块 1：身份认证（Authentication）

**模块描述**：为业主管理员与现场销售员提供账号、口令与会话管理；销售员账号与员工档案一一对应。

**事务需求**

| 类型 | 需求 |
|---|---|
| 数据录入 | 创建角色账号（OWNER / SALESPERSON）；登录生成会话；改密 |
| 更新/删除 | 修改口令；启用/停用账号（不物理删除）；登出撤销会话 |
| 查询 | 当前登录者 /auth/me；账号列表；审计日志 |

**局部概念 ERD**

```
SALESPERSON 1─────1  USER  1─────N  SESSION
                        │
                        └─────N  AUDIT_EVENT
```

#### 4.2.2.2 模块 2：主数据（Master Data）

**模块描述**：维护场地、活动类别、活动、展位、商品、员工档案及其排班。

**事务需求**

| 类型 | 需求 |
| --- | --- |
| 数据录入 | 场地（含拆分地址）；活动（日期、类别、税率、政策确认）；展位；商品（SKU、成本、底价、建议价）；员工（姓名、地址、电话、佣金率）；班次（展位、员工、业务日期、计划时段） |
| 更新/删除 | 商品价格与成本 PATCH；停用而非删除；活动开放/关闭/取消；班次开班/收班/取消 |
| 查询 | 各类列表（分页 20）；按活动查展位与班次；按员工与日期查班次 |

**局部概念 ERD**

```
CATEGORY 1──N EVENT N──1 VENUE

EVENT 1──N BOOTH 1──N SHIFT N──1 SALESPERSON
```

#### 4.2.2.3 模块 3：库存（Inventory）

**模块描述**：管理主仓与展位两类库位、库内单据（收货/调拨/盘点/退货）、余额缓存与权威流水，以及待收款占用。

**事务需求**

| 类型 | 需求 |
| --- | --- |
| 数据录入 | 创建收货单（供应商→主仓）、调拨单（主仓→展位）、盘点单（差额调整） |
| 更新/删除 | **确认**调拨（实物交接后）；**不删除**任何流水或单据；占用状态 ACTIVE→CONSUMED/RELEASED |
| 查询 | 库位列表与余额；按库位+商品的流水明细；单据详情；可售量 = on_hand − reserved |

**局部概念 ERD**

```
LOCATION(WAREHOUSE|BOOTH) 1—N STOCK_BALANCE N—1 PRODUCT

LOCATION 1—N STOCK_DOCUMENT 1—N STOCK_DOCUMENT_LINE N—1 PRODUCT

LOCATION 1—N STOCK_LEDGER N—1 PRODUCT

SALE_LINE 1—1 STOCK_HOLD N—1 LOCATION
```

关键概念约束：`on_hand ≥ reserved`；`STOCK_LEDGER` 只增不改（`delta ≠ 0`）；`STOCK_BALANCE` 可由流水重算。

#### 4.2.2.4 模块 4：销售（Sales）

**模块描述**：现场录单、占用库存、线下收款确认与过账；成交行冻结全部价格与费率快照。

**事务需求**

| 类型 | 需求 |
| --- | --- |
| 数据录入 | 创建销售草稿（班次、明细行、商定单价）；准备收款（复核并占用）；过账（登记收款确认） |
| 更新/删除 | 草稿可改可取消；CHECKOUT 状态取消须先确认款项与商品去向；**已 POSTED 不可改，须走退货** |
| 查询 | 单据分页（销售列表默认 100/页，上限 100）；按班次/活动/日期过滤；上一页/下一页 |

**局部概念 ERD**

```
SHIFT 1—N SALE 1—N SALE_LINE N—1 PRODUCT

SALE_LINE 1—1 STOCK_HOLD

SALE_LINE 1—1 COMMISSION_ENTRY
```

#### 4.2.2.5 模块 5：退货（Returns）

**模块描述**：对已过账单据做部分或整单退货，分摊原税佣，区分良品回库与损坏品。

**事务需求**

| 类型 | 需求 |
| --- | --- |
| 数据录入 | 选择原单与原行；登记退货数量、处置方式（RESTOCK/DAMAGED）、理由 |
| 更新/删除 | 授权 → 过账；未过账可取消；**不物理删除**退货记录 |
| 查询 | 退货单详情；按原单查累计退货 |

关键概念约束：累计退货数量 ≤ 原销量；一行在同一退货单中只能出现一次。

#### 4.2.2.6 模块 6：佣金（Commissions）

**模块描述**：按销售行计提正向佣金、按退货行计提负向冲回；按员工结算并占用未结流水。

**事务需求**

| 类型 | 需求 |
| --- | --- |
| 数据录入 | 过账/退货自动生成计提条目；创建结算（锁定员工全部未结条目） |
| 更新/删除 | 结算登记支付；未支付可取消并释放占用；**条目本身不更新不删除** |
| 查询 | 按员工查计提明细；结算列表与明细 |

关键概念约束：**一笔计提条目只能被一个结算占用**（`commission_claims` 以 `entry_id` 为主键）。

#### 4.2.2.7 模块 7：平台服务（Platform）

**模块描述**：幂等键、审计事件、迁移校验和。

```
USER 1──N IDEMPOTENCY_KEY

USER 1──N AUDIT_EVENT
```

---

## 4.3 逻辑数据库设计

### 4.3.1 逻辑 ERD

逻辑模型即把概念模型中的多对多（ PRODUCT_SALE / STOCK 关联 ）解析为关联实体，并为所有关系确定可选性与基数。渲染件见 `diagrams/physical-erd.svg`（物理层同时表达逻辑结构）； 逻辑关系的文字化表达见 §3.3 关系模式。

### 4.3.2 逻辑数据字典（实体级）

| 实体 | 标识符（#） | 强制属性（*） | 可选属性（o） | 基数要点 |
| --- | --- | --- | --- | --- |
| CATEGORY | `id` | `code`, `type` | — | 1:N → EVENT |
| VENUE | `id` | `name`, `street`, `city`, `province`, `postal_code` | `description` | 1:N → EVENT |
| EVENT | `id` | `venue_id`, `category_id`, `name`, `start_date`, `end_date`, `status` | `tax_bps`, `description` | 1:N → BOOTH / SHIFT / SALE |
| BOOTH | `id` | `event_id`, `location` | — | 1:N → SHIFT；1:1 → LOCATION(BOOTH) |
| PRODUCT | `id` | `sku`, `name`, `wholesale_cost_cents`, `min_price_cents` | `suggested_price_cents` | 1:N → SALE_LINE / BALANCE |

| 实体 | 标识符（#） | 强制属性（*） | 可选属性（o） | 基数要点 |
|---|---|---|---|---|
| SALESPERSON | id | first_name, last_name | street,city,province, postal_code,phone,commission_bps | 1:N → SHIFT / USER / COMMISSION |
| SHIFT | id | salesperson_id, booth_id, event_id, business_date, planned_start, planned_end, status | actual_start, actual_end | 1:N → SALE |
| USER | id | username, password_hash, role, active | salesperson_id, locked_until | 1:N → SESSION |
| SESSION | id | user_id, token_hash, csrf_hash, expires_at | revoked_at | N:1 → USER |
| SALE | id | shift_id, event_id, state, created_by | sale_no, posted_by, payment_method, payment_reference, cash_received_cents | 1:N → SALE_LINE |
| SALE_LINE | sale_id+line_no | product_id, quantity, final_unit_price_cents | 全部快照列 | 弱实体 |
| LOCATION | id | code, name, kind | booth_id | 1:N → BALANCE / LEDGER |
| STOCK_BALANCE | location_id+product_id | on_hand, reserved | — | 关联实体 |
| STOCK_DOCUMENT | id | kind, reason, created_by | from_location_id, to_location_id | 1:N → LINE |
| STOCK_DOCUMENT_LINE | document_id+line_no | product_id, quantity | — | 弱实体 |
| STOCK_LEDGER | id | location_id, product_id, delta, source_kind, source_id, source_line_no | — | 只增 |
| STOCK_HOLD | sale_id+line_no | location_id, product_id, quantity, status | — | 弱实体 |
| RETURN | id | sale_id, state, reason, created_by | refund_reference, destination_location_id | 1:N → RETURN_LINE |

| 实体 | 标识符（#） | 强制属性（*） | 可选属性（o） | 基数要点 |
|---|---|---|---|---|
| RETURN_LINE | return_id+line_no | sale_line_no, quantity, disposition | — | 弱实体 |
| COMMISSION_ENTRY | id | salesperson_id, amount_cents, source_kind, source_id, source_line_no | — | 只增 |
| COMMISSION_SETTLEMENT | id | salesperson_id, state, cutoff_at, total_cents, created_by | payment_reference, paid_at | 1:N → LINE / CLAIM |
| COMMISSION_SETTLEMENT_LINE | settlement_id+entry_id | amount_cents | — | 关联实体 |
| COMMISSION_CLAIM | entry_id | settlement_id | — | 1:1 占用 |
| IDEMPOTENCY_KEY | actor_id+operation+command_key | request_hash, response_json | — | 去重 |
| AUDIT_EVENT | id | action, subject_type | actor_id, subject_id, detail_json | 追加 |

### 4.3.3 关系模式（Relational Schema）

下划线为主码，(FK) 标外码，(UQ) 标唯一。

```
event_categories        (id, code(UQ), type(UQ))

venues                  (id, name, street, city, province, postal_code, description, active, version)

events                  (id, venue_id(FK), category_id(FK), name, start_date, end_date,
                         description, status, timezone, tax_bps, policy_confirmed, version)

booths                  (id, event_id(FK), location, active,
                         UQ(event_id,location), UQ(id,event_id))

products                (id, sku(UQ), name, wholesale_cost_cents, min_price_cents,
                         suggested_price_cents, active, version)

salespeople             (id, first_name, last_name, street, city, province, postal_code,
                         phone, description, active, commission_bps, version)

shifts                  (id, salesperson_id(FK), booth_id(FK), event_id(FK), business_date,
                         planned_start, planned_end, actual_start, actual_end, status, version,
                         UQ(id,event_id))

users                   (id, username(UQ), password_hash, role, salesperson_id(FK,UQ), active,
                         must_change_password, failed_logins, locked_until, created_at)

sessions                (id, user_id(FK), token_hash(UQ), csrf_hash, created_at,
                         last_seen_at, expires_at, revoked_at)

sales                   (id, sale_no(UQ), shift_id(FK), event_id(FK), state, created_at,
                         prepared_at, posted_at, created_by(FK), posted_by(FK), payment_method,
                         payment_reference, cash_received_cents, net_cents, tax_cents,
                         total_cents, version, 复合FK(shift_id,event_id)→shifts(id,event_id))

sale_lines              (sale_id(FK), line_no, product_id(FK), quantity,
                         final_unit_price_cents, line_net_cents, line_tax_cents,
                         line_commission_cents, cost_snapshot_cents, min_snapshot_cents,
                         tax_bps_snapshot, commission_bps_snapshot, name_snapshot, sku_snapshot)

stock_locations         (id, code(UQ), name, kind, booth_id(FK,UQ), active)

stock_balances          (location_id(FK), product_id(FK), on_hand, reserved, version)

stock_documents         (id, kind, from_location_id(FK), to_location_id(FK), reason,
                         created_by(FK), created_at)
```

```
stock_document_lines     (document_id(FK), line_no, product_id(FK), quantity)

stock_ledger             (id, location_id(FK), product_id(FK), delta, source_kind, source_id,

                          source_line_no, created_at,
UQ(source_kind,source_id,source_line_no,location_id))

stock_holds              (sale_id(FK), line_no(FK), location_id(FK), product_id(FK), quantity, status)

returns                  (id, sale_id(FK), state, reason, refund_reference,

                          destination_location_id(FK), created_by(FK), authorized_at, posted_at,

                          net_cents, tax_cents, total_cents, created_at)

return_lines             (return_id(FK), line_no, sale_line_no, quantity, disposition,

                          net_cents, tax_cents, commission_cents, UQ(return_id,sale_line_no))

commission_entries       (id, salesperson_id(FK), amount_cents, source_kind, source_id,

                          source_line_no, created_at, UQ(source_kind,source_id,source_line_no))

commission_settlements   (id, salesperson_id(FK), state, cutoff_at, total_cents,

                          payment_reference, created_by(FK), created_at, paid_at)

commission_settlement_lines (settlement_id(FK), entry_id(FK), amount_cents)

commission_claims        (entry_id(FK) PK, settlement_id(FK))

idempotency_keys         (actor_id(FK), operation, command_key, request_hash, response_json, created_at)

audit_events             (id, actor_id(FK), action, subject_type, subject_id, detail_json, created_at)

schema_migrations        (version, checksum_sha256, applied_at)
```

**复合外键链（归属不可篡改）**

```
booths  UQ(id, event_id)  ──>  shifts(booth_id, event_id)

shifts  UQ(id, event_id)  ──>  sales (shift_id, event_id)
```

该链使”把一笔销售改挂到另一活动”或”把班次改挂到另一活动的展位”在数据库层即被拒绝，无需依赖应用层校验。

### 4.3.4 归一化

**目标：3NF。** 逐范式论证：

| 范式 | 论证 |
| --- | --- |
| **1NF** | 所有列均为原子标量；无重复组。多值明细（销售明细、单据明细、退货明细）建模为独立弱实体，而非逗号拼接字段 |

| 范式 | 论证 |
| --- | --- |
| **2NF** | 所有弱实体的非主属性完全依赖于**整个**复合主码：sale_lines 的数量与价格依赖 (sale_id,line_no)；stock_balances 的 on_hand/reserved 依赖 (location_id,product_id)。不存在仅依赖部分主码的属性 |
| **3NF** | 消除传递依赖：venues 的地址只依赖 venues.id；sales 不冗余保存 venue_id（由 event_id 可达）；products 的名称不复制到 sale_lines 的**可编辑**字段中——名称只作为 name_snapshot 存在于历史行 |

**有意保留的冗余（已记录，非范式违规）**

| 冗余 | 理由 | 一致性保障 |
| --- | --- | --- |
| sale_lines 的 7 个快照列（单价、名称、SKU、成本、底价、税率、佣金率） | **历史口径冻结**：主数据后续变更不得改变已成交单据的金额与佣金解释（ADR-03） | 快照在过账时一次性写入，之后只读 |
| stock_balances（on_hand, reserved） | **性能**：避免每次可售量查询都聚合全表流水 | 不变量 I-02/I-04 核查”余额 = 流水汇总”，可随时重算 |
| sales 的 net_cents/tax_cents/total_cents | 单据级合计便于列表与报表直接读取 | 不变量 I-06 核查”合计 = 明细汇总” |
| commission_settlements.total_cents | 结算单总额 | 不变量 I-16 核查与明细一致 |

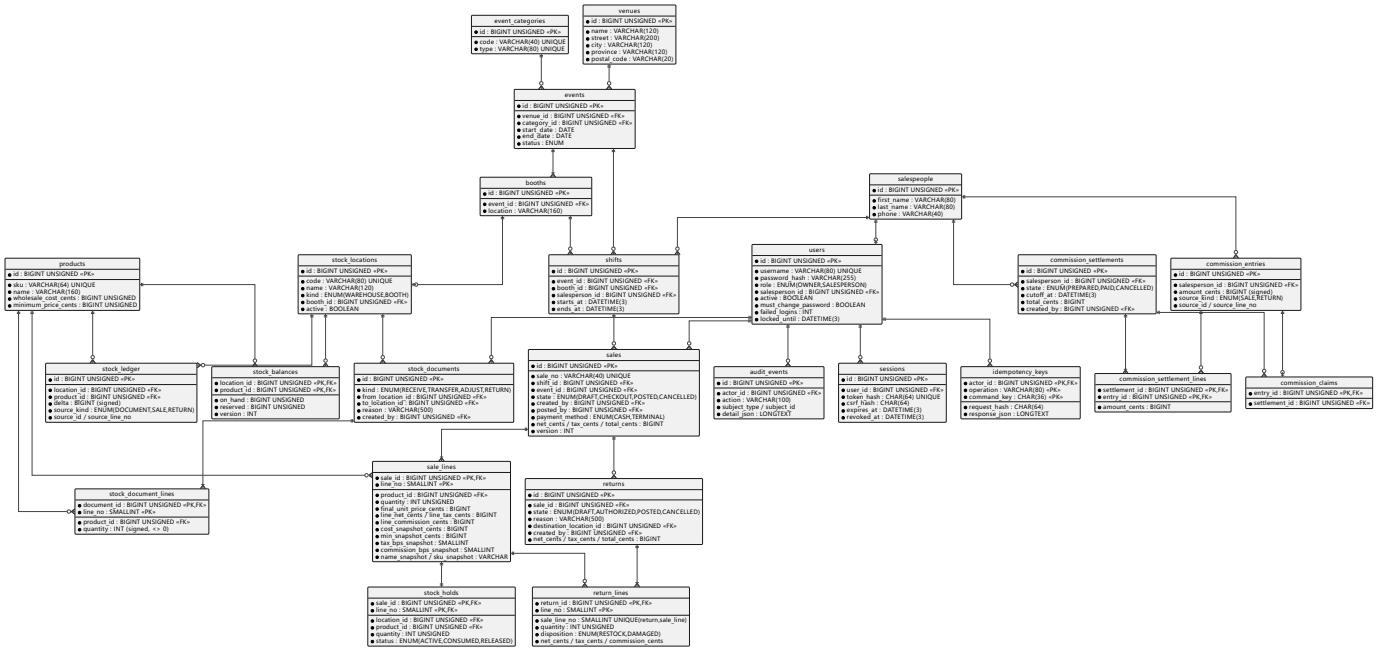
以上冗余均为**可由权威数据重算的派生值**，且每项都有对应的不变量核查，因此不构成更新异常风险。

---

# 4.4 物理数据库设计

---

## 4.4.1 物理 ERD




**物理 ERD**

**图 4-2 物理 ERD（24 张表 + `schema_migrations`）**（源：`diagrams/physical-erd.puml`）

**存储引擎**：全部 InnoDB（事务、行锁、外键、CHECK 支持）。 **字符集**：`utf8mb4`。 **金额**：一律 `BIGINT`（分），带符号列用于可正可负的流水（`delta`、`amount_cents`）。

## 4.4.2 物理数据字典

> 记号：**PK** 主码 / **FK** 外码 / **UQ** 唯一 / **CK** 检查约束 / **NN** 非空。

### 4.4.2.1 主数据（7 张）

**`event_categories`** — 活动类别（种子表）

| 列 | 类型 | 空 | 键 | 说明 |
|---|---|---|---|---|
| `id` | BIGINT UNSIGNED | NN | PK | 自增 |
| `code` | VARCHAR(40) | NN | UQ | 机器可读代码，如 `trade_show` |
| `type` | VARCHAR(80) | NN | UQ | 展示名，如 `trade show` |

**`venues`** — 场地

| 列 | 类型 | 空 | 键 | 说明 |
|---|---|---|---|---|
| `id` | BIGINT UNSIGNED | NN | PK | |
| `name` | VARCHAR(120) | NN | | 场地名 |
| `street` / `city` / `province` | VARCHAR(120/80/80) | NN | | A 图拆分地址的三部分 |
| `postal_code` | VARCHAR(16) | NN | | 邮编 |
| `description` | VARCHAR(500) | √ | | 备注 |

| 列 | 类型 | 空 | 键 | 说明 |
|---|---|---|---|---|
| active | BOOLEAN | NN | | 默认 TRUE，停用而非删除 |
| version | INT UNSIGNED | NN | | 乐观并发版本，默认 1 |

**events** — 活动

| 列 | 类型 | 空 | 键 | 说明 |
|---|---|---|---|---|
| id | BIGINT UNSIGNED | NN | PK | |
| venue_id | BIGINT UNSIGNED | NN | FK→venues | |
| category_id | BIGINT UNSIGNED | NN | FK→event_categories | |
| name | VARCHAR(120) | NN | | |
| start_date / end_date | DATE | NN | CK | end_date >= start_date |
| description | VARCHAR(500) | √ | | |
| status | ENUM(DRAFT,OPEN,CLOSED,CANCELLED) | NN | | 默认 DRAFT |
| timezone | VARCHAR(64) | NN | | 默认 America/Toronto |
| tax_bps | SMALLINT UNSIGNED | √ | CK | 税率（万分比），<= 10000 |
| policy_confirmed | BOOLEAN | NN | | 管理员确认税率与政策 |
| version | INT UNSIGNED | NN | | 默认 1 |

**booths** — 展位

| 列 | 类型 | 空 | 键 | 说明 |
|---|---|---|---|---|
| id | BIGINT UNSIGNED | NN | PK | |
| event_id | BIGINT UNSIGNED | NN | FK→events | |
| location | VARCHAR(80) | NN | UQ(event_id,location) | 同一活动下位置唯一 |
| active | BOOLEAN | NN | | 默认 TRUE |
| — | — | — | UQ(id,event_id) | **复合外键链起点** |

**products** — 商品

| 列 | 类型 | 空 | 键 | 说明 |
|---|---|---|---|---|
| id | BIGINT UNSIGNED | NN | PK | |
| sku | VARCHAR(64) | NN | UQ | 库存单位编码 |
| name | VARCHAR(160) | NN | | |
| wholesale_cost_cents | BIGINT UNSIGNED | NN | | 批发成本（分）——**销售员不可见** |
| min_price_cents | BIGINT UNSIGNED | NN | | 最低成交价（分），准备收款时复核 |
| suggested_price_cents | BIGINT UNSIGNED | √ | CK | 建议价，>= min_price_cents |
| active | BOOLEAN | NN | | |
| version | INT UNSIGNED | NN | | |

**salespeople** — 员工档案

| 列 | 类型 | 空 | 键 | 说明 |
|---|---|---|---|---|
| id | BIGINT UNSIGNED | NN | PK | |
| first_name / last_name | VARCHAR(80) | NN | | |
| street / city / province / postal_code | VARCHAR | √ | | 可选住址（A 图保留） |
| phone | VARCHAR(32) | √ | | A 图保留，可空 |
| description | VARCHAR(500) | √ | | |
| active | BOOLEAN | NN | | |
| commission_bps | SMALLINT UNSIGNED | √ | CK | 默认佣金率，<= 10000 |
| version | INT UNSIGNED | NN | | |

**shifts** — 班次（SALESPERSON_SHIFT）

| 列 | 类型 | 空 | 键 | 说明 |
|---|---|---|---|---|
| id | BIGINT UNSIGNED | NN | PK | |
| salesperson_id | BIGINT UNSIGNED | NN | FK→salespeople | |
| booth_id | BIGINT UNSIGNED | NN | 复合 FK→booths(id,event_id) | 一班一展位 |
| event_id | BIGINT UNSIGNED | NN | 同上 | |

| 列 | 类型 | 空 | 键 | 说明 |
| --- | --- | --- | --- | --- |
| business_date | DATE | NN | | 业务日期（多伦多时区） |
| planned_start / planned_end | DATETIME(3) | NN | CK | planned_end > planned_start |
| actual_start / actual_end | DATETIME(3) | √ | | 实际开班/收班 |
| status | ENUM(PLANNED,ACTIVE,CLOSED,CANCELLED) | NN | | 默认 PLANNED |
| version | INT UNSIGNED | NN | | |
| — | — | — | UQ(id,event_id) / IX(salesperson_id,business_date) | 复合外键链 + 常用查询索引 |

### 4.4.2.2 身份与会话（2 张）

**users**

| 列 | 类型 | 空 | 键 | 说明 |
| --- | --- | --- | --- | --- |
| id | BIGINT UNSIGNED | NN | PK | |
| username | VARCHAR(80) | NN | UQ | |
| password_hash | VARCHAR(255) | NN | | Argon2id 散列，**永不存明文** |
| role | ENUM(OWNER,SALESPERSON) | NN | | |
| salesperson_id | BIGINT UNSIGNED | √ | FK→salespeople, UQ | 销售员账号与档案一一对应 |
| active | BOOLEAN | NN | | 默认 TRUE |
| must_change_password | BOOLEAN | NN | | 首次登录强制改密 |
| failed_logins | INT UNSIGNED | NN | | 失败计数，默认 0 |
| locked_until | DATETIME(3) | √ | | 锁定时点 |
| created_at | DATETIME(3) | NN | | 默认当前时间(3) |
| — | — | — | CK | (role='OWNER' AND salesperson_id IS NULL) OR (role='SALESPERSON' AND salesperson_id IS NOT NULL) |

**sessions**

| 列 | 类型 | 空 | 键 | 说明 |
|---|---|---|---|---|
| id | BIGINT UNSIGNED | NN | PK | |
| user_id | BIGINT UNSIGNED | NN | FK→users | |
| token_hash | CHAR(64) | NN | UQ | 会话令牌的 **SHA-256 摘要**（不存令牌本身） |
| csrf_hash | CHAR(64) | NN | | CSRF 双重提交摘要 |
| created_at / last_seen_at | DATETIME(3) | NN | | 创建与最近活跃（空闲 30 分钟过期） |
| expires_at | DATETIME(3) | NN | | 绝对过期（12 小时） |
| revoked_at | DATETIME(3) | √ | | 主动撤销（登出/改密/停用） |

### 4.4.2.3 交易（2 张）

**sales**

| 列 | 类型 | 空 | 键 | 说明 |
|---|---|---|---|---|
| id | BIGINT UNSIGNED | NN | PK | |
| sale_no | VARCHAR(40) | √ | UQ | 业务单号（过账时生成） |
| shift_id / event_id | BIGINT UNSIGNED | NN | 复合 FK→shifts(id,event_id) | **双关联，归属不可篡改** |
| state | ENUM(DRAFT,CHECKOUT,POSTED,CANCELLED) | NN | | 默认 DRAFT |
| created_at / prepared_at / posted_at | DATETIME(3) | √/NN | | 创建、准备收款、过账时点 |
| created_by | BIGINT UNSIGNED | NN | FK→users | |
| posted_by | BIGINT UNSIGNED | √ | FK→users | |
| payment_method | ENUM(CASH,TERMINAL) | √ | | 线下收款方式 |
| payment_reference | VARCHAR(120) | √ | | 终端参考号 |
| cash_received_cents | BIGINT UNSIGNED | √ | | 现金实收（分） |

| 列 | 类型 | 空 | 键 | 说明 |
| --- | --- | --- | --- | --- |
| net_cents / tax_cents / total_cents | BIGINT UNSIGNED | NN | | 合计（分），默认 0 |
| version | INT UNSIGNED | NN | | 默认 1 |
| — | — | — | IX | (event_id,posted_at)、(shift_id,posted_at) 供报表扫描 |

**sale_lines**（弱实体）

| 列 | 类型 | 空 | 键 | 说明 |
| --- | --- | --- | --- | --- |
| sale_id | BIGINT UNSIGNED | NN | PK, FK→sales | |
| line_no | SMALLINT UNSIGNED | NN | PK | 行号 |
| product_id | BIGINT UNSIGNED | NN | FK→products | |
| quantity | INT UNSIGNED | NN | CK | BETWEEN 1 AND 9999 |
| final_unit_price_cents | BIGINT UNSIGNED | NN | | 成交单价（分） |
| line_net_cents / line_tax_cents | BIGINT UNSIGNED | NN | | 行净额/税额 |
| line_commission_cents | BIGINT UNSIGNED | NN | | 行佣金 |
| cost_snapshot_cents | BIGINT UNSIGNED | √ | | **成本快照** |
| min_snapshot_cents | BIGINT UNSIGNED | √ | | **底价快照** |
| tax_bps_snapshot / commission_bps_snapshot | SMALLINT UNSIGNED | √ | | **费率快照** |
| name_snapshot / sku_snapshot | VARCHAR(160/64) | √ | | **名称与 SKU 快照** |
| — | — | — | IX | (product_id,sale_id) |

### 4.4.2.4 库存（6 张）

**stock_locations**

| 列 | 类型 | 空 | 键 | 说明 |
| --- | --- | --- | --- | --- |
| id | BIGINT UNSIGNED | NN | PK | |
| code | VARCHAR(80) | NN | UQ | 如 MAIN |
| name | VARCHAR(120) | NN | | 如 主仓 |
| kind | ENUM(WAREHOUSE,BOOTH) | NN | | |
| booth_id | BIGINT UNSIGNED | √ | FK→booths, UQ | 展位库位必关联展位 |
| active | BOOLEAN | NN | | |
| — | — | — | CK | (kind='WAREHOUSE' AND booth_id IS NULL) OR (kind='BOOTH' AND booth_id IS NOT NULL) |

**stock_balances**（可重算缓存）

| 列 | 类型 | 空 | 键 | 说明 |
| --- | --- | --- | --- | --- |
| location_id | BIGINT UNSIGNED | NN | PK, FK→stock_locations | |
| product_id | BIGINT UNSIGNED | NN | PK, FK→products | |
| on_hand | BIGINT UNSIGNED | NN | | 在手量 |
| reserved | BIGINT UNSIGNED | NN | | 已占用量（可售 = on_hand − reserved） |
| version | INT UNSIGNED | NN | | 乐观并发 |
| — | — | — | CK | on_hand >= reserved |

**stock_documents**

| 列 | 类型 | 空 | 键 | 说明 |
| --- | --- | --- | --- | --- |
| id | BIGINT UNSIGNED | NN | PK | |
| kind | ENUM(RECEIVE,TRANSFER,ADJUST,RETURN) | NN | | 单据类型 |
| from_location_id / to_location_id | BIGINT UNSIGNED | √ | FK→stock_locations | 收货只有 to；盘点按方向 |
| reason | VARCHAR(500) | NN | | **必填理由**，不可为空 |

| 列 | 类型 | 空 | 键 | 说明 |
| --- | --- | --- | --- | --- |
| created_by | BIGINT UNSIGNED | NN | FK→users | |
| created_at | DATETIME(3) | NN | | |

**stock_document_lines**

| 列 | 类型 | 空 | 键 | 说明 |
| --- | --- | --- | --- | --- |
| document_id / line_no | BIGINT / SMALLINT UNSIGNED | NN | PK | 复合主码 |
| product_id | BIGINT UNSIGNED | NN | FK→products | |
| quantity | INT | NN | CK | 带符号，<> 0 |

**stock_ledger**（权威流水，只增不改）

| 列 | 类型 | 空 | 键 | 说明 |
| --- | --- | --- | --- | --- |
| id | BIGINT UNSIGNED | NN | PK | |
| location_id / product_id | BIGINT UNSIGNED | NN | FK | |
| delta | BIGINT | NN | CK | 带符号增减，<> 0 |
| source_kind | ENUM(DOCUMENT, SALE,RETURN) | NN | | 来源类型 |
| source_id / source_line_no | BIGINT / SMALLINT UNSIGNED | NN | | 来源定位 |
| created_at | DATETIME(3) | NN | | |
| — | — | — | UQ | (source_kind, source_id, source_line_no, location_id) —— **同一来源不重复入账** |
| — | — | — | IX | (location_id, product_id) |

**stock_holds**（待收款占用）

| 列 | 类型 | 空 | 键 | 说明 |
| --- | --- | --- | --- | --- |
| sale_id + line_no | BIGINT / SMALLINT UNSIGNED | NN | PK, 复合 FK→sale_lines(sale_id,line_no) | 与销售行一一对应 |
| location_id / product_id | BIGINT UNSIGNED | NN | FK | |
| quantity | INT UNSIGNED | NN | | |

| 列 | 类型 | 空 | 键 | 说明 |
| --- | --- | --- | --- | --- |
| status | ENUM(ACTIVE,CONSUMED,RELEASED) | NN | | 默认 ACTIVE |

### 4.4.2.5 退货（2 张）

**returns**

| 列 | 类型 | 空 | 键 | 说明 |
| --- | --- | --- | --- | --- |
| id | BIGINT UNSIGNED | NN | PK | |
| sale_id | BIGINT UNSIGNED | NN | FK→sales | 原单 |
| state | ENUM(DRAFT,AUTHORIZED,POSTED,CANCELLED) | NN | | 默认 DRAFT |
| reason | VARCHAR(500) | NN | | 必填 |
| refund_reference | VARCHAR(120) | √ | | 线下退款参考号 |
| destination_location_id | BIGINT UNSIGNED | √ | FK→stock_locations | 良品回库目标 |
| created_by | BIGINT UNSIGNED | NN | FK→users | |
| authorized_at / posted_at | DATETIME(3) | √ | | |
| net_cents / tax_cents / total_cents | BIGINT UNSIGNED | NN | | 默认 0 |
| created_at | DATETIME(3) | NN | | |

**return_lines**

| 列 | 类型 | 空 | 键 | 说明 |
| --- | --- | --- | --- | --- |
| return_id + line_no | BIGINT / SMALLINT UNSIGNED | NN | PK | |
| sale_line_no | SMALLINT UNSIGNED | NN | UQ(return_id,sale_line_no) | 同一退货单中一行只退一次 |
| quantity | INT UNSIGNED | NN | CK | > 0 |
| disposition | ENUM(RESTOCK,DAMAGED) | NN | | 良品回库 / 损坏不入可售 |
| net_cents / tax_cents / commission_cents | BIGINT UNSIGNED | NN | | 按原单分摊 |

### 4.4.2.6 佣金（4 张）

**commission_entries**（只增）

| 列 | 类型 | 空 | 键 | 说明 |
|---|---|---|---|---|
| id | BIGINT UNSIGNED | NN | PK | |
| salesperson_id | BIGINT UNSIGNED | NN | FK→salespeople | |
| amount_cents | BIGINT | NN | CK | 带符号（销售正、退货负），<> 0 |
| source_kind | ENUM(SALE,RETURN) | NN | | |
| source_id / source_line_no | BIGINT / SMALLINT UNSIGNED | NN | UQ | 同一来源只计提一次 |
| created_at | DATETIME(3) | NN | IX(salesperson_id,created_at) | |

**commission_settlements**

| 列 | 类型 | 空 | 键 | 说明 |
|---|---|---|---|---|
| id | BIGINT UNSIGNED | NN | PK | |
| salesperson_id | BIGINT UNSIGNED | NN | FK→salespeople | |
| state | ENUM(PREPARED,PAID,CANCELLED) | NN | | 默认 PREPARED |
| cutoff_at | DATETIME(3) | NN | | 结算截止时点 |
| total_cents | BIGINT | NN | CK | > 0 |
| payment_reference | VARCHAR(120) | √ | | 线下付款参考 |
| created_by | BIGINT UNSIGNED | NN | FK→users | |
| created_at / paid_at | DATETIME(3) | NN/√ | | |

**commission_settlement_lines**

| 列 | 类型 | 空 | 键 | 说明 |
|---|---|---|---|---|
| settlement_id / entry_id | BIGINT UNSIGNED | NN | PK, FK | 复合主码保证一条目不在同一结算中重复 |
| amount_cents | BIGINT | NN | | 该条目计入结算的金额 |

**commission_claims**（占用表）

| 列 | 类型 | 空 | 键 | 说明 |
|---|---|---|---|---|
| entry_id | BIGINT UNSIGNED | NN | **PK**, FK→commission_entries | **主键即保证一条目只能被一个结算占用** |

| 列 | 类型 | 空 | 键 | 说明 |
| --- | --- | --- | --- | --- |
| settlement_id | BIGINT UNSIGNED | NN | FK→commission_settlements | |

#### 4.4.2.7 平台（3 张）

**idempotency_keys**

| 列 | 类型 | 空 | 键 | 说明 |
| --- | --- | --- | --- | --- |
| actor_id / operation / command_key | BIGINT / VARCHAR(80) / CHAR(36) | NN | **PK** | 操作者+操作+UUID 键 |
| request_hash | CHAR(64) | NN | | 请求体摘要，不一致返回 409 |
| response_json | LONGTEXT | NN | | 首次响应，命中即原样返回 |
| created_at | DATETIME(3) | NN | | |

**audit_events**

| 列 | 类型 | 空 | 键 | 说明 |
| --- | --- | --- | --- | --- |
| id | BIGINT UNSIGNED | NN | PK | |
| actor_id | BIGINT UNSIGNED | √ | FK→users | 空表示系统动作 |
| action | VARCHAR(100) | NN | | 如 sale.post |
| subject_type / subject_id | VARCHAR(60) / BIGINT | NN/√ | | 对象类型与标识 |
| detail_json | LONGTEXT | √ | | 明细快照 |
| created_at | DATETIME(3) | NN | IX | 按时间检索 |

**schema_migrations**

| 列 | 类型 | 空 | 键 | 说明 |
| --- | --- | --- | --- | --- |
| version | VARCHAR | NN | PK | 迁移文件名（按序） |
| checksum_sha256 | CHAR(64) | NN | | 文件内容校验和，变动即拒绝启动 |
| applied_at | DATETIME(3) | NN | | |

### 4.4.3 索引策略

| 类别 | 索引 | 目的 |
| --- | --- | --- |
| 报表扫描 | sales(event_id, posted_at)、sales(shift_id, posted_at) | 按活动/班次汇总已过账销售 |

| 类别 | 索引 | 目的 |
| --- | --- | --- |
| 商品维度 | sale_lines(product_id, sale_id) | 按商品汇总 |
| 班次查询 | shifts(salesperson_id, business_date) | 员工日历视图 |
| 佣金查询 | commission_entries(salesperson_id, created_at) | 员工计提流水 |
| 审计查询 | audit_events(created_at) | 按时间倒序 |
| 库存查询 | stock_ledger(location_id, product_id) | 库位+商品流水 |
| 归属链 | booths(id,event_id)、shifts(id,event_id) | 复合外键的被引用端唯一键 |

**未建索引的列**：低基数的 active/status 单列索引未建（选择性差，收益低于写放大代价）；审计与流水表按时间范围查询已由主时间索引覆盖。

### 4.4.4 约束清单（CHECK / 业务规则下沉）

| 约束 | 表达式要点 | 保障的规则 |
| --- | --- | --- |
| ck_event_dates | end_date >= start_date | 活动日期合法 |
| ck_event_tax | tax_bps IS NULL OR <= 10000 | 税率不超过 100% |
| ck_product_prices | suggested >= min | 建议价不低于底价 |
| ck_commission_rate | commission_bps IS NULL OR <= 10000 | 佣金率上限 |
| ck_shift_times | planned_end > planned_start | 班次时段合法 |
| ck_user_salesperson_role | 角色与档案绑定的异或 | OWNER 无档案、销售员必有档案 |
| ck_line_qty | quantity BETWEEN 1 AND 9999 | 行数量合理 |
| ck_balance_reserved | on_hand >= reserved | **可售量非负** |
| ck_docline_nonzero | quantity <> 0 | 单据行不为零 |
| ck_ledger_nonzero | delta <> 0 | 流水不写空操作 |
| ck_returnline_qty | quantity > 0 | 退货量为正 |
| ck_commission_nonzero | amount_cents <> 0 | 计提不写零 |
| ck_settlement_positive | total_cents > 0 | 结算金额为正 |
| ck_location_kind | 库位类型与展位绑定的异或 | 主仓不挂展位、展位库位必挂展位 |

**已成交业务引用的主数据不使用级联删除**：所有外键均为 RESTRICT 语义，防止删除场地/商品导致历史单据悬空。

---

## 4.5 数据完整性与不变量

---

不变量由 `db/check-invariants.sql` 实现，`npm run db:check` 执行；**任一差异非 0 即退出非 0**。当前基线：16 项全为 0。

| 编号 | 不变量 | 断言 |
|---|---|---|
| I-01 | 销售-班次活动一致 | `sales.event_id = shifts.event_id` |
| I-02 | 班次-展位活动一致 | `shifts.event_id = booths.event_id` |
| I-03 | 余额等于流水汇总 | `stock_balances.on_hand = SUM(stock_ledger.delta)` |
| I-04 | 流水必有余额 | 每条流水的 `(location,product)` 在余额表中存在 |
| I-05 | 占用等于有效 hold | `stock_balances.reserved = SUM(ACTIVE holds.quantity)` |
| I-06 | 可售量非负 | `on_hand >= reserved` |
| I-07 | 已过账销售金额一致 | `net/tax/total` = 明细汇总，`total = net + tax` |
| I-08 | 已过账销售必有库存流水 | 每条已过账销售行存在 `delta = −quantity` 的 SALE 流水 |
| I-09 | 已过账销售必有佣金 | 行佣金 > 0 时存在金额一致的 SALE 计提 |
| I-10 | 已过账退货金额一致 | `net/tax/total` = 明细汇总 |
| I-11 | 退货不超原销量 | 累计退货数 ≤ 原行数量 |
| I-12 | 退货行指向真实原行 | `return_lines.sale_line_no` 在原单中存在 |
| I-13 | 已过账退货必有回库流水 | `RESTOCK` 行存在 `delta = quantity` 的 RETURN 流水 |
| I-14 | 已过账退货必有负佣金 | 退货佣金 > 0 时存在金额相反的 RETURN 计提 |
| I-15 | 取消结算无占用 | `CANCELLED` 结算在 `commission_claims` 中无记录 |
| I-16 | 结算金额一致 | `settlements.total_cents = SUM(settlement_lines.amount_cents)` |

**运行建议**：日常收档后执行一次；发现异常应立即停止新交易、备份当前卷并保留现场（见 07 部署运维）。

---

## 4.6 迁移、种子数据与恢复

### 4.6.1 迁移规程

| 规则 | 说明 |
| --- | --- |
| 顺序 | 按文件名升序执行（001_core.sql 之后按编号递增） |
| 校验和 | 每个文件应用后记录 SHA-256 到 schema_migrations；**已应用文件内容变动将拒绝启动** |
| 不可修改 | 已应用的迁移文件不得编辑；变更须**新增编号迁移** |
| 隐式提交 | MariaDB 的 DDL 会隐式提交，因此**不能靠单一事务回滚整份 DDL**；迁移前必须备份，失败即停止并修复或从受控备份恢复 |
| 执行者 | 本地由一次性 migrate 容器以管理员身份执行，随后收紧运行账号权限；云端由 scripts/vercel-setup.mjs 幂等执行 |
| 权限收紧 | 运行账号仅对必要表拥有 UPDATE/DELETE；**库存流水、佣金流水、审计表不授 UPDATE/DELETE** |

### 4.6.2 种子数据

| 种子 | 内容 | 来源 |
| --- | --- | --- |
| 活动类别 | 7 类（trade show / rib fest / waterfront festival / music festival / beer festival / sporting event / street festival） | 随 001_core.sql 插入 |
| 主仓库位 | MAIN / 主仓 / WAREHOUSE | 随 001_core.sql 插入 |
| 首位管理员 | owner 账号 + 随机临时口令（≥32 字符） | 启动脚本生成，首次登录强制改密 |
| 样例业务数据 | 七类活动、17 名员工、30 个商品、300 笔已过账销售及取消/待收款/退货/已付佣金 | **独立**执行 npm run seed:sample，不随正常启动导入；**全部为合成教学数据** |

### 4.6.3 备份与恢复

- **备份窗口**：安排停写窗口后执行逻辑导出，文件仅留在受控本机磁盘并加密、限制访问；
- **恢复验证**：必须先在**独立空卷**验证（表数、单据数、16 项不变量），再制定正式恢复步骤；
- **持久化**：db_data 为命名卷，docker compose down 不删除数据；**禁止 down -v** 作为”修复”手段；
- **导出命令**见 07 部署运维 第 7.4 节（Windows 需注意重定向编码，建议用 sh 或二进制安全方式）。

---

## 4.7 可追踪性

### 4.7.1 案例实体 → 物理表

完整映射见 §2.1；追踪文件见 traceability.csv（CASE-01~12 为案例要素，DERIVED-* 为派生设计，EXT-* 为运行扩展）。

### 4.7.2 需求 → 表

| 需求 | 主要表 |
| --- | --- |
| FR-01 认证 | users, sessions |
| FR-02/03 场地活动展位 | venues, events, booths, event_categories |
| FR-04 商品 | products |
| FR-05 员工班次 | salespeople, shifts, users |
| FR-06/07 库存 | stock_locations, stock_documents, stock_document_lines, stock_balances, stock_ledger |
| FR-08/09 销售 | sales, sale_lines, stock_holds |
| FR-10 退货 | returns, return_lines |
| FR-11 佣金 | commission_entries, commission_settlements, commission_settlement_lines, commission_claims |
| FR-12 报表 | 只读查询（上述表） |
| NFR-REL 幂等 | idempotency_keys |
| NFR-AUD 审计 | audit_events |

---

# 附录：SQL 实现示例

以下片段摘自权威 DDL `db/migrations/001_core.sql`，用于说明实现手法；**实际执行以迁移文件为准。**

## A.1 复合外键链（DDL）

```
CREATE TABLE booths (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  event_id BIGINT UNSIGNED NOT NULL,
  location VARCHAR(80) NOT NULL,
  active BOOLEAN NOT NULL DEFAULT TRUE,
  UNIQUE KEY uq_booth_event_location (event_id,location),
  UNIQUE KEY uq_booth_id_event (id,event_id),          -- 复合外键的被引用端
  CONSTRAINT fk_booths_event FOREIGN KEY (event_id) REFERENCES events(id)
) ENGINE=InnoDB;

CREATE TABLE shifts (
  ...
  UNIQUE KEY uq_shift_id_event (id,event_id),
  CONSTRAINT fk_shift_booth_event FOREIGN KEY (booth_id,event_id)
    REFERENCES booths(id,event_id),                     -- 展位与活动必须同源
  ...
) ENGINE=InnoDB;
```

## A.2 库存流水唯一性（防重复入账）

```
CONSTRAINT ... UNIQUE KEY uq_stock_source (source_kind,source_id,source_line_no,location_id)
```

同一来源（单据/销售/退货）的同一行在同一库位只能入账一次——这是”重试不产生重复流水”的数据库级保证。

## A.3 佣金占用（一笔条目只属一个结算）

```
CREATE TABLE commission_claims (
  entry_id BIGINT UNSIGNED NOT NULL PRIMARY KEY,        -- 主键即唯一占用
  settlement_id BIGINT UNSIGNED NOT NULL,
  CONSTRAINT fk_claim_entry FOREIGN KEY (entry_id) REFERENCES commission_entries(id),
  CONSTRAINT fk_claim_settlement FOREIGN KEY (settlement_id) REFERENCES commission_settlements(id)
) ENGINE=InnoDB;
```

## A.4 不变量核查（DML）

```
SELECT 'balance_ledger_difference' AS issue, COUNT(*) AS violations

FROM stock_balances b

LEFT JOIN (SELECT location_id,product_id,SUM(delta) AS quantity

           FROM stock_ledger GROUP BY location_id,product_id) l

  ON l.location_id=b.location_id AND l.product_id=b.product_id

WHERE b.on_hand <> COALESCE(l.quantity,0);
```

## A.5 事务骨架（应用层约定）

```
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;

START TRANSACTION;

SELECT ... FROM idempotency_keys WHERE ... FOR UPDATE;   -- 1) 幂等行最先

SELECT ... FROM events         WHERE id=? FOR UPDATE;    -- 2) 活动

SELECT ... FROM shifts/sales   WHERE id=? FOR UPDATE;    -- 3) 班次与单据

SELECT ... FROM products       WHERE id=? FOR UPDATE;    -- 4) 商品（重读底价）

SELECT ... FROM stock_balances WHERE ...  FOR UPDATE;    -- 5) 余额最后

-- 业务写入 ...

COMMIT;
```

固定加锁顺序是避免死锁的关键。`db.ts` 的 `tx()` 在捕获 `ER_LOCK_DEADLOCK`（errno 1213）、`ER_LOCK_WAIT_TIMEOUT`（1205）与 `ER_CHECKREAD`（1020）时回滚，随机退避 20–70 ms 后重试，最多执行 3 次尝试（首次 + 最多 2 次重试）；其余错误不重试，直接抛出。

---

**文档结束。** 上游设计见 03 SDD，测试与不变量验证见 05 STD。

> 作业文档与图纸仓库：https://github.com/xingchenyd/foxcore-retail —— 文档位于 `docs/`，图纸位于 docs/diagrams（实体类图 class-diagram.png/svg、物理 ERD physical-erd.png、概念 ERD core-barker.png/svg、架构图、业务序列图、用例图、状态图等全套 .puml/.svg/.png）。

# 05 软件测试与验收说明（STD）

**Foxcore Retail — Software Test Description & Acceptance**

## 5.1 引言

### 5.1.1 目的

本文档定义 Foxcore Retail 的**测试策略、测试用例与执行结果**，用于：

1. 证明系统满足 SRS 中定义的功能与非功能需求；
2. 提供可复现的验收步骤，供首次启动与周期性回归使用；
3. 如实记录**未执行**的测试项及其原因，避免将”未测”误读为”通过”。

### 5.1.2 范围

覆盖：金额计算单元测试、HTTP+数据库集成测试、数据不变量核查、浏览器端到端测试、类型检查与生产构建、依赖安全审计、Compose 配置静态校验、发布包完整性复验。

不覆盖：性能基准（10000 条规模）、跨平台镜像构建、容器端到端启动（见 §10）、渗透测试与代码审计工具扫描。

### 5.1.3 定义与缩写

| 术语 | 定义 |
| --- | --- |
| **UT** | Unit Test，单元测试（本项目的金额与格式计算） |
| **TC** | Test Case，本报告中的集成/系统级验收用例 |
| **IV** | Invariant Verification，数据完整性核查项 |
| **E2E** | End-to-End，浏览器端到端测试 |
| **Pass / Fail / Blocked** | 通过 / 失败 / 因环境或权限受限无法执行 |
| **合成数据** | 为测试构造的虚构业务数据，非课程案例人物与真实交易 |

### 5.1.4 参考文献

| 编号 | 文献 |
| --- | --- |
| REF-01 | 02 SRS —— 验收准则来源 |
| REF-02 | 03 SDD —— 设计约定（加锁顺序、幂等、快照） |
| REF-03 | 04 数据库设计 —— 16 项不变量定义 |
| REF-04 | 07 部署运维 —— 环境准备与验收检查表 |

| 编号 | 文献 |
| --- | --- |
| REF-05 | `tests/unit/money.test.ts`、`tests/integration/workflow.test.ts`、`tests/e2e/workbench.spec.ts` |

### 5.1.5 概述

测试分六个层次自下而上执行：**静态类型与构建 → 金额计算单元 → HTTP+数据库集成 → 数据不变量 → 浏览器端到端 → 配置与发布完整性。**已执行项全部通过；容器端到端启动因本机 Docker 服务不可用而列为受阻（§10），需在具备容器环境时补验。

---

## 5.2 测试策略

### 5.2.1 测试层级

| 层级 | 方法 | 工具 | 验收标准 |
| --- | --- | --- | --- |
| L1 静态 | TypeScript 严格模式类型检查；Vite/tsc 生产构建 | `tsc`, `vite build` | 无错误 |
| L2 计算 | 金额解析、格式化、half-up 舍入、退款分摊 | Vitest | 全部断言通过 |
| L3 事务 | HTTP + 真实 MariaDB：并发抢库存、幂等重试、销售/退货/佣金 | Vitest + fetch | 无超卖、无重复过账；预期拒绝为 409 |
| L4 数据 | 16 项不变量 SQL 核查 | `npm run db:check` | 所有差异数为 0 |
| L5 浏览器 | Chromium 桌面与 iPhone 13 视口 | Playwright | 登录、导航、刷新成功，无页面异常 |
| L6 配置与发布包 | Compose 静态解析；依赖审计；发布包 SHA-256 复验 | `docker compose config`, `npm audit`, 校验脚本 | 解析成功、0 已知漏洞、逐文件校验通过 |

### 5.2.2 准入与准出准则

**准入（Entry）**

- 应用在 `127.0.0.1:3000` 可访问，`/health/ready` 返回 `status: ready` 与迁移版本；
- `TEST_CONFIRM=yes` 且 `TEST_BASE_URL` 为 localhost（集成测试的安全前置）；
- 管理员口令通过 `TEST_OWNER_PASSWORD` 或 `TEST_OWNER_PASSWORD_FILE` **私有**提供（不写入仓库或发布包）。

**准出（Exit）**

- L1–L6 全部通过；
- 16 项不变量差异全为 0；
- 无未关闭的阻塞级缺陷；
- 已知限制已记录于 §10。

### 5.2.3 测试数据策略

- **全部为合成数据**：由 `npm run seed:sample` 或测试脚本经 HTTP API 创建；
- 课程案例中的人物与交易**未导入**；
- 集成测试使用随机 UUID 前缀隔离数据（如 `INT-<tag>`），避免相互污染；
- 生产/演示库禁止执行种子脚本（见 07）。

---

## 5.3 测试环境与工具

| 项 | 版本/配置 |
| --- | --- |
| 操作系统 | Windows（本机开发机） |
| 运行时 | Node 24（项目 `.node-version` 约束 ≥ 24） |
| 数据库 | MariaDB 11.8.9（独立本地实例） |
| 浏览器 | Chromium（Playwright 托管） |
| 测试框架 | Vitest（单元与集成）、Playwright（E2E） |
| 关键命令 | `npm run typecheck`、`npm run test:unit`、`npm run test:integration`、`npm run build`、`npm run db:check`、`npm run test:e2e` |

**安全前置说明**：集成测试脚本会校验 `TEST_BASE_URL` 必须匹配 `^http://(localhost|127.0.0.1):\d+$` 且 `TEST_CONFIRM=yes`，防止误对远程环境执行破坏性操作；E2E 测试口令从 `TEST_OWNER_PASSWORD_FILE` 指向的私有文件读取，该文件**不得进入发布包**。

---

## 5.4 单元测试

**对象**：`apps/server/src/amounts.ts`（`halfUp`、`parseMoney`/`money`、`moneyText`）。 **文件**：`tests/unit/money.test.ts`。 **结果**：**10/10 通过**。

| ID | 用例 | 断言要点 | 结果 |
| --- | --- | --- | --- |
| UT-01 | 解析精确分与格式化 | `'0'→0n; '12.3'→1230n; '9007199254071992547.01'→900719925470101n; moneyText(-1234n)→'-12.34'` | Pass |
| UT-02 | 拒绝 `1e2` | 抛异常 | Pass |
| UT-03 | 拒绝 `1,000.00`（千分位） | 抛异常 | Pass |
| UT-04 | 拒绝 `0.001`（超过分精度） | 抛异常 | Pass |
| UT-05 | 拒绝 `-1`（负金额输入） | 抛异常 | Pass |
| UT-06 | 拒绝 `+1` | 抛异常 | Pass |

| ID | 用例 | 断言要点 | 结果 |
| --- | --- | --- | --- |
| UT-07 | 拒绝 `true`（布尔） | 抛异常 | Pass |
| UT-08 | 拒绝 `[]`（数组） | 抛异常 | Pass |
| UT-09 | 拒绝 `null` | 抛异常 | Pass |
| UT-10 | half-up 舍入与退款分摊 | `halfUp(1005n*1300n)=131n`；原税 2 分、3 件的三次分摊为 `[1,0,1]` 且合计 = 原税（**不丢分、不多退**） | Pass |

**UT-10 的意义**：多次部分退款时，按累计比例分摊原税额并逐次求差，保证分摊序列之和**恰好等于**原始税额，避免”多退一分”或”少退一分”的累积误差。

---

## 5.5 集成与系统测试用例

**对象**：真实 MariaDB + Fastify HTTP API。 **文件**：`tests/integration/workflow.test.ts`（1 个复合场景，超时 120 s）。 **环境前置**：`TEST_CONFIRM=yes`、`TEST_BASE_URL=http://127.0.0.1:3000`、`TEST_OWNER_PASSWORD`。

| ID | 测试目标 | 前置条件 | 操作步骤 | 预期结果 | 关联需求 | 结果 |
| --- | --- | --- | --- | --- | --- | --- |
| **TC-001** | 管理员登录与角色识别 | owner 账号已激活且已改密 | `POST /auth/login` | 200；`user.role='OWNER'`；`mustChangePassword=false`；返回 CSRF 令牌 | FR-01 | Pass |
| **TC-002** | 主数据创建链路 | 已登录 OWNER | 依次创建场地 → 取首个类别 → 创建活动（`taxBps=1300`，`policyConfirmed=true`）→ 创建展位 → 开放活动 → 创建商品（成本 4.00 / 底价 8.00 / 建议价 10.00）→ 创建员工（`commissionBps=500`）→ 创建销售员账号 → 创建班次 → 开班 | 全部 2xx；活动状态 OPEN；班次 ACTIVE | FR-02~FR-05 | Pass |
| **TC-003** | 底价拦截 | 商品底价 8.00 | 以单价 `7.99` 建单并调用 `prepare` | **409**（低于最低成交价） | FR-08, BR-价格 | Pass |

| ID | 测试目标 | 前置条件 | 操作步骤 | 预期结果 | 关联需求 | 结果 |
|---|---|---|---|---|---|---|
| **TC-004** | 数量合法性 | 同上 | 建单 `quantity=0` | **400** | FR-08 | Pass |
| **TC-005** | 收货与调拨 | 主仓 `MAIN` 与展位库位存在 | 收货 1 件入主仓；调拨 1 件至展位（`handoverConfirmed=true`） | 两端余额与流水相抵；展位可售量 1 | FR-06 | Pass |
| **TC-006** | **并发抢最后一件** | 展位可售量 = 1 | 并行创建 20 张草稿，再并行 `prepare` | **恰 1 个 200，19 个 409**；无超卖 | NFR-PERF-02 | Pass |
| **TC-007** | **幂等重试不重复过账** | TC-006 中成功的单据 | 用**同一** `Idempotency-Key` 并行 20 次 `post`（现金实收 12.00） | 20 次均 200 且响应**完全相同**（集合大小 = 1）；仅一份库存流水与佣金流水 | NFR-REL-01 | Pass |
| **TC-008** | 同键异体请求被拒 | 同上 | 用同一键提交**不同**请求体（实收 13.00） | **409**（请求摘要不一致） | NFR-REL-01 | Pass |
| **TC-009** | 金额计算正确 | 单价 10.00、税率 1300 bps | 查询成交单详情 | `total_cents = '1130'`（1000 + 130 税） | FR-09 | Pass |
| **TC-010** | **角色隔离** | 销售员账号登录 | 分别请求 `/salespeople`、`/audit`、`/stock/locations` | 均 **403** | NFR-SEC-02 | Pass |
| **TC-011** | **成本不泄露** | 同上 | 销售员请求 `/products` | 目标商品的 `wholesale_cost_cents` **为 undefined** | NFR-SEC-03 | Pass |
| **TC-012** | 本人单据可见 | 同上 | 销售员请求其参与的单据详情 | 200（按 `salesperson_id` 限定范围内） | NFR-SEC-02 | Pass |
| **TC-013** | 毛利口径隔离 | 同上 | 销售员请求 `/reports/sales?group=product` | 目标行 `estimated_margin_cents` **为 undefined** | NFR-SEC-03 | Pass |
| **TC-014** | 退货授权与过账 | TC-007 已过账单 | 建退货单（原行、1 件、RESTOCK、目标主仓）→ 授权 → 过账 | 授权 `totalCents='1130'`；过账 200；生成负向佣金与回库流水 | FR-10 | Pass |

| ID | 测试目标 | 前置条件 | 操作步骤 | 预期结果 | 关联需求 | 结果 |
|---|---|---|---|---|---|---|
| **TC-015** | 报表口径（已过账减已过账退货） | 该活动仅 1 笔销售且已全退 | `GET /reports/sales?group=event` | 该活动 `net_cents='0'` | FR-12 | Pass |
| **TC-016** | 未知路由处理 | 已登录 | `GET /api/v1/no-such-route` | **404** | NFR-错误模型 | Pass |

## 5.6 数据完整性测试

**命令**：`npm run db:check`（执行 `db/check-invariants.sql`）。 **执行时机**：样例数据导入后 + 并发测试后，在真实 MariaDB 上执行。 **结果**：**16/16 差异为 0。**

| ID | 核查项 | 预期 | 结果 |
|---|---|---|---|
| IV-01 | 销售-班次活动一致 | 0 | Pass |
| IV-02 | 班次-展位活动一致 | 0 | Pass |
| IV-03 | 余额 = 流水汇总 | 0 | Pass |
| IV-04 | 流水必有余额 | 0 | Pass |
| IV-05 | 占用 = 有效 hold 汇总 | 0 | Pass |
| IV-06 | 可售量非负 | 0 | Pass |
| IV-07 | 已过账销售金额一致 | 0 | Pass |
| IV-08 | 已过账销售必有库存流水 | 0 | Pass |
| IV-09 | 已过账销售必有佣金 | 0 | Pass |
| IV-10 | 已过账退货金额一致 | 0 | Pass |
| IV-11 | 退货不超原销量 | 0 | Pass |
| IV-12 | 退货行指向真实原行 | 0 | Pass |
| IV-13 | 已过账退货必有回库流水 | 0 | Pass |
| IV-14 | 已过账退货必有负佣金 | 0 | Pass |
| IV-15 | 取消结算无占用 | 0 | Pass |
| IV-16 | 结算金额 = 明细合计 | 0 | Pass |

完整定义见 04 第 4.5 节。

## 5.7 浏览器与界面测试

**工具**：Playwright + Chromium；视口：桌面与 iPhone 13。 **结果**：**2/2 通过**（`tests/e2e/workbench.spec.ts`）。

| ID | 步骤 | 预期 | 结果 |
| --- | --- | --- | --- |
| E2E-01（桌面） | 打开 / → 填用户名/密码 → 点击”登录系统 →” | 显示”运营工作台”标题 | Pass |
| | 点击”进入销售工作台 →” | 显示”销售工作台”与”新建销售草稿” | Pass |
| | `page.reload()` | 刷新后仍显示”销售工作台”（会话保持） | Pass |
| | 导航至 `/reports` | 显示”经营报表” | Pass |
| | 点击”导出 CSV” | 触发下载，文件名 `foxcore-report.csv` | Pass |
| | 断言 `pageerror` 收集数组 | **为空**（无未捕获页面异常） | Pass |
| E2E-02（iPhone 13） | 同上流程，移动视口 | 同上；布局可用 | Pass |

**证据**：`docs/evidence/<project>-sales.png`（桌面与移动各一份）。

---

## 5.8 静态、安全与配置测试

| ID | 项目 | 命令/方式 | 预期 | 结果 |
| --- | --- | --- | --- | --- |
| ST-01 | 类型检查 | `npm run typecheck`（双工作区严格模式） | 无错误 | Pass |
| ST-02 | 生产构建 | `npm run build` | Vite 86 模块；前端 JS 约 299 KB 原始大小 | Pass |
| ST-03 | 依赖安全 | `npm audit`（2026-09-27） | 0 项已知漏洞 | Pass |
| ST-04 | Compose 静态校验 | `docker compose config --quiet` | 解析成功 | Pass |
| ST-05 | 迁移脚本幂等性 | 独立 MariaDB 上首次应用 / 再次校验 | 首次建表并收紧授权；第二次跳过 | Pass |
| ST-06 | 管理员初始化幂等 | `runtime-owner.mjs` 首次生成 / 第二次运行 | 首次生成临时口令；第二次跳过 | Pass |
| ST-07 | 发布包完整性 | 逐文件 SHA-256 校验；新目录 `npm ci` | 校验通过；`npm ci` 后 0 已知漏洞，类型检查/构建/10 项单测/Compose 校验全部通过 | Pass |
| ST-08 | 安全响应头 | 人工核对响应头 | CSP、`frame-ancestors`、`no-sniff`、API `Cache-Control: no-store` 均已设置 | Pass |

| ID | 项目 | 命令/方式 | 预期 | 结果 |
| --- | --- | --- | --- | --- |
| ST-09 | 错误模型 | 触发各类错误 | 统一 `{code,message,requestId}`；500 **不回显 SQL 栈** | Pass |

## 5.9 测试结果汇总

| 类别 | 总数 | 通过 | 失败 | 受阻 | 通过率 |
| --- | --- | --- | --- | --- | --- |
| 单元测试（UT） | 10 | 10 | 0 | 0 | 100% |
| 集成/系统用例（TC） | 16 | 16 | 0 | 0 | 100% |
| 数据不变量（IV） | 16 | 16 | 0 | 0 | 100% |
| 浏览器用例（E2E） | 2 | 2 | 0 | 0 | 100% |
| 静态安全配置（ST） | 9 | 9 | 0 | 0 | 100% |
| **合计** | **53** | **53** | **0** | **0** | **100%** |

**未执行（Blocked）项**见 §10，不计入通过率。

## 5.10 已知限制与未执行项

### 5.10.1 未执行项（诚实声明）

| 编号 | 项目 | 原因 | 建议补验时机 |
| --- | --- | --- | --- |
| BLK-01 | `docker compose up -d --build` 完整启动 | 本机 Docker Desktop 服务未运行，且当前 Windows 进程无权启动该服务 | 具备容器环境后的首次启动（按 07 检查表） |
| BLK-02 | 容器健康检查与自愈行为 | 依赖 BLK-01 | 同上 |
| BLK-03 | 备份与恢复实际演练 | 依赖容器环境 | 正式环境上线前，在独立空卷演练 |
| BLK-04 | 跨平台镜像构建（如 linux/arm64） | 本机无构建环境 | 如需在非 x86 主机部署时补验 |

**说明**：Compose **结构**、首次迁移/管理员脚本逻辑、以及本地真实 MariaDB 上的业务行为均已分别验证，但这**不能替代**容器端到端验收。

### 5.10.2 系统边界（非缺陷）

| 项 | 说明 |
| --- | --- |
| 资金渠道 | 系统登记线下收款、退款与佣金支付的**依据**，不与任何支付渠道自动核对 |
| 税率与佣金率 | 由管理员填写并确认，系统不校验其税务合规性 |
| 毛利口径 | 报表毛利为**商品标准成本估算**，不含活动费、差旅与工资 |
| 分页与规模 | 销售列表每页 100 笔（`sales.ts` 默认 `limit=100`）、目录类列表每页 20 笔（`catalog.ts` 默认 `limit=20`），两类上限均为 100；商品/活动等选择器最多载入 100 项；**未做** 10000 条规模性能测试 |
| 时区 | 单一时区 `America/Toronto`，不支持跨时区门店 |

### 5.10.3 缺陷记录

缺陷按下表登记：

| 字段 | 说明 |
| --- | --- |
| 缺陷编号 | `DEF-<序号>` |
| 关联用例 | TC / UT / E2E 编号 |
| 严重级别 | Blocker / Critical / Major / Minor |
| 复现步骤 | 环境 + 操作 + 数据 |
| 期望与实际 | — |
| 状态 | Open / Fixed / Retest / Closed |

---

## 5.11 可追踪矩阵

| 需求 | 测试用例 | 设计章节 | 状态 |
| --- | --- | --- | --- |
| FR-01 认证 | TC-001, E2E-01 | SDD 2.3.1 | Pass |
| FR-02/03 场地活动展位 | TC-002 | SDD 2.3.2 | Pass |
| FR-04 商品 | TC-002, TC-011 | SDD 2.3.2 | Pass |
| FR-05 员工班次 | TC-002 | SDD 2.3.2 | Pass |
| FR-06/07 库存 | TC-005, IV-03~IV-06 | SDD 2.3.3 | Pass |
| FR-08 销售草稿与准备 | TC-003, TC-004, TC-006 | SDD 2.3.4 | Pass |
| FR-09 过账与取消 | TC-007~TC-009, IV-07~IV-09 | SDD 2.3.4 | Pass |
| FR-10 退货 | TC-014, IV-10~IV-14 | SDD 2.3.5 | Pass |
| FR-11 佣金 | TC-014, IV-15, IV-16 | SDD 2.3.6 | Pass |
| FR-12 报表 | TC-015, E2E-01（CSV） | SDD 2.3.7 | Pass |

| 需求 | 测试用例 | 设计章节 | 状态 |
| --- | --- | --- | --- |
| NFR-PERF-02 并发不超卖 | TC-006 | SDD 4.4 | Pass |
| NFR-REL-01 重试幂等 | TC-007, TC-008 | SDD 2.3.8 | Pass |
| NFR-SEC-02 角色隔离 | TC-010, TC-012 | SDD 2.3.1 | Pass |
| NFR-SEC-03 成本不泄露 | TC-011, TC-013 | SDD 2.3.8 | Pass |
| NFR-ERR 错误模型 | TC-016, ST-09 | SDD 2.3.8 | Pass |
| NFR-AUD 审计 | ST-08 及代码审查 | SDD 2.3.8 | Pass（人工） |

---

## 附录：证据清单

| 文件/输出 | 内容 |
| --- | --- |
| `docs/evidence/*-sales.png` | 桌面与 iPhone 13 视口的销售工作台截图 |
| Vitest 输出 | 单元测试 10/10；集成场景通过 |
| `npm run db:check` 输出 | 16 项不变量差异全为 0 |
| `npm audit` 输出（2026-09-27） | 0 项已知漏洞 |
| `docker compose config --quiet` | 退出码 0 |
| SHA-256 校验结果 | 发布包逐文件一致 |

**复现命令**

```
npm run typecheck

npm run test:unit

npm run test:integration      # 需 TEST_CONFIRM=yes 与本地口令

npm run build

npm run db:check

npm run test:e2e              # 需 TEST_OWNER_PASSWORD_FILE

docker compose config --quiet
```

---

**文档结束。** 上游设计与数据定义见 03 SDD、04 数据库设计；部署与验收检查表见 07。

> 作业文档与图纸仓库：https://github.com/xingchenyd/foxcore-retail —— 文档位于 `docs/`，图纸位于 docs/diagrams（实体类图 class-diagram.png/svg、物理 ERD physical-erd.png、概念 ERD core-barker.png/svg、架构图、业务序列图、用例图、状态图等全套 .puml/.svg/.png）。

# 06 用户界面与操作流程手册（UI & Flow）

**Foxcore Retail — 用户操作手册**

## 6.1 引言

### 6.1.1 目的

本文档面向**最终使用者**，以”界面 + 流程”的方式说明如何使用 Foxcore Retail 完成一次完整的现场零售作业：从系统登录、开场准备、现场成交，到退货、佣金结算与报表核对。

### 6.1.2 使用约定

- 文档中 「」 表示界面上的按钮或链接名称；【】 表示页面或标签名称；
- 步骤编号即操作顺序，请勿跳步；
- 涉及**线下动作**（收款、退款、付款、实物交接）的步骤已单独标注，系统不会代替你完成这些动作；
- 本文档中所有金额单位为 **CAD（加元）**，界面按 CAD 显示，系统内部以”分”整数存储。

### 6.1.3 界面设计原则（摘要）

| 原则 | 在界面上的体现 |
| --- | --- |
| 角色可见即角色可用 | 销售员不渲染【退货与核对】【操作日志】入口 |
| 服务端为唯一真相 | 页面刷新即从服务端重取，不缓存金额；不保存登录令牌 |
| 错误可定位 | 错误提示附带 `requestId`，可与服务端日志对应 |
| 现场可用性 | 关键按钮尺寸适配移动端；目录类列表每页 20 条、销售列表每页 100 条；选择器上限 100 项 |
| 口径透明 | 报表标注”仅计已过账销售 − 已过账退货”；毛利标注为”标准成本估算” |

## 6.2 登录与首次设置

### 6.2.1 首次登录（管理员）

| 步骤 | 操作 | 界面反馈 |
| --- | --- | --- |
| 1 | 浏览器打开 `http://localhost:3000` | 显示登录页：用户名、密码输入框与「登录系统 →」按钮 |
| 2 | 获取初始密码：在本机终端运行 `docker compose exec app cat /run/foxcore-app/owner_initial_password` | 终端输出临时口令（≥32 字符） |

| 步骤 | 操作 | 界面反馈 |
| --- | --- | --- |
| 3 | 用户名填 owner，密码填步骤 2 的临时口令，点击「登录系统 →」 | 进入【运营工作台】；**系统强制要求修改密码** |
| 4 | 设置至少 **16 字符**的个人密码并确认 | 旧会话失效；密码文件在改密成功后自动删除 |
| 5 | 使用新密码重新登录 | 正常进入系统 |

> 注意：**安全提醒**：初始口令只出现在你自己的终端里。请勿将口令粘贴到截图、问题报告、聊天记录或任何压缩包中。

### 6.2.2 登录页字段

| 字段 | 说明 |
| --- | --- |
| 用户名 | 管理员默认 owner；销售员账号由管理员在【人员与班次】发放 |
| 密码 | 区分大小写；连续失败会触发临时锁定（等待若干分钟后自动解锁） |
| 会话有效期 | 登录后 **12 小时**绝对过期；**30 分钟**无操作自动登出 |

### 6.2.3 销售员账号

管理员在【人员与班次】中为员工创建账号：一名员工档案**只能关联一个**登录账号；停用账号不会删除其历史单据。

---

## 6.3 界面总览与导航

### 6.3.1 页面清单

| 路径 | 页面 | 主要用途 | OWNER | SALESPERSON |
| --- | --- | --- | --- | --- |
| / | 运营工作台 | 今日概览与快捷入口 | √ | √ |
| /events | 活动与展位 | 场地、活动、展位维护；活动开放/关闭 | √ | √（查看） |
| /products | 商品 | SKU、成本、底价、建议价维护 | √ | √（查看） |
| /people | 人员与班次 | 员工档案、账号、排班；开班/收班 | √ | √（查看） |
| /stock | 库存 | 库位余额、收货、调拨、盘点、单据查询 | √ | √（查询与确认） |
| /sales | 销售工作台 | 建草稿、准备收款、确认成交、取消 | √ | √（仅本人班次） |

| 路径 | 页面 | 主要用途 | OWNER | SALESPERSON |
|---|---|---|---|---|
| `/returns` | 退货与核对 | 退货授权与过账 | √ | — |
| `/commissions` | 佣金 | 计提查询、结算准备、付款登记 | √ | √（仅本人） |
| `/reports` | 报表 | 按活动/商品/销售员汇总；CSV 导出 | √ | √（不含成本口径） |
| `/audit` | 操作日志 | 关键写入的追溯 | √ | — |

### 6.3.2 通用界面元素

| 元素 | 说明 |
|---|---|
| 顶部导航 | 按角色渲染；当前页面高亮 |
| 页面标题 + 说明 | 每页顶部给出该页用途的一句话说明 |
| 数据表格 | 目录类列表每页 20 条，销售列表每页 100 条，均支持上一页/下一页 |
| 选择器 | 商品/活动/员工等下拉最多载入 100 项；超过请使用搜索或筛选 |
| 错误提示 | 红色条幅显示 `message` 与 `requestId` |
| 加载状态 | 数据取回前显示加载指示，不展示过期数据 |

### 6.3.3 界面证据

桌面与 iPhone 13 视口的销售工作台截图保存在 `docs/evidence/`（E2E 测试自动生成，见 05 STD 第 5.7 节）。

---

## 6.4 核心操作流程

### 6.4.1 流程 A：开场准备

**目标**：让一个活动具备可销售条件（活动开放、展位就位、货品到位、班次开始）。

| 步骤 | 页面 | 操作 | 说明/校验 |
|---|---|---|---|
| A-1 | 【活动与展位】 | 录入场地（名称、街道、城市、省份、邮编） | 地址按字段分别录入，界面组合显示完整地址 |
| A-2 | 【活动与展位】 | 创建活动：选择场地与类别、填写名称与起止日期、税率（bps）、勾选政策确认 | 结束日期 ≥ 开始日期；税率 ≤ 10000 bps（即 100%） |
| A-3 | 【活动与展位】 | 为该活动至少建立一个展位 | **无展位的活动不能开放** |
| A-4 | 【活动与展位】 | 点击「开放活动」 | 状态变为 OPEN |
| A-5 | 【商品】 | 录入 SKU、名称、批发成本、最低成交价、建议价 | 建议价不得低于最低成交价 |

| 步骤 | 页面 | 操作 | 说明/校验 |
| --- | --- | --- | --- |
| A-6 | 【人员与班次】 | 建立员工档案（姓名、可选地址与电话、佣金率） | 佣金率 ≤ 10000 bps |
| A-7 | 【人员与班次】 | 为员工创建账号（用户名 + 初始密码） | 一名员工仅一个账号 |
| A-8 | 【人员与班次】 | 排班：选择员工、展位、业务日期与计划时段 | 时段须包含 UTC 偏移；业务日期按**多伦多时间**计算 |
| A-9 | 【库存】 | 收货：选择商品与数量，货物入**主仓** | 需填写理由 |
| A-10 | 【库存】 | 调拨：主仓 → 展位库位，填写数量与理由 | **实物交接完成后**再在系统中确认 |
| A-11 | 【人员与班次】 | 到班次时间后开班 | 销售员只能操作**本人**进行中的班次 |

> **可售量说明**：界面显示的可售量 = 在手量 − 已占用量。已被”准备收款”的单据占用的货品不计入可售。

### 6.4.2 流程 B：现场成交

**目标**：录入一笔销售并完成过账。

| 步骤 | 页面 | 操作 | 说明/校验 |
| --- | --- | --- | --- |
| B-1 | 【销售工作台】 | 点击「新建销售草稿」，选择进行中的班次 | 仅本人进行中的班次可选 |
| B-2 | 【销售工作台】 | 选择商品、数量，填写与顾客商定的 CAD 单价 | 单价可高于底价；低于底价会在下一步被拒 |
| B-3 | 【销售工作台】 | 保存草稿 | 状态 DRAFT；此时**不占用库存** |
| B-4 | 【销售工作台】 | 点击「准备收款」 | 系统重新核对：底价、活动状态、税率、佣金率、展位可售量；通过后**占用**商品（状态 CHECKOUT） |
| B-5 | **线下动作** | 按界面显示的固定应收额向顾客收款（现金或刷卡终端） | 系统**不会**发起任何银行卡扣款 |
| B-6 | 【销售工作台】 | 登记现金实收额或终端参考号，点击「确认成交」 | 库存 在手 与 占用 同量下降；生成库存流水、佣金计提、成交时间与审计记录 |
| B-7 | 【销售工作台】 | 使用”上一页/下一页”查询最近单据 | 每页 20 笔 |

> 注意：**响应不明时怎么办**：若点击「确认成交」后网络中断或结果未知，**先查询该单据状态**，确认未过账后再用**同一请求**重试。**不要**再次向顾客收款——重复收款在系统外发生，系统无法为你撤销。

**草稿处理规则**

| 状态 | 可否取消 | 说明 |
| --- | --- | --- |
| DRAFT | 可以 | 直接取消，释放无占用 |
| CHECKOUT | 可以，但需确认 | 必须先确认款项与商品去向后再取消，占用随之释放 |
| POSTED | **不可以** | 已成交单据只能通过【退货与核对】更正，不能删除 |

### 6.4.3 流程 C：退货与核对（仅管理员）

| 步骤 | 页面 | 操作 | 说明/校验 |
| --- | --- | --- | --- |
| C-1 | 【退货与核对】 | 选择原已成交单据 | 仅已过账单据可退 |
| C-2 | 【退货与核对】 | 选择退货行，填写退货数量与理由 | 同一退货单中，原单的每一行只能出现一次 |
| C-3 | 【退货与核对】 | 选择处置方式：RESTOCK（良品）或 DAMAGED（损坏） | 良品需指定回库库位；损坏品**不计入可售** |
| C-4 | 【退货与核对】 | 点击「授权」 | 系统按累计退货量分摊原单税额与佣金，显示退款额 |
| C-5 | **线下动作** | 按授权金额向顾客退款 | 系统不触发任何退款渠道 |
| C-6 | 【退货与核对】 | 登记退款参考号，点击「过账」 | 生成负向佣金计提；良品生成回库流水 |
| C-7 | — | 校验 | 累计退货数量**超过原销量**会被拒绝（409） |

### 6.4.4 流程 D：佣金结算（仅管理员）

| 步骤 | 页面 | 操作 | 说明 |
| --- | --- | --- | --- |
| D-1 | 【佣金】 | 选择销售员，查看其计提明细 | 销售为正、退货为负 |
| D-2 | 【佣金】 | 点击「准备结算」 | 系统**占用**该员工当前全部未结正负流水 |
| D-3 | 【佣金】 | 核对结算金额与明细 | 结算金额 = 明细合计 |
| D-4 | **线下动作** | 通过公司既有方式向员工付款 | 系统不执行转账 |
| D-5 | 【佣金】 | 登记付款参考号 | 状态 PREPARED → PAID |
| D-6 | 【佣金】 | 尚未付款时可「取消准备」 | 释放占用，流水回到未结状态 |

### 6.4.5 流程 E：报表与导出

| 步骤 | 页面 | 操作 | 说明 |
| --- | --- | --- | --- |
| E-1 | 【报表】 | 选择维度：活动 / 商品 / 销售员 | 维度为白名单，不接受任意字段 |

| 步骤 | 页面 | 操作 | 说明 |
| --- | --- | --- | --- |
| E-2 | 【报表】 | 查看汇总 | 口径：**已过账销售 − 已过账退货** |
| E-3 | 【报表】 | 点击「导出 CSV」 | 仅管理员可用；文件名 `foxcore-report.csv` |

> **毛利口径**：报表中的毛利为**商品标准成本估算**（成交价 − 成交时冻结的成本），**不含**活动费、差旅与工资，且销售员角色不可见。

### 6.4.6 流程 F：异常处置

| 步骤 | 操作 |
| --- | --- |
| F-1 | 发现余额异常时，**立即停止新的交易** |
| F-2 | 备份当前数据卷（见 07 部署运维） |
| F-3 | 执行 `npm run db:check` 排查 16 项不变量 |
| F-4 | 携带 `requestId` 与不变量输出定位问题；必要时从受控备份恢复 |

---

## 6.5 角色差异一览

| 能力 | OWNER | SALESPERSON |
| --- | --- | --- |
| 创建/修改场地、活动、展位、商品、员工 | √ | — |
| 创建销售员账号、启用/停用 | √ | — |
| 收货、调拨、盘点 | √ | 查询与确认 |
| 建单、准备收款、确认成交 | √（任意班次） | √（仅本人进行中的班次） |
| 退货授权与过账 | √ | — |
| 佣金结算准备与付款登记 | √ | 仅查看本人计提 |
| 报表查看 | √（含成本口径） | √（不含成本口径） |
| CSV 导出 | √ | — |
| 操作日志 | √ | — |

---

## 6.6 常见提示与处置

| 提示 | 含义 | 处置 |
| --- | --- | --- |
| 库存不足 / 已被占用 | 展位可售量不足 | 查看展位可售量与待收款单据；确认是否需要实物调拨 |

| 提示 | 含义 | 处置 |
| --- | --- | --- |
| 商品低于最低价 | 商定单价低于商品底价 | 提高单价，或由管理员调整商品底价后重新「准备收款」 |
| 班次不能开/关 | 活动未开放、时间不符或有未处理单据 | 检查活动状态、当前时间与该班次的待处理销售 |
| 幂等键冲突（409） | 同一个幂等键提交了不同内容 | 原键只用于重试**相同**请求；新动作应生成新键 |
| 记录已变化 | 服务端版本与本地不一致 | 刷新页面，重新核对版本、余额与金额后再操作 |
| 累计退货超过原销量 | 退货量超出原单 | 核对原单数量与历史退货，调整退货数量 |
| 403 权限不足 | 当前角色无权访问 | 确认是否使用了正确的账号；销售员不可访问管理接口 |
| 会话已过期 | 超过 12 小时或空闲 30 分钟 | 重新登录 |

## 6.7 操作禁忌清单

| 编号 | 禁止 | 原因 |
| --- | --- | --- |
| DON-01 | 不要删除已成交单据 | 历史不可篡改；应使用退货流程更正 |
| DON-02 | 不要用同一幂等键提交不同请求 | 会被拒绝（409），且造成排查困难 |
| DON-03 | 不要在响应不明时重复收款 | 系统无法撤销系统外的收款 |
| DON-04 | 不要在未实物交接前确认调拨 | 会造成账面与实物不符 |
| DON-05 | 不要执行 `docker compose down -v` | 会删除持久数据卷 |
| DON-06 | 不要把初始密码、备份文件放入截图或分发包 | 凭据泄露风险 |
| DON-07 | 不要在生产/演示库执行 `npm run seed:sample` | 会写入合成业务数据 |
| DON-08 | 不要手动修改已应用的迁移文件 | 校验和不一致将导致拒绝启动 |

## 附录：字段与格式说明

| 字段 | 格式 | 示例 | 说明 |
| --- | --- | --- | --- |
| 税率 / 佣金率 | bps（万分之一） | `1300` = 13% | 上限 10000 |
| 金额输入 | CAD 十进制字符串 | `12.34` | 系统内部以整数分存储 |
| 金额输出（API） | 整数分字符串 | `"1130"` | 界面按 CAD 格式化显示，即 11.30 CAD |

| 字段 | 格式 | 示例 | 说明 |
|---|---|---|---|
| 时间输入 | 含 UTC 偏移的 ISO 时间 | `2026-09-27T09:00:00.000-04:00` | 业务日期按多伦多时区计算 |
| 业务日期 | 日期 | `2026-09-27` | 班次归属日 |
| SKU | 字符串 | `INT-a1b2c3d4` | 全表唯一 |
| 幂等键 | UUID | `3f2a…` | 每次新业务动作生成一个 |
| 理由 / 备注 | 文本 | — | 库存单据理由**必填** |

> **金额示例**：成交单价 10.00 CAD（1000 分）、税率 1300 bps 时，税额 = 1000 × 13% = 130 分，合计 `1130` 分 = **11.30 CAD**（与 05 STD TC-009 一致）。界面一律按 CAD 格式化显示。

---

**文档结束。** 部署与运维见 07，测试与验收见 05 STD。

> 作业文档与图纸仓库：https://github.com/xingchenyd/foxcore-retail —— 文档位于 `docs/`，图纸位于 docs/diagrams（实体类图 class-diagram.png/svg、物理 ERD physical-erd.png、概念 ERD core-barker.png/svg、架构图、业务序列图、用例图、状态图等全套 .puml/.svg/.png）。

# 07 本地部署、运维与验收

**Foxcore Retail — Deployment, Operations & Acceptance Guide（本地形态）**

## 7.1 引言与前提

### 7.1.1 目的

本文档给出 Foxcore Retail **本地（或自有服务器）Docker 形态**的完整部署、首次验收、日常运维与故障处置规程。目标是”解压后一条命令可启动，启动后一份检查表可验收”。

### 7.1.2 前提条件

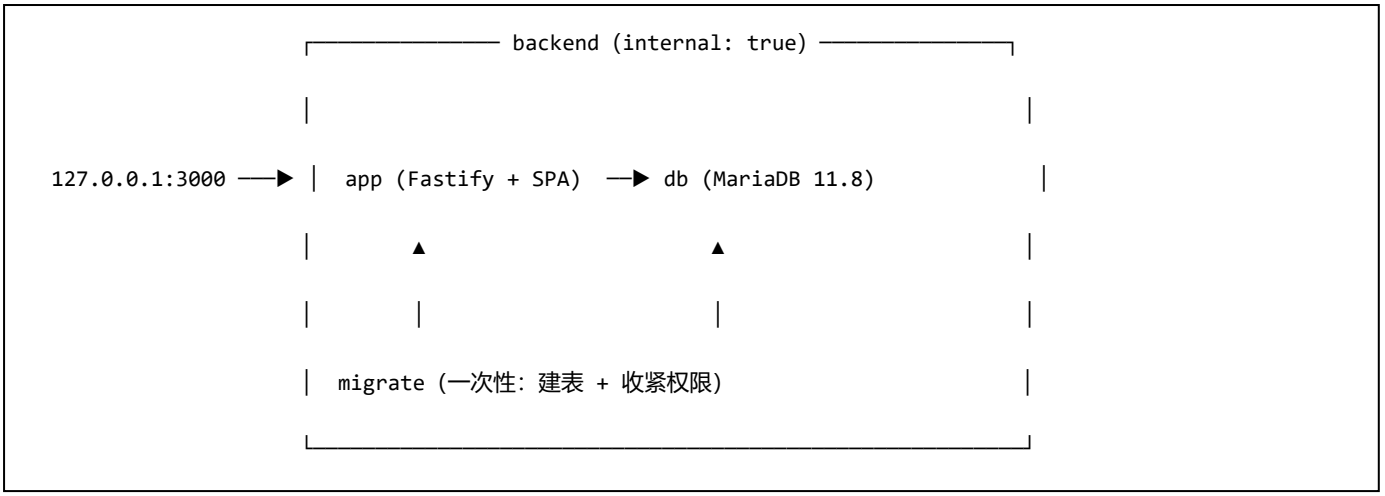
| 项 | 要求 |
|---|---|
| Docker | Docker Engine / Docker Desktop（含 Compose v2） |
| 内存 | 容器可用 ≥ 2 GB |
| 磁盘 | 镜像与数据库余量 ≈ 2 GB（随业务数据增长） |
| 网络 | 首次构建需访问镜像仓库与 npm registry |
| 端口 | 主机 `3000` 空闲（仅绑定 `127.0.0.1`） |
| 知识 | 具备基础命令行与 Docker Compose 操作能力 |

### 7.1.3 本文档不覆盖的内容

- 云端（Vercel + TiDB Cloud）部署 → 见 08；
- 业务操作步骤 → 见 06 操作手册；
- 测试用例与判定 → 见 05 STD。

## 7.2 服务拓扑与密钥设计

### 7.2.1 服务拓扑




| 服务 | 镜像 | 职责 | 网络 | 重启策略 |
| --- | --- | --- | --- | --- |
| db | foxcore-db:1.0.0（Dockerfile.db） | MariaDB 11.8 数据持久化；健康检查 mariadb-admin ping | 仅 backend（内部网络） | unless-stopped |
| migrate | foxcore-app:1.0.0 | 一次性：以管理员身份应用迁移，随后收紧运行账号权限 | backend | 退出码 0 即完成 |
| app | foxcore-app:1.0.0 | Fastify API + 静态资源；入口 runtime-owner.mjs && node apps/server/dist/index.js | backend + frontend | unless-stopped |

**网络隔离要点**：backend 网络声明 internal: true，数据库**不映射任何主机端口**；只有 app 通过 frontend 网络向主机发布 127.0.0.1:3000。

### 7.2.2 密钥设计（无需手工配置口令）

| Secret 卷 | 内容 | 挂载方 | 说明 |
| --- | --- | --- | --- |
| root_secret | 数据库 root 口令（随机生成） | db（读写）、migrate（只读） | **不挂载给 app** |
| app_secret | 应用数据库口令 + 首位管理员初始口令 | app | 应用只持受限账号口令 |
| db_data | 命名卷：数据库文件 | db | **持久**，普通 down 不删除 |

**最小权限原则**：运行账号仅对必要表拥有 UPDATE/DELETE；**库存流水、佣金流水、审计表不授 UPDATE/DELETE。**

### 7.2.3 镜像构建要点

Dockerfile 为**两阶段构建**：

1. **build 阶段**：node:24.15.0-bookworm-slim，npm ci → npm run build，产出 apps/server/dist 与 apps/web/dist；

2. **运行阶段**：`npm ci --omit=dev`（仅生产依赖）→ 从 build 阶段复制产物 → 创建 `/run/foxcore-app` 并以 `USER node` 运行 → 声明 `HEALTHCHECK` 轮询 `/health/ready`。

---

## 7.3 首次启动

### 7.3.1 启动步骤

解压发布包后，进入含 `compose.yaml` 的 `foxcore-retail` 目录：

```
docker compose up -d --build

docker compose ps
```

**等待三个条件全部满足**（首次构建需下载镜像，耗时较长）：

| 服务 | 期望状态 |
| --- | --- |
| db | healthy（健康检查每 5 s 一次，最多重试 30 次） |
| migrate | exited (0) —— 退出码必须为 0 |
| app | healthy（HEALTHCHECK 轮询 /health/ready） |

随后访问 http://localhost:3000。

### 7.3.2 获取管理员初始口令

```
docker compose exec app cat /run/foxcore-app/owner_initial_password
```

- 用户名：`owner`；
- 口令：随机生成（≥ 32 字符），**首次登录强制改密**，改密成功后该文件自动删除；
- 改密后旧会话失效，需用新密码重新登录。

### 7.3.3 停止与重启

```
docker compose down            # 停止并移除容器：不删除数据

docker compose up -d           # 重新启动
```

> 注意：**严禁 `docker compose down -v`**：`-v` 会一并移除 `db_data` 命名卷，导致**全部业务数据丢失。**这不是一种”修复”手段。

---

## 7.4 首次验收检查表

**用途**：首次启动必须逐项执行并签字确认（对应 05 STD 的 AT/TC 用例）。

```
docker compose config --quiet                 # 静态解析
docker compose ps                             # 三服务状态
docker compose logs --tail=100 migrate app    # 迁移与启动日志
curl http://localhost:3000/health/ready       # 就绪探针
```

| # | 检查项 | 判定标准 | 结论 |
|---|---|---|---|
| A-1 | Compose 静态解析 | `config --quiet` 退出码 0 | □ |
| A-2 | db 健康 | `healthy` | □ |
| A-3 | `migrate` 退出码 | `0` | □ |
| A-4 | app 健康 | `healthy` | □ |
| A-5 | 就绪探针 | 返回 `status: ready` 与迁移版本 | □ |
| A-6 | 首页可访问 | 浏览器打开 `http://localhost:3000` 显示登录页 | □ |
| A-7 | 管理员改密 | 取得临时口令 → 登录 → 改密 → 重新登录成功；旧会话失效 | □ |
| A-8 | 主数据链路 | 建场地 → 活动 → 展位 → 开放活动 → 建商品 → 建员工 → 排班 → 开班 | □ |
| A-9 | 库存链路 | 主仓收货 → 调拨至展位 → 确认交接 → 可售量正确 | □ |
| A-10 | 成交链路 | 建草稿 → 准备收款 → 线下收款 → 确认成交 → 单号唯一 | □ |
| A-11 | 幂等重试 | 同键重试返回相同响应；异体同键返回 409 | □ |
| A-12 | 并发抢库存 | 最后一件同时准备多单：恰 1 成功，其余 409；库存不为负 | □ |
| A-13 | 退货链路 | 部分退货成功；超额退货 409；原单保留 | □ |
| A-14 | 佣金链路 | 准备结算 → 登记付款；重复占用被拒 | □ |
| A-15 | 报表与导出 | 仅计已过账销售 − 已过账退货；CSV 限管理员 | □ |
| A-16 | 重启持久化 | `down` 后 `up -d`，单据仍在 | □ |
| A-17 | 不变量核查 | `npm run db:check` 16 项差异全为 0 | □ |

> **发布包默认为空业务库**，不会自动放入合成演示销售。若仅用于训练，可在**独立测试实例**手动执行 `npm run seed:sample`；这需要本地 Node 安装、登录凭据与正确的 API URL，**不可在生产经营库运行。**

---

# 7.5 配置与可迁移性

## 7.5.1 环境变量矩阵

| 变量 | 本地（compose 默认） | 说明 |
|---|---|---|
| APP_ENV | local | 环境标识 |
| NODE_ENV | production | 运行时模式 |
| HOST | 0.0.0.0 | 容器内监听地址（容器网络内可达即可） |
| PORT | 3000 | 监听端口 |
| PUBLIC_ORIGIN | http://localhost:3000 | **必须与浏览器实际同源地址一致** |
| COOKIE_SECURE | false | **HTTPS 下必须设为 true** |
| DB_HOST / DB_PORT | db / 3306 | 数据库地址 |
| DB_NAME / DB_USER | foxcore / foxcore | 库名与受限账号 |
| DB_PASSWORD_FILE | /run/foxcore-app/db_password | 应用口令（Secret 文件）；亦支持 DB_PASSWORD 环境变量 |
| OWNER_PASSWORD_FILE | /run/foxcore-app/owner_initial_password | 管理员初始口令文件 |
| DB_SSL | 未设置（本地） | 云端设为 true 开启 TLS（见 08） |

## 7.5.2 迁移到 HTTPS / 其他主机

1. 设置 PUBLIC_ORIGIN 为浏览器实际访问的同源地址（如 https://retail.example.com）；
2. 设置 COOKIE_SECURE=true；
3. 以安全方式注入 DB_PASSWORD_FILE 或 DB_PASSWORD——**不要把真实口令写进镜像、仓库或发布包**；
4. 对外访问需另行配置 TLS 终止、备份、访问控制与审计告警，这属于部署方职责；
5. 应用容器只需受限数据库用户；迁移使用独立管理员身份（保持最小权限）。

## 7.5.3 可迁移性保证

- Dockerfile / Dockerfile.db 为通用标准构建，不绑定特定云平台；
- .env.example 提供全部环境变量入口与注释；
- **云端适配以环境变量门控**（VERCEL、DB_SSL 等），本地 Compose 流程零改动（约束 DC-07 / ADR-09）。

---

# 7.6 日志、备份与恢复

## 7.6.1 日志

```
docker compose logs -f app

docker compose logs -f db

docker compose logs --tail=100 migrate
```

**日志脱敏**：Cookie、CSRF 令牌与口令字段已遮蔽。排查问题时使用响应中的 `requestId` 与服务端日志对应。

## 7.6.2 备份

**原则**

- 备份前安排**停写窗口**（避免跨事务的不一致快照）；
- 备份文件仅留在自己控制的本机磁盘，**加密并限制访问**；
- **不要**把备份文件放进分发压缩包、网盘或聊天附件。

**逻辑导出（Linux/macOS shell）**

```
docker compose exec -T db sh -c \

  'exec mariadb-dump --single-transaction --routines --triggers \

   -u root -p"$(cat /run/foxcore-root/password)" foxcore' > foxcore-backup.sql
```

> **Windows PowerShell 注意**：请使用二进制安全导出方式或在 `sh` 中运行上述脚本，避免 PowerShell 重定向改变编码导致 SQL 文件损坏。

## 7.6.3 恢复

**必须先在独立空卷验证，再制定正式恢复步骤：**

| 步骤 | 操作 | 校验 |
| --- | --- | --- |
| R-1 | 新建独立空实例（新 Compose 项目名 + 新卷） | `migrate` 退出 0 |
| R-2 | 导入备份 SQL | 无报错 |
| R-3 | 核对表数量 | 与 04 数据字典 一致（24 张 + `schema_migrations`） |
| R-4 | 核对单据数量与金额 | 与备份前一致 |
| R-5 | 执行 `db/check-invariants.sql` | 16 项差异全为 0 |
| R-6 | 验证通过后，才在正式环境执行恢复 | — |

## 7.7 日常运维检查表

| 频率 | 项目 | 命令/方式 |
| --- | --- | --- |
| 每日收档 | 不变量核查 | `npm run db:check`（差异应为 0） |
| 每日收档 | 检查失败登录与锁定账号 | 【操作日志】或 `audit_events` 查询 |
| 每周 | 逻辑备份并异地/离线保存 | §6.2 |
| 每周 | 检查磁盘与卷占用 | `docker system df -v` |
| 每月 | 恢复演练（独立空卷） | §6.3 |
| 每季度 | 依赖安全审计 | `npm audit` |
| 变更时 | 迁移前备份、迁移后核查 | §8 F-01 |

## 7.8 故障处置

| 编号 | 现象 | 可能原因 | 处置 |
| --- | --- | --- | --- |
| F-01 | `migrate` 失败 | 迁移文件校验和不一致 / SQL 错误 / 权限不足 | 查看 `docker compose logs migrate`：① 若提示校验和不一致，**恢复原迁移文件**或**新增编号迁移**，不得编辑已应用文件；② MariaDB 的 DDL 会**隐式提交**，迁移中途失败须备份后修复或从受控备份恢复，不可盲目反复改表 |
| F-02 | 端口 3000 占用 | 主机已有服务占用 | 释放端口后再启动（或修改 compose 端口映射，注意同步 `PUBLIC_ORIGIN`） |
| F-03 | app 无法连接数据库 | `db` 未 healthy / 卷权限异常 | `docker compose ps` 检查 `db` 状态；检查 `db_data` 卷权限与磁盘空间 |
| F-04 | 健康检查反复失败 | 启动期较长或依赖未就绪 | 观察 `start-period=30s` 内是否恢复；查看 `docker compose logs app` |
| F-05 | 登录后立刻掉线 | `PUBLIC_ORIGIN` 与实际地址不一致 / HTTPS 下 `COOKIE_SECURE=false` | 校正 `PUBLIC_ORIGIN`；HTTPS 环境设 `COOKIE_SECURE=true` |
| F-06 | 不变量非 0 | 数据异常或程序缺陷 | **停止新交易** → 备份当前卷 → 保留现场与 `requestId` → 依据 04 第 4.5 节 定位 |

| 编号 | 现象 | 可能原因 | 处置 |
| --- | --- | --- | --- |
| F-07 | 忘记管理员密码 | — | 无法在线找回（口令仅存 Argon2id 散列）；须通过数据库层面由运维重置并强制改密，操作前先备份 |
| F-08 | 数据被误删 | 执行了 `down -v` 或手工删除 | 从最近的受控备份恢复（§6.3） |

> **禁止将临时口令、备份文件或含口令的日志粘贴到工单、截图或分发包中。**

---

# 附录：命令速查

---

```
# 启动与状态

docker compose up -d --build

docker compose ps

docker compose logs -f app



# 管理员初始口令

docker compose exec app cat /run/foxcore-app/owner_initial_password



# 就绪探针

curl http://localhost:3000/health/ready



# 不变量核查（需本地 Node 与数据库可达）

npm run db:check



# 停止（保留数据卷）

docker compose down



# 备份

docker compose exec -T db sh -c \

  'exec mariadb-dump --single-transaction --routines --triggers \

   -u root -p"$(cat /run/foxcore-root/password)" foxcore' > foxcore-backup.sql
```

---

**文档结束。** 云端部署见 08；验收用例见 05 STD；数据库迁移细节见 04。

# 08 • 云端部署（Vercel + TiDB Cloud）

本文档描述如何把 Foxcore Retail 部署到 Vercel，获得一个所有人都能访问的公开链接。 **本地 Docker 部署（见 `07-deployment-local.md`）保持不变**：云端方案完全是增量式的， 所有代码改动均由环境变量门控（`VERCEL`、`DB_SSL`），不设置这些变量时本地行为与原版一致。

## 架构对照

| 层 | 本地部署 | 云端部署 |
|---|---|---|
| 前端 | Fastify 通过 `@fastify/static` 分发 `apps/web/dist` | Vercel 静态托管直接分发（全球 CDN） |
| API | 同一 Fastify 进程（端口 3000） | Vercel Serverless 函数（`api/index.js` → `apps/server/dist/index.js`） |
| 数据库 | Docker 容器内 MariaDB 11.8 | TiDB Cloud Starter（MySQL 协议兼容，TLS 连接） |
| 密码/密钥 | Docker 私有卷（`/run/foxcore-app`） | Vercel 环境变量 |
| 迁移 | `runtime-migrate.mjs`（root + GRANT） | `scripts/vercel-setup.mjs`（应用账号，无 GRANT，幂等） |

## 第 0 步 • 前置条件

- GitHub 账号（仓库 `foxcore-retail`，`vercel-deploy` 分支）
- Vercel 账号（免费 Hobby 档即可，用 GitHub 登录）
- TiDB Cloud 账号（免费 Starter 档，无需信用卡）

## 第 1 步 • 创建 TiDB Cloud 免费数据库

1. 打开 https://tidbcloud.com，注册/登录。
2. 创建组织后，选择 **Starter**（免费档：5 GiB 行存储 + 5 GiB 列存储 + 5000 万 RU/月）。
3. 创建集群：区域任选（建议新加坡或东京，离 Vercel 香港节点近），集群名如 `foxcore`。
4. 集群创建完成后，在集群页点击 **Connect**：
    - 选择 **Node.js** 连接示例；
    - 先按提示创建数据库密码（保存好）；
    - 记录连接参数：Host（形如 `gateway01.<region>.prod.aws.tidbcloud.com`）、Port（一般 `4000`）、User（形如 `<前缀>.root`）。
5. 在集群的 **SQL 控制台**执行 `CREATE DATABASE IF NOT EXISTS foxcore;`（TiDB 免费档每个集群自带一个 `test` 库，需另建业务库）。

## 第 2 步 • 在 Vercel 导入项目

1. 打开 https://vercel.com/new，找到 `xingchenyd/foxcore-retail` 仓库，**Import**。
2. Framework Preset 选择 **Other**（我们用 vercel.json 显式配置）。
3. 在 **Environment Variables** 中添加以下变量（全部环境：Production + Preview）：

| 变量名 | 值 | 说明 |
|---|---|---|
| DB_HOST | TiDB 的 Host | 第 1 步第 4 项 |
| DB_PORT | 4000 | TiDB 默认端口 |
| DB_USER | TiDB 用户名（含前缀） | 如 xxxxxxxx.root |
| DB_PASSWORD | TiDB 数据库密码 | 与 DB_NAME 同级的明文密码 |
| DB_NAME | foxcore | 第 1 步第 5 项创建的库 |
| DB_SSL | true | TiDB 强制 TLS，必填 |
| APP_ENV | production | 跳过本地 .env 加载 |
| NODE_ENV | production | |
| ALLOW_PRODUCTION_MIGRATION | yes | 允许构建期运行初始化脚本 |
| OWNER_PASSWORD | ≥32 字符随机串 | 首次部署创建 owner 用；登录后会被强制修改 |
| COOKIE_SECURE | true | HTTPS 下必须 |
| PUBLIC_ORIGIN | https://<你的域名>.vercel.app | **首次部署后回填**，见第 4 步 |

4. 点击 **Deploy**。构建过程会自动：安装依赖 → 构建前端/后端 → 运行 `scripts/vercel-setup.mjs`（建表 + 初始化 owner）。

## 第 3 步 • 验证

1. 打开分配的 `https://<域名>.vercel.app`，应看到登录页。
2. 访问 `https://<域名>.vercel.app/health/ready`，应返回 `{"status":"ready","schema":"001_core.sql"}`。
3. 用 `owner` + `OWNER_PASSWORD` 登录，系统会强制修改初始密码；修改后即可正常使用。

## 第 4 步 • 回填 PUBLIC_ORIGIN 并重新部署

首次部署完成后，把 Vercel 分配的域名（或后续绑定的自定义域名）填入环境变量 `PUBLIC_ORIGIN`（协议、域名、端口需完全一致，不要带尾部斜杠），然后在 Vercel 项目页 **Deployments → Redeploy**。 该变量用于登录接口的 Origin 校验，不回填会导致登录被拒（403）。

## 与本地部署的差异说明

- **权限模型**：本地迁移脚本用 root 给应用账号发 GRANT；TiDB 云端不允许（也不需要）， `vercel-setup.mjs` 直接用应用账号建表。

- **静态资源**：云端由 Vercel CDN 分发，Serverless 函数内不再注册 `@fastify/static`（`apps/web/dist` 不在函数包内，`index.ts` 的 `existsSync` 检查会自动跳过）。
- **冷启动**：Serverless 空闲后首个请求有 1–2 秒冷启动延迟，属正常现象。
- **数据持久性**：数据存于 TiDB Cloud，与 Vercel 部署解耦；重新部署不丢数据。

## 免费额度与限制

- **Vercel Hobby**：免费，仅限非商业用途；函数执行时长本项目配置为 30s（够用）。
- **TiDB Cloud Starter**：5 GiB 存储 + 5000 万 RU/月，教学演示规模远用不完；超量后拒绝新连接（不会产生账单）。

## 故障处理

| 现象 | 原因与处理 |
| --- | --- |
| 构建日志报 `Migration checksum changed` | 远端已应用过同版本号但内容不同的迁移：不得修改 `db/migrations` 中已发布的 SQL，应新增版本文件 |
| 登录返回 403 `FORBIDDEN` | `PUBLIC_ORIGIN` 与实际访问域名不一致（含协议/端口），回填后 Redeploy |
| 登录返回连接超时 | 检查 `DB_HOST/DB_PORT/DB_SSL`，TiDB 必须走 4000 端口 + TLS |
| 函数日志报 `Missing DB_HOST` | Vercel 环境变量未保存到当前环境（Production/Preview 勾选） |
| argon2 模块加载失败 | 极少见：Vercel 运行时区域与预编译二进制不匹配时，在项目设置中把 Node 版本固定为 24.x 后重试 |